

# **Computer Networks Lecture Notes**

Son Dang Huu Trung

Lecturer: Prof. Dr. Oliver Hahm

Frankfurt am Main, January 4, 2026

# Contents

|          |                                                                    |          |
|----------|--------------------------------------------------------------------|----------|
| <b>I</b> | <b>Lecture Notes</b>                                               | <b>1</b> |
| <b>1</b> | <b>Basics</b>                                                      | <b>2</b> |
| 1.1      | Historical background . . . . .                                    | 2        |
| 1.2      | Components and Terms . . . . .                                     | 2        |
| 1.2.1    | Purpose of Computer Networks . . . . .                             | 2        |
| 1.2.2    | Requirement Components to set up a Computer Network . . . . .      | 2        |
| 1.2.3    | Network Services . . . . .                                         | 3        |
| 1.2.4    | Transmission Media . . . . .                                       | 3        |
| 1.2.5    | Protocols . . . . .                                                | 4        |
| 1.2.6    | Computer Networks Distinguished by their Dimension . . . . .       | 4        |
| 1.2.6.1  | Local Area Network (LAN) . . . . .                                 | 4        |
| 1.2.6.2  | Metropolitan Area Network (MAN) . . . . .                          | 4        |
| 1.2.6.3  | Wide Area Network (WAN) . . . . .                                  | 5        |
| 1.2.7    | Unicast and Broadcast . . . . .                                    | 5        |
| 1.2.7.1  | Unicast . . . . .                                                  | 5        |
| 1.2.7.2  | Broadcast . . . . .                                                | 5        |
| 1.2.8    | Group Communication: Multicast and Anycast . . . . .               | 6        |
| 1.2.8.1  | Multicast . . . . .                                                | 6        |
| 1.2.8.2  | Anycast . . . . .                                                  | 6        |
| 1.2.9    | Connection-Orientation . . . . .                                   | 7        |
| 1.2.9.1  | Connection-Oriented . . . . .                                      | 7        |
| 1.2.9.2  | Connectionless . . . . .                                           | 7        |
| 1.2.10   | Directional Dependence (Anisotropy) of Data Transmission . . . . . | 7        |
| 1.2.11   | Bandwidth, Throughput and Goodput . . . . .                        | 8        |
| 1.2.12   | Latency . . . . .                                                  | 8        |
| 1.3      | Reference Models . . . . .                                         | 9        |
| 1.3.1    | "Philosopher-Translator-Secretary" Architecture . . . . .          | 10       |
| 1.3.2    | TCP/IP Reference Model or DoD Model . . . . .                      | 10       |
| 1.3.3    | TCP/IP Model - Message Structure . . . . .                         | 11       |
| 1.3.4    | OSI Reference Model . . . . .                                      | 12       |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 1.3.5    | OSI Model Concepts . . . . .                 | 13        |
| 1.3.6    | Physical Layer . . . . .                     | 13        |
| 1.3.7    | Data Link Layer . . . . .                    | 14        |
| 1.3.8    | Network Layer . . . . .                      | 14        |
| 1.3.9    | Transport Layer . . . . .                    | 15        |
| 1.3.10   | Session Layer . . . . .                      | 16        |
| 1.3.11   | Presentation Layer . . . . .                 | 16        |
| 1.3.12   | Application Layer . . . . .                  | 17        |
| 1.4      | Topologies . . . . .                         | 18        |
| 1.4.1    | Topologies of Computer Networks . . . . .    | 18        |
| 1.4.2    | Bus Network . . . . .                        | 18        |
| 1.4.3    | Ring Network . . . . .                       | 19        |
| 1.4.4    | Star Network . . . . .                       | 19        |
| 1.4.5    | Mesh Network . . . . .                       | 20        |
| 1.4.6    | Tree Network . . . . .                       | 20        |
| 1.4.7    | Cellular Network . . . . .                   | 21        |
| 1.4.8    | Current Situation . . . . .                  | 21        |
| <b>2</b> | <b>Physical Layer</b>                        | <b>22</b> |
| 2.1      | Fundamentals of Data Signals . . . . .       | 22        |
| 2.1.1    | Physical Representation of Data . . . . .    | 22        |
| 2.1.2    | The Telephone Example . . . . .              | 23        |
| 2.1.3    | Basics of Signal Processing . . . . .        | 23        |
| 2.1.4    | Quantization and Sampling . . . . .          | 24        |
| 2.1.5    | Sampling, Quantization, and Coding . . . . . | 24        |
| 2.1.6    | Symbol Rate . . . . .                        | 25        |
| 2.1.7    | Bit Rate and Symbol Rate . . . . .           | 26        |
| 2.1.8    | Data Rate . . . . .                          | 27        |
| 2.1.9    | Ideal vs. Real Transmission . . . . .        | 28        |
| 2.1.10   | Attenuation . . . . .                        | 28        |
| 2.1.11   | Noise and Distortion . . . . .               | 29        |
| 2.1.12   | Bit Error Rate (BER) . . . . .               | 30        |
| 2.1.13   | Data Rate on a Noisy Channel . . . . .       | 31        |
| 2.2      | Data Encoding . . . . .                      | 32        |
| 2.2.1    | Line Encoding . . . . .                      | 32        |
| 2.2.1.1  | Baseband and Broadband (part 1) . . . . .    | 32        |
| 2.2.1.2  | Encoding Schemes . . . . .                   | 32        |
| 2.2.1.3  | Non-Return-to-Zero (NRZ) . . . . .           | 33        |
| 2.2.1.4  | Manchester Code . . . . .                    | 33        |
| 2.2.2    | Modulation . . . . .                         | 34        |
| 2.2.2.1  | Baseband and Broadband (part 2) . . . . .    | 34        |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 2.2.2.2  | Principle of Modulation . . . . .                                  | 35        |
| 2.2.2.3  | Amplitude Shift Keying (ASK) . . . . .                             | 36        |
| 2.2.2.4  | Frequency Shift Keying (FSK) . . . . .                             | 37        |
| 2.2.2.5  | Phase Shift Keying (PSK) . . . . .                                 | 38        |
| 2.2.2.6  | Overview . . . . .                                                 | 39        |
| 2.3      | Transmission Media . . . . .                                       | 40        |
| 2.3.1    | Classification of Transmission Media . . . . .                     | 40        |
| 2.3.2    | Guided Transmission Media . . . . .                                | 41        |
| 2.3.2.1  | Coaxial Cables (Coax Cables) . . . . .                             | 41        |
| 2.3.2.2  | Twisted Pair Cables . . . . .                                      | 42        |
| 2.3.2.3  | Shielding of different Twisted Pair Cables . . . . .               | 43        |
| 2.3.2.4  | Categories of Twisted Pair Cables . . . . .                        | 44        |
| 2.3.2.5  | Fiber-optic Cables . . . . .                                       | 45        |
| 2.3.3    | Unguided Transmission Media . . . . .                              | 46        |
| 2.3.3.1  | Wireless Communication . . . . .                                   | 46        |
| 2.3.3.2  | Challenges of Wireless Networks . . . . .                          | 47        |
| 2.3.4    | The Last Mile . . . . .                                            | 49        |
| 2.3.4.1  | Bridging the Last Mile . . . . .                                   | 49        |
| 2.3.4.2  | History: The Classical Modem . . . . .                             | 50        |
| 2.3.4.3  | Digital Subscriber Line (DSL) . . . . .                            | 50        |
| 2.3.4.4  | Data Over Cable Service Interface Specification (DOCSIS) . . . . . | 51        |
| 2.3.4.5  | 3GPP Standards . . . . .                                           | 52        |
| 2.4      | Technologies . . . . .                                             | 52        |
| 2.4.1    | Standardization . . . . .                                          | 52        |
| 2.4.2    | Ethernet . . . . .                                                 | 53        |
| 2.4.2.1  | Ethernet (IEEE 802.3) . . . . .                                    | 53        |
| 2.4.2.2  | The Evolution of Ethernet . . . . .                                | 53        |
| 2.4.3    | Wireless Local Area Network (WLAN) . . . . .                       | 54        |
| 2.4.3.1  | WLAN (IEEE 802.11) . . . . .                                       | 54        |
| 2.4.3.2  | The Evolution of WLAN standards . . . . .                          | 54        |
| 2.4.3.3  | Non-overlapping Channels of IEEE 802.11b . . . . .                 | 55        |
| 2.4.3.4  | WLAN Security: WEP and WPA . . . . .                               | 56        |
| <b>3</b> | <b>Data Link Layer - Framing and Switching</b>                     | <b>58</b> |
| 3.1      | Framing . . . . .                                                  | 58        |
| 3.1.1    | Frame Detection . . . . .                                          | 58        |
| 3.1.1.1  | Character Count in the Frame Header . . . . .                      | 59        |
| 3.1.1.2  | Control Sequences . . . . .                                        | 60        |
| 3.1.1.3  | Byte/Character stuffing . . . . .                                  | 60        |
| 3.1.1.4  | Example: Byte/Character stuffing (BISYNC) . . . . .                | 60        |
| 3.1.1.5  | Bit Stuffing . . . . .                                             | 62        |



|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 3.1.1.6  | Example: Bit Stuffing (HDLC)                                     | 62        |
| 3.1.1.7  | Line Code Violation                                              | 63        |
| 3.1.2    | Ethernet (IEEE 802.3) Frames                                     | 64        |
| 3.1.2.1  | Preamble and SFD                                                 | 64        |
| 3.1.2.2  | Addresses and VLAN Tag                                           | 65        |
| 3.1.2.3  | Frame Size and Checksum                                          | 67        |
| 3.1.2.4  | Interframe Spacing                                               | 68        |
| 3.2      | Addresses                                                        | 69        |
| 3.2.1    | MAC Address                                                      | 69        |
| 3.2.2    | Broadcast and Multicast MAC Address                              | 70        |
| 3.2.3    | Uniqueness of MAC Addresses                                      | 70        |
| 3.2.4    | Security Aspects of MAC Addresses                                | 72        |
| 3.3      | Switching                                                        | 72        |
| 3.3.1    | Devices                                                          | 72        |
| 3.3.1.1  | Devices of the Data Link Layer: Bridges                          | 72        |
| 3.3.1.2  | Example: WLAN Bridge                                             | 73        |
| 3.3.2    | Forwarding                                                       | 74        |
| 3.3.2.1  | Learning Bridges                                                 | 74        |
| 3.3.2.2  | Forwarding Strategies                                            | 75        |
| 3.3.3    | Loops                                                            | 76        |
| 3.3.3.1  | Loops on the Data Link Layer                                     | 76        |
| 3.3.3.2  | Example of Loops in a LAN                                        | 77        |
| 3.3.3.3  | Handle Loops in the LAN                                          | 79        |
| 3.3.3.4  | Spanning Tree Protocol                                           | 79        |
| 3.3.3.5  | Spanning Tree Protocol - Bridge ID                               | 80        |
| 3.3.3.6  | Spanning Tree Protocol - Cisco Bridge IDs                        | 81        |
| 3.3.3.7  | Spanning Tree Protocol - Functioning                             | 81        |
| <b>4</b> | <b>Data Link Layer - Medium Access Control and Error Control</b> | <b>84</b> |
| 4.0.1    | Coordinated Medium Access                                        | 84        |
| 4.1      | Contention-based                                                 | 85        |
| 4.1.1    | ALOHA                                                            | 85        |
| 4.1.1.1  | ALOHA                                                            | 85        |
| 4.1.1.2  | Slotted ALOHA                                                    | 86        |
| 4.1.1.3  | Performance of ALOHA                                             | 87        |
| 4.1.2    | CSMA                                                             | 88        |
| 4.1.2.1  | CSMA                                                             | 88        |
| 4.1.2.2  | Performance of CSMA                                              | 89        |
| 4.1.3    | CSMA/CA and MACA                                                 | 90        |
| 4.1.3.1  | CSMA/CA                                                          | 90        |
| 4.1.3.2  | Functioning of CSMA/CA                                           | 91        |

|     |         |                                                           |     |
|-----|---------|-----------------------------------------------------------|-----|
|     | 4.1.3.3 | MACA (Multiple Access with Collision Avoidance) . . . . . | 92  |
|     | 4.1.3.4 | Functioning of MACA . . . . .                             | 93  |
| 4.2 |         | Contention-free . . . . .                                 | 94  |
|     | 4.2.1   | Token Passing . . . . .                                   | 94  |
|     | 4.2.2   | TDMA . . . . .                                            | 95  |
|     | 4.2.3   | FDMA . . . . .                                            | 96  |
|     | 4.2.3.1 | Frequency Division Multiple Access (FDMA) . . . . .       | 96  |
|     | 4.2.3.2 | Example: IEEE 802.15.4e . . . . .                         | 97  |
|     | 4.2.4   | CDMA . . . . .                                            | 98  |
| 4.3 |         | Error Control . . . . .                                   | 99  |
|     | 4.3.1   | Failure Causes . . . . .                                  | 99  |
|     | 4.3.2   | Error Detection . . . . .                                 | 100 |
|     | 4.3.2.1 | Checksum . . . . .                                        | 100 |
|     | 4.3.2.2 | Hamming Distance . . . . .                                | 100 |
|     | 4.3.2.3 | One-dimensional Parity-check Code . . . . .               | 101 |
|     | 4.3.2.4 | Two-dimensional Parity-check Code . . . . .               | 102 |
|     | 4.3.2.5 | Cyclic Redundancy Check (CRC) . . . . .                   | 102 |
|     | 4.3.2.6 | CRC Generator Polynomial . . . . .                        | 103 |
|     | 4.3.2.7 | Selection of common Generator Polynomials . . . . .       | 103 |
|     | 4.3.2.8 | CRC Example . . . . .                                     | 103 |
|     | 4.3.2.9 | Properties of CRCs . . . . .                              | 104 |
|     | 4.3.3   | Error Correction . . . . .                                | 105 |
|     | 4.3.3.1 | Forward Error Correction (FEC) . . . . .                  | 105 |
|     | 4.3.3.2 | Simple Example of Error Correction . . . . .              | 106 |

## **Part I**

# **Lecture Notes**

# Chapter 1

## Basics

### 1.1 Historical background

(Tự đọc)

### 1.2 Components and Terms

#### 1.2.1 Purpose of Computer Networks

General task: enable communication among participants.

- Resource sharing: assign different tasks to different computers → Chia sẻ tài nguyên để tránh bị nghẽn.
- Resource pooling: make multiple small computers act like one large powerful system → Gộp tài nguyên thành một cụm để chạy tốt hơn.
- Resource balancing: increase the availability of the service by redundancy → thay vì 1 máy chạy thì nó sẽ duplicate ra cho nhiều máy  $\Rightarrow$  khi có lỗi thì còn máy khác để chạy.

#### 1.2.2 Requirement Components to set up a Computer Network

Phải có 3 yếu tố chính:

1. Computers  $\rightarrow$  phải có  $\geq 2$  máy tính trở lên.
2. Transmission medium  $\rightarrow$  phải có cho nhận (send/receive); có 2 hình thức: **có dây** (wire) và **không dây** (wireless).
3. Network protocols  $\rightarrow$  phải có cách giao thức, như là luật để tụi nó giao tiếp với nhau.

### 1.2.3 Network Services

Network service → cung cấp tài nguyên cho các thiết bị khác trong cái network đó.

Distinguished by their role:

- Server (máy chủ) → hold secures (file, database, etc.) and wait for requests from others ⇒ provide a network service.
- Client (máy khách) → send a request to the server and waits for the response ⇒ uses (consumes) a network service.

Peer-to-peer networks (mạng ngang hàng) → every computer can download files from others (client role) and upload files to other (server role) at the same time.

⇒ Nói nôm na là, mạng peer-to-peer là là tụi nó cứ liên tục swap role cho nhau, tụi nó ở ngang hàng nhau, không thằng nào hơn role thằng nào cả.

⇒ Chú ý: các thuật ngữ như "server", "client", "peer" chỉ dùng cho network và không đưng đến phần cứng.

### 1.2.4 Transmission Media

Có 2 loại truyền:

- Guided transmission media (là truyền bằng dây).
- Wireless transmission.

Wireless transmission có 2 cách là **directed** và **undirected**:

- Directed (có hướng) → must aim hardware physically.
- Undirected (đa hướng) → no aiming needed.

**Bonus:** In security problem:

- Directed → harder to intercept.
- Undirected → easier to intercept.

Directed transmission technologies:

- Radio technology.
- Infrared.
- Laser.

Undirected wireless transmission → mostly based on radio technology (WLAN, cellular networks, etc.).

### 1.2.5 Protocols

Protocol → set of all previously made agreements between communication partners:

- Connection establishment & termination.
- Synchronization.
- Error detection.
- Vocabulary → valid messages.
- Encoding.

⇒ Specifies the syntax (message valid format) and semantics (vocabulary and meaning of valid messages).

Self-note: Tại sao lại phải cần check coi có đúng "semantic" (nghĩa là xem coi nó nghe có hợp lý) không?

→ Thử tưởng tượng ha, câu "tôi ăn đá" vẫn đúng cấu trúc ngữ pháp, tuy nhiên là trên thực tế câu này nghe rất vô lý ⇒ chính vì thế nên mới phải check semantics.

### 1.2.6 Computer Networks Distinguished by their Dimension

Self-note:

- PAN (Personal Area Network) → for personal use within a user's intermediate workspace.
- BAN (Body Area Network) → specifically centered around the human body.

⇒ Nói tóm lại: đây là network có vùng phủ nhỏ (few meters), thường có trong các điện thoại thông minh.

#### 1.2.6.1 Local Area Network (LAN)

- Local network.
- Range: apartment, building, company site, or university campus.
- Dimension: 500-1000 m.

#### 1.2.6.2 Metropolitan Area Network (MAN)

- Connects LANs.
- Range: a city, or agglomeration area (khu có nhiều dân cư).
- Dimension: 100 km.

### 1.2.6.3 Wide Area Network (WAN)

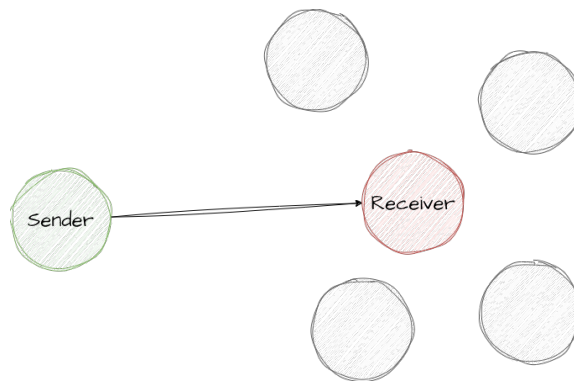
- Connects several networks.
- Range: large geographic inside a country or continent.
- Dimension: 1000 km.

So sánh về độ phủ sóng: LAN < MAN < WAN.

## 1.2.7 Unicast and Broadcast

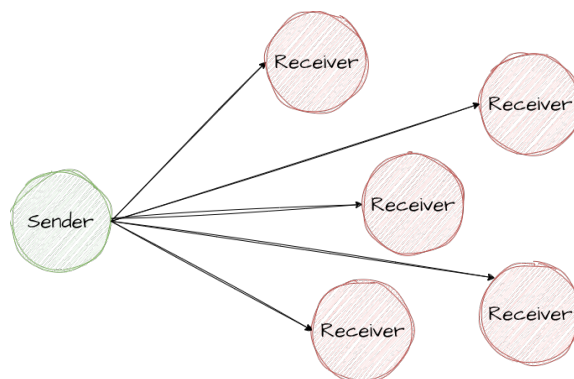
### 1.2.7.1 Unicast

One-to-one communication.



### 1.2.7.2 Broadcast

One-to-all communication.

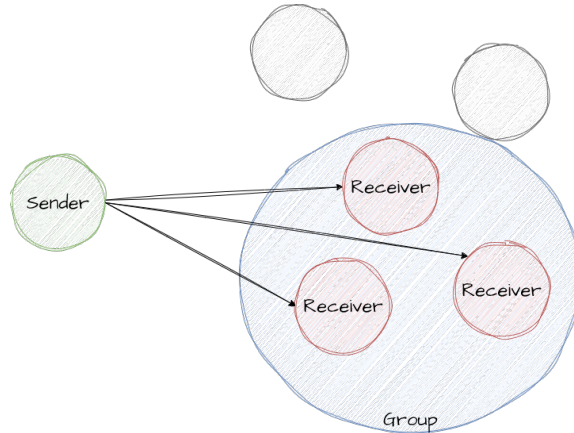


### 1.2.8 Group Communication: Multicast and Anycast

Self-note: Multicast và anycast ở đây chỉ gửi cho nhóm, nhưng multicast thì gửi mọi người trong nhóm, còn anycast thì chỉ gửi cho một người bất kỳ trong nhóm thôi.

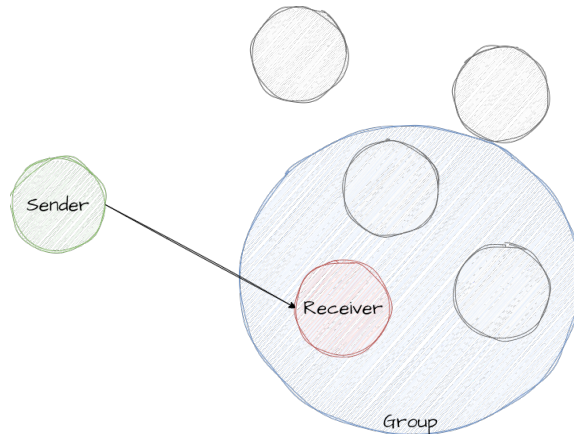
#### 1.2.8.1 Multicast

One-to-group communication (group communication).



#### 1.2.8.2 Anycast

One-to-any communication.



Self-note: Unicast và broadcast gửi cho từng người một, khác với multicast và anycast là gửi cho nhóm → khác nhau giữa gửi không trong nhóm và gửi trong nhóm.



## 1.2.9 Connection-Orientation

Có 2 phương thức: **connection-oriented** (có kết nối) và **connectionless** (phi kết nối).

### 1.2.9.1 Connection-Oriented

→ The service operates **stateful** (có trạng thái).

- Three phases: connection establishment, data transfer, connection termination.
- Virtual path between the involved hosts is established.
- Sequent data is exchanged between the same hosts.

→ Server, hoặc thiết bị mạng, lưu trữ thông tin về các session hoặc connection đang diễn ra ⇒ cung cấp khả năng lọc & quản lý connection thông minh hơn.

### 1.2.9.2 Connectionless

→ The service operates **stateless** (không trạng thái).

- No path between the involved hosts is established.

→ Xử lý từng yêu cầu một cách độc lập mà không cần ghi nhớ các yêu cầu trước đó ⇒ đơn giản và nhanh hơn do không có overhead để theo dõi.

Self-note: Overhead nghĩa là phần tốn thêm để được truyền đi đúng cách, là mấy cái như header, trailer của gói tin chẳng hạn.

## 1.2.10 Directional Dependence (Anisotropy) of Data Transmission

Self-note: Anisotropy nghĩa là tính định hướng.

Directional có 3 chế độ truyền (communication modes): **simplex**, **half-duplex**, và **full-duplex**.

A communication channel with two (or more) endpoints:

- Simplex (đơn công) → chỉ có một bên có thể gửi tín hiệu.
- Full-duplex (song công toàn phần) → cả hai bên đều có thể gửi và nhận cùng lúc (simultaneously).
- Half-duplex (bán song công) → mỗi bên chỉ có thể gửi và nhận nhưng không cùng thời điểm với nhau được.

Self-note: Endpoint có nghĩa là điểm giao tiếp.

### 1.2.11 Bandwidth, Throughput and Goodput

Các yếu tố ảnh hưởng trong network:

- Bandwidth (băng thông) → throughput.
- Latency (độ trễ).

#### Mở rộng định nghĩa:

- Bandwidth: băng thông → là khả năng mà đường truyền mạng có thể truyền dữ liệu trong 1 giây.
- Throughput: thông lượng → là tốc độ thực tế được truyền qua mạng trong một khoảng thời gian, bao gồm tổng số bit đã truyền, kể cả những bit truyền không thành công.
- Goodput: thông lượng hữu ích → đo lường tốc độ dữ liệu hữu ích mà một ứng dụng nhận được trên mạng, remove phần dữ liệu dư thừa (actual achieved data rate); chẳng hạn như các bit dư thừa, hoặc các gói bị mất hay truyền lại.

Self-note: goodput  $\leq$  throughput.

### 1.2.12 Latency

Latency (độ trễ) → là khoảng thời gian một gói tin mất để đi từ nơi gửi (sender) đến nơi nhận (receiver).

#### Công thức:

$$\text{Latency} = \text{Propagation delay} + \text{Transmission delay} + \text{Waiting time}$$

$$\text{Propagation delay} = \frac{\text{Distance}}{\text{Speed of light} * \text{Velocity factor}}$$

Chú giải:

- Distance: khoảng cách giữa sender và receiver trong cái network connection.
- Speed of light: tốc độ ánh sáng.
- Velocity factor: hệ số vận tốc → nó là hệ số cụ thể, ra thì đề sẽ cho; ví dụ: Vacuum (môi trường chân không) là 1.

$$\text{Transmission delay} = \frac{\text{Message size}}{\text{Bandwidth}}$$

**Lưu ý:**

- Nếu message chỉ có single bit → transmission delay = 0.
- Nó cần phải cache receive data (nhận dữ liệu về cache để lưu trữ) trước khi forward.
- Waiting time → bị gây ra do network devices (chẳng hạn như Switch).
- Nếu network connection giữa điểm gửi và điểm nhận chỉ là một đường thẳng hoặc một kênh truyền duy nhất → waiting time = 0.

### 1.3 Reference Models

Self-note: Reference model nghĩa là mô hình tham chiếu

Reference models → mô tả cách hoạt động của computer network một cách tổng quát, không phụ thuộc vào công nghệ cụ thể (như hardware hay software).

**Cách thức hoạt động:**

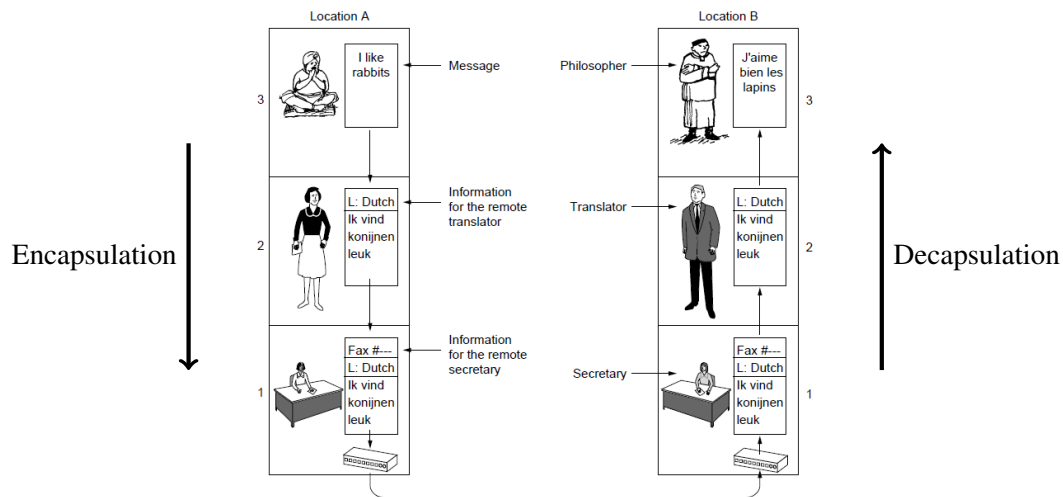
- Mô tả network độc lập với công nghệ (mấy cái hardware/software) → nói lên tầng này làm những công việc gì (không quan tâm đến việc sử dụng ngôn ngữ lập trình gì, etc.) ⇒ có thể quy được cùng mô hình tham chiếu để communicate với nhau.
- Decomposition (chia nhỏ vấn đề khó) → mô hình chia thành các phần nhỏ gọi là tầng (layers) ⇒ mỗi tầng chỉ giải quyết một vấn đề duy nhất, tự xử lý dữ liệu của mình, và đóng gói (encapsulation) trước khi move đến các tầng khác.
- Modularity (tính linh hoạt và thay thế được) → có thể sửa đổi hoặc thay thế protocol ở một tầng mà không bị hư toàn bộ system.
- Interfaces & Services (giao diện & dịch vụ) → quy định các tầng rõ ràng để giao tiếp như tầng dưới sẽ cung cấp gì cho tầng trên sử dụng và ngược lại.

⇒ Tạo ra một tiêu chuẩn chung để có thể connect và thay đổi linh hoạt.

2 reference models phổ biến:

- TCP/IP reference model → có 4 layers.
- ISO/OSI reference model → có 7 layers.

### 1.3.1 "Philosopher-Translator-Secretary" Architecture



→ Ví dụ này mô tả cách giao tiếp bằng việc lấy tiếng Hà Lan làm protocol để đóng gói và đưa thông tin ⇒ Layered Architecture.

### 1.3.2 TCP/IP Reference Model or DoD Model

Self-note: IP nghĩa là Internet Protocol.

For each layer, it is specified, what functionality it provides.

TCP: Transmission Control Protocol → like Certified Mail: guarantees that the recipient gets the package, and send receipt back ⇒ if lost, it will re-send.

syn → syn-ack → ack received

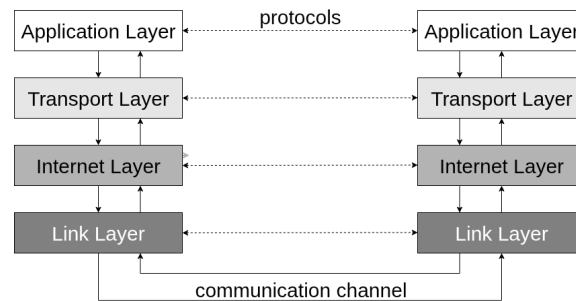
UDP: User Datagram Protocol → like sending post card: drop in mailbox and hope it arrives ⇒ faster and cheaper, but no confirmation receipt, and may lost.

**Has 4 layers:**

- Application Layer → contains protocols that interact directly with programs ⇒ format the actual "payload".
- Transport Layer → connects applications on different devices ⇒ chops data into manageable pieces called segments.

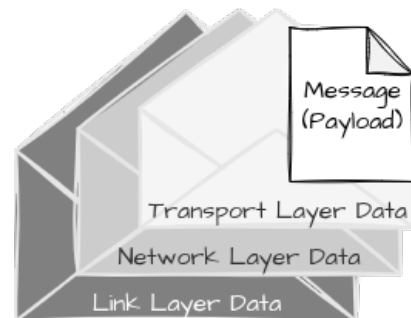
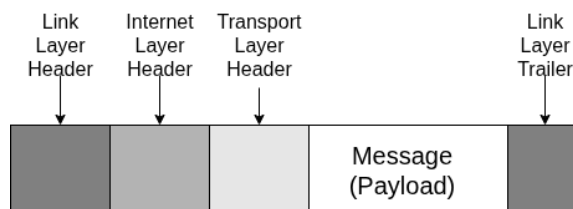
- Internet Layer → handles logical addressing and routing; pack the segments into packets.
- Link Layer → handles physical transmission errors and control access to the wire/air; uses physical address (MAC address).

### 1.3.3 TCP/IP Model - Message Structure



Each layer adds additional information as header to the message.

- Có một số protocol có link layer không chỉ có header mà còn có trailer phía cuối message; ví dụ như Ethernet.
- Receiver phân tích header (và trailer nếu có) trên cùng layer.



- Link Layer Header (or Link Layer) → check MAC (physical address).
- Internet Layer Header → check IP address.
- Transport Layer Header → check port number.

- Application Layer Header (Message/Payload) → content layer.
- Link Layer Trailer (nếu có) → checksum (or frame check sequence) ⇒ error checking.

Self-note:

- Trong cái Link Layer Header, cái header đó để check gói tin đang gọi có đúng không.
- Cái tail (trailer) để check xử lý triệt để hết data trong đó chưa thông qua việc checksum.
- Checksum → kiểm tra xem file có lỗi gì khi tải về không (thường thấy dưới dạng dãy số SHA).

### 1.3.4 OSI Reference Model

→ Two additional layers are placed below the Application Layer and above the Transport Layer.

| TCP/IP Reference Model |  | OSI Reference Model |
|------------------------|--|---------------------|
| Application Layer      |  | Application Layer   |
|                        |  | Presentation Layer  |
|                        |  | Session Layer       |
| Transport Layer        |  | Transport Layer     |
| Internet Layer         |  | Network Layer       |
| Link Layer             |  | Data Link Layer     |
|                        |  | Physical Layer      |

- Presentation Layer → defines format of the data so receiver can understand it (data format).
- Session Layer → establishes, maintains, and terminates the connection between applications (dialogue).
- Data Link Layer → groups raw bits into frames, handles MAC address & check errors (local logic).
- Physical Layer → transmits 1s and 0s (bit) over the wire (raw signal).

Self-note:

⇒ Model OSI là model hoàn chỉnh hơn so với TCP/IP.

- OSI hơn 2 layer so với TCP/IP; trong đó có 2 layer được thêm vào là Session Layer và Presentation Layer, với modify lại Link Layer thành Data Link Layer và Physical Layer.
- Trên thực tế, người ta dùng TCP/IP nhiều hơn, do nhanh hơn (vì có ít layer và đỡ phức tạp hơn). OSI rất phức tạp nhưng đầy đủ, nên sử dụng chủ yếu trong giảng dạy.

### 1.3.5 OSI Model Concepts

Self-note: OSI nghĩa là Open Systems Interconnection → là một khung khái niệm chia các chức năng truyền thông mạng thành 7 lớp.

⇒ Là mô hình quan trọng do nó chuẩn hóa cách mạng hoạt động, chia nhỏ quá trình truyền thông mạng phức tạp thành 7 tầng để quản lý.

Central concepts:

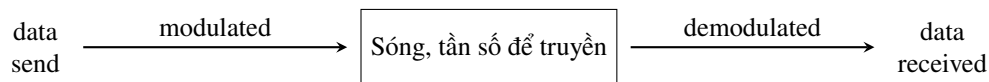
- Service → define what layer does.
- Interfaces → defines how to access.
- Protocols → defines how the layer is implemented (communication).

### 1.3.6 Physical Layer

→ Main concept: raw signal with 0s and 1s.

- Sender → signals are modulated (điều chỉnh) onto the medium.
- Receiver → signals are demodulated (giải điều chỉnh) from the medium.

Self-note:



Finalize:

The core function: moving bits.

- Raw transmission → to transmit the raw stream of 1s and 0s.
- Physical connection → defines the physical specifications for the connection (quy định các thông số kĩ thuật cho connection; ví dụ: hình dạng đầu cắm dây cáp, loại dây cáp).

### 1.3.7 Data Link Layer

→ Main concept: group into frames, handle transmission errors (with checksums), specifies physical network address (MAC address).

- Sender → pack the network into frames and transmits them via a physical network from one device to another.
- Receiver → identifies frames in the bit stream from the Physical Layer.

Self-note:

Frame (khung dữ liệu) → dùng để chứa dữ liệu và vận chuyển nó qua đường dây vật lý (physical link) hoặc sóng vô tuyến (wireless) từ sender đến receiver trong cùng một network.

→ Tầng liên kết (data link layer) nhận một gói tin (packet) từ tầng mạng ở trên, sau đó gắn header ở trên và trailer ở dưới.

Finalize:

The core function: Node-to-Node Delivery.

- Local Delivery → to move the data between 2 devices connected to the same physical medium (ví dụ: Laptop → Switch → Router).
- Framing → takes the raw stream of bits from the physical layer and organizes them into distinct "sentences" called frames (gồm các tín hiệu rồi rạc thành các gói dữ liệu).

### 1.3.8 Network Layer

→ Main concept: connects different networks together.

→ It doesn't care about the MAC address; only cares the IP to connect with WAN.

- Sender → packs the segments of the Transport Layer in packets.
- Receiver → unpacks the packets in the frames from the Data Link Layer; **routers** and **Layer-3-Switches** connect logical networks.
- Usually the connectionless IP is used → other protocols have been replaced by IP (e.g., IPX).

⇒ **3 main tasks:**

- Logical addressing.



- Routing.
- Encapsulation.

Finalize:

The core function: Routing and Addressing.

- Internetworking → to forward packets between sender and receiver in the network.
- Logical Addressing → uses IP addresses to identify devices ⇒ unlike MAC addresses, IP addresses are assigned based on where you are in the network (MAC address xác định danh tính của Card mạng (NIC)). Ví dụ, IP sẽ hỏi là "máy tính này đang ở đâu?", trong khi đó MAC sẽ hỏi là "đâu là cái máy nào của này đang dùng dây?".

### 1.3.9 Transport Layer

→ Main concept: ensures the network get the correct application.

- Sender → packs the data of the Application Layer into segments.
- Receiver → unpacks the segments inside the packets from the network layer.
- Addresses process with **port number**.

→ **Main task:** end-to-end communication between processes.

→ Because the computer receive thousands of packets every seconds ⇒ it uses port to identify.

**2 main modes:**

- Connection-oriented (TCP) → reliable, stateful ⇒ ensure data arrives perfectly.
- Connectionless (UDP) → unreliable, stateless, fast ⇒ no guarantee of delivery.

Self-note:

- TCP → nó đảm bảo đủ data qua ⇒ tốn thời gian (ví dụ: download file nào đó).
- UDP → nó đảm bảo kịp tiến độ truyền qua ⇒ dễ thiếu data (ví dụ: voice call).

Finalize:

The core function: End-to-End Communication.

- Process-to-Process → to transport segments between specific processes (applications) on different devices.
- Port numbers → uses to identify which application should receive the data (phân chia gói tin vào các ứng dụng).

### 1.3.10 Session Layer

→ Main concept: dialogue control: setup, manage, and terminate connections (sessions).

The Problem: Nếu dung lượng 5GB nhưng mà nó download tới 4.9GB thì rớt mạng. Phải làm sao?

⇒ Không có session layer thì nó sẽ download lại từ con số 0 (cho nó không có checkpoint).

→ **3 main tasks:**

- Checkpointing (and recovery).
- Authentication.
- Authorization.

Self-note: Trong TCP/IP, người ta cho cái layer này vào trực tiếp cái Application Layer, ví dụ như dùng cookie để maintain session.

Finalize:

The core function: dialogue control → to setup, maintain, and terminate connections (sessions) between applications.

- Synchronization → decides whose turn it is to speak and keep the data stream organized (thiết lập & quản lý các phiên làm việc giữa các ứng dụng).
- State Management → tracks if the connection is active, idle, or broken.
- security → xác thực và cấp quyền trước khi bắt đầu một connection.

### 1.3.11 Presentation Layer

→ Defines format (presentation) of the data.

The Problem: một bên dùng ASCII để render text, còn một bên dùng EBCDIC. Phải làm sao?

⇒ Phải có layer này để cho ra common format (format chung) mới có thể giao tiếp được.

→ **3 main tasks:**

- Translation → connect to standard format.
- Encryption → security.
- Compression → smaller data for faster travelling.

Finalize:

The core function: Formatting and Syntax → to define the format (presentation) of the data.

- Translation → đảm bảo máy tính nhận hiểu được dữ liệu mà máy tính gửi truyền đi.
- Encryption → đảm nhận việc mã hóa dữ liệu để tăng tính bảo mật.
- Compression → thu gọn dữ liệu để truyền nhanh hơn.

### 1.3.12 Application Layer

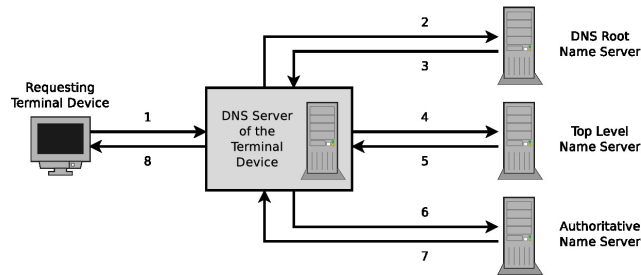
→ User interaction.

→ **2 main tasks:**

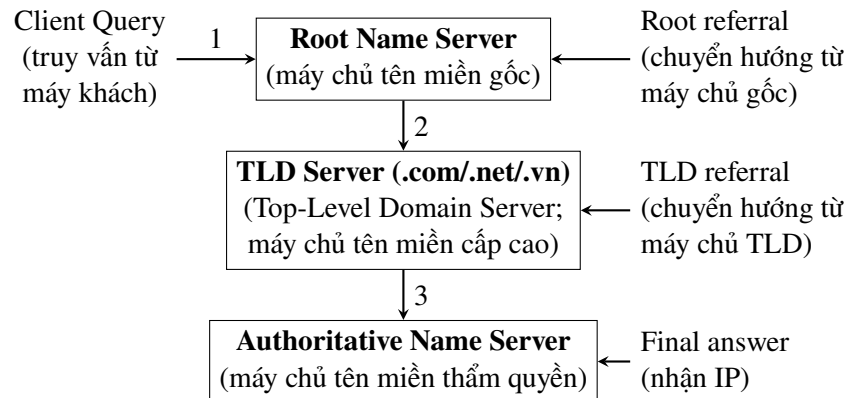
- The payload → holds actual data we care.
- Semantics → focus on the meaning and logic of the application (don't care about IP addresses, cables, or routing).

Self-note:

DNS nghĩa là Domain Name Server.



→ Tóm tắt DNS: Client → Root → TLD → Authoritative → nhận IP.



The Problem: Computers communicate using numbers called IP address, but we can't remember them. What can we do?

⇒ We prefer like `google.com`, etc.

⇒ DNS translates (resolves) the human-readable Domain Names into machine readable IP address.

#### Finalize:

The core function: user interaction → contains all protocols that interact directly with application programs.

- The payload (giao diện người dùng) → chứa các giao thức để communicate giữa phần mềm trong network.
- Semantics (ngữ nghĩa) → chứa nội dung muốn gửi (content).

## 1.4 Topologies

### 1.4.1 Topologies of Computer Networks

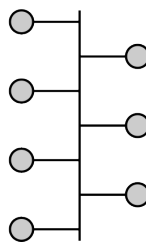
→ Mô tả cách bố trí & kết nối các thiết bị trong network.

Large-scale network is often a combination of different topologies. Physical and Logical topology may differ:

- Physical topology → actual layout of the devices (cách đi dây thực tế).
- Logical topology → how data flows within the network (cách data di chuyển trong network).

Topologies are graphically represented with nodes and edges.

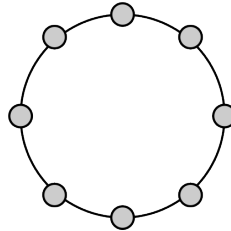
### 1.4.2 Bus Network



→ Tất cả các thiết bị được kết nối vào một đường dây truyền dẫn chung gọi là Bus ⇒ không có thiết bị trung tâm nào ở giữa.

- **Pros:** cheap.
- **Cons:** dây cáp chính đứt → dead; một thời điểm chỉ có 1 máy gửi dữ liệu, 2 máy gửi cùng lúc sẽ bị collision.

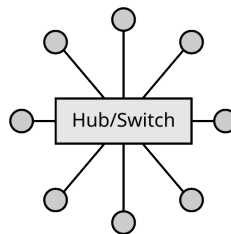
### 1.4.3 Ring Network



→ Mỗi nút nối với đúng 2 nút khác, tạo thành một vòng khép kín, dữ liệu truyền 1 chiều từ nút này sang nút khác.

- **Pros:** mỗi nút hoạt động như một bộ khuếch đại (repeater) → tín hiệu có thể truyền đi xa hơn.
- **Cons:** một dây đứt or 1 máy hỏng → dead (trừ khi dùng vòng kép dự phòng).

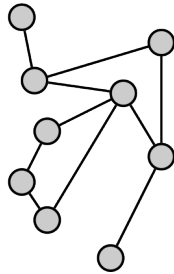
### 1.4.4 Star Network



→ Tất cả các nút kết nối trực tiếp vào 1 thiết bị trung tâm (Hub / Switch).

- **Pros:** stable → 1 máy or dây bị hư vẫn hoạt động bình thường; dễ mở rộng → thêm dây vào Switch là có thể thêm máy để hoạt động.
- **Cons:** nếu Switch hỏng → dead.

### 1.4.5 Mesh Network

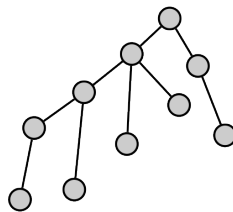


Self-note: mesh network nghĩa là mạng lưới.

→ Mỗi nút được kết nối với nhiều nút khác.  $\Rightarrow$  Full mesh  $\rightarrow$  mọi nút được kết nối với nhau.

- **Pros:** safe  $\rightarrow$  nếu bị đứt dây thì còn đường dự phòng.
- **Cons:** chi phí cao; đi dây phức tạp.

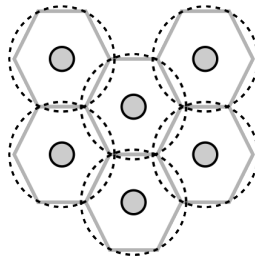
### 1.4.6 Tree Network



→ Sự kết hợp phân cấp của star network; có nút root trên cùng.

- **Pros:** tốt cho việc quản lý các mạng lớn.
- **Cons:** root hỏng  $\rightarrow$  các nhánh dead; dễ bị nghẽn  $\rightarrow$  nếu large tree  $\Rightarrow$  communication from one half of the tree to the other half must go through the root.

### 1.4.7 Cellular Network



→ Dùng cho mạng không dây. Mạng được chia thành các khu vực gọi là cell, mỗi cell có một trạm phủ sóng.

- **Pros:** người dùng không cần dây dẫn; node failure không bị ảnh hưởng.
- **Cons:** cần cơ chế tránh xung đột (collision) (CSMA/CA) do nhiều thiết bị dùng chung một tần số; bị giới hạn vùng phủ sóng (based on stations and their positions).

### 1.4.8 Current Situation

- LAN → dùng star network.
- Wireless → dùng cellular network.
- Lõi Internet (backbone) → dùng mesh network.
- Bus & Ring → hiếm khi dùng nữa (no longer used).

## Chapter 2

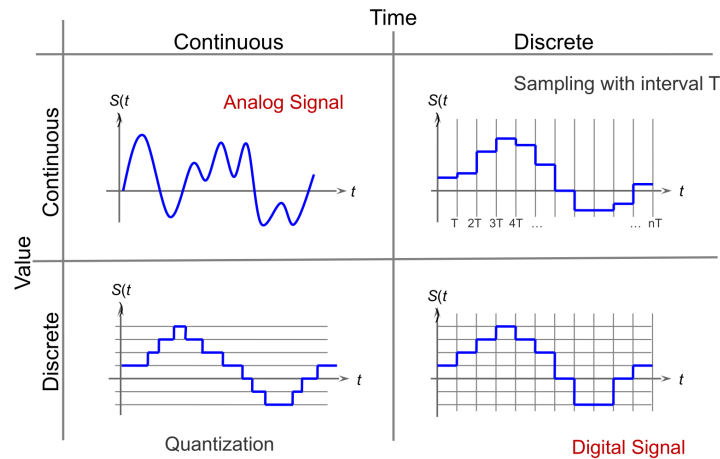
# Physical Layer

### 2.1 Fundamentals of Data Signals

#### 2.1.1 Physical Representation of Data

Signal → a physical representation of data.

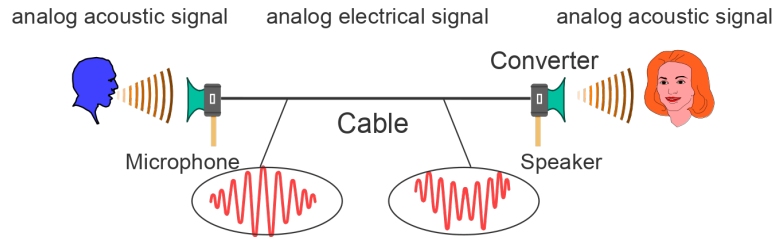
- Analog signal (continuous signal) → tín hiệu liên tục (continuous); biến đổi liên tục theo thời gian (ví dụ, nó sẽ chạy từ 0 đến 1, chấp nhận các giá trị trong khoảng này như 0.1, 0.01, 0.001, ...).
- Digital signal (discrete signal) → tín hiệu rời rạc (discrete); biến đổi theo từng bước (ví dụ, nó chỉ chấp nhận các giá trị 0 và 1).



The translator  $\Rightarrow$  NIC (Network Interface Card, nghĩa là card mạng) hoạt động như bộ mã hóa và giải mã (**coder** and **decoder**  $\rightarrow$  CODEC).



### 2.1.2 The Telephone Example

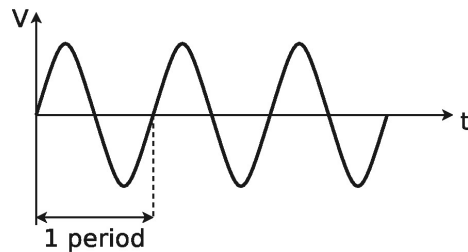


#### Giải thích:

1. Giọng nói tạo ra sóng âm (ra tín hiệu âm thanh (analog acoustic signal)) → đi qua microphone, hoạt động như bộ chuyển đổi (→ biến sóng âm thành dòng điện).
2. Tín hiệu điện (analog electrical signal) truyền trong dây dẫn (cable) tới converter (là speaker).
3. Ở đây, speaker sẽ biến dòng điện trở lại thành âm thanh (acoustic signal) để người nghe có thể nghe được.

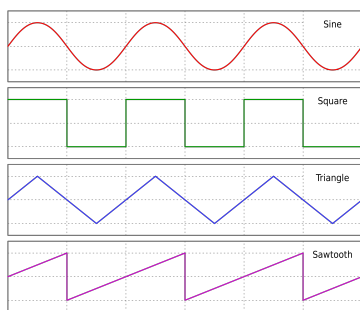
### 2.1.3 Basics of Signal Processing

Periodic signal (tín hiệu tuần hoàn) → simplest signals.



#### Parameters:

- Period ( $T$ )
- Frequency ( $f$ ) → số chu kỳ trong 1 giây (Hz)  $\Rightarrow f = \frac{1}{T}$ .
- Amplitude ( $S(t)$ ) → độ lớn tín hiệu.
- Phase ( $\phi$ ) → vị trí bắt đầu của sóng trong chu kỳ.



- Sine  $\rightarrow$  period of  $2\pi$ .
- Square wave.
- Triangular wave.
- Sawtooth wave.

### 2.1.4 Quantization and Sampling

Để truyền dữ liệu (transmit data) qua môi trường chuyển dẫn (transmission medium), tín hiệu phải được:

- converted ( $\rightarrow$  quantization (lượng hóa))  $\Rightarrow$  chuyển đổi một dãy giá trị liên tục vào một cái gì đó cố định hơn  $\Rightarrow$  continuous values (giá trị liên tục)  $\rightarrow$  cần quantization.
- measured ( $\rightarrow$  sampling (lấy mẫu))  $\Rightarrow$  đo tín hiệu tại các thời điểm khác nhau  $\Rightarrow$  discrete time (thời gian rời rạc)  $\rightarrow$  cần sampling.

Self-note:

- Máy tính lưu dữ liệu bằng các bit (0 và 1); nó cần một danh sách các con số cố định để lựa chọn  $\Rightarrow$  cần quantization để chuyển tín hiệu liên tục thành tín hiệu rời rạc để lựa chọn (ví dụ: cân đo nhưng kim nằm giữa 1kg và 1.1kg, nhưng mình không thể đọc được cái con số siêu lẻ  $\rightarrow$  đọc 1kg luôn).
- Máy tính không thể lưu trữ vô hạn trong một khoảng khắc nào đó, nếu nó làm thế thì sẽ dễ bị đầy bộ nhớ  $\Rightarrow$  cần sampling để đo tín hiệu tại các thời điểm khác nhau  $\rightarrow$  biến thời gian liên tục thành thời gian rời rạc để xử lý (ví dụ: quay video ở 30fps thay vì vô hạn fps).

### 2.1.5 Sampling, Quantization, and Coding

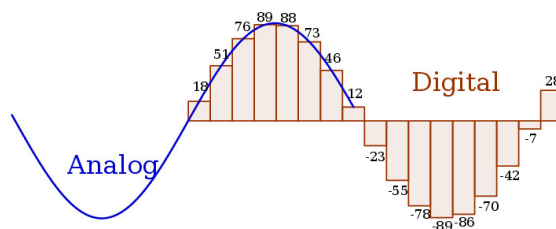
Target: chuyển tín hiệu analog sang digital representation.

Sampling and quantization:

- Periodic measurement (đo đạc có chu kì)  $\rightarrow$  sampling  $\Rightarrow$  xử lý việc tín hiệu biến thiên liên tục theo thời gian bằng cách đo ngắt quãng.
- Dividing signal range (chia nhỏ dải tín hiệu)  $\rightarrow$  quantization  $\Rightarrow$  xử lý việc biên độ tín hiệu liên tục bằng cách chia nhỏ thành các khoảng (intervals) và gán giá trị cho chúng.

Coding (mã hóa):

→ Focus on translating to binary  $\Rightarrow$  gán một mã nhị phân (binary code) cho từng khoảng lượng tử hóa (quantization intervals).

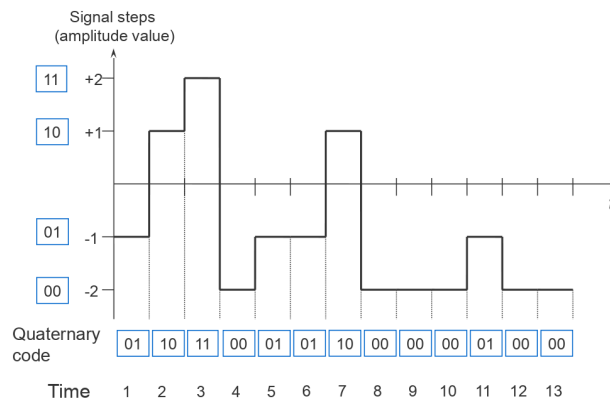


→ **Giải thích hình:**

- Đường cong analog  $\rightarrow$  tín hiệu thực tế; giá trị liên tục (continuous values).
- Các cột digital  $\rightarrow$  dữ liệu máy tính ghi lại được  $\Rightarrow$  mỗi cột đại diện cho một lần lấy mẫu (sampling).
- Các con số trên đỉnh mỗi cột  $\rightarrow$  kết quả của lượng tử hóa (quantization).

## 2.1.6 Symbol Rate

Self-note: symbol rate nghĩa là tốc độ kí hiệu (định nghĩa ở mục tiếp theo).



→ Biểu đồ minh họa quaternary (hệ tứ phân; nghĩa là có 4 trạng thái).

- Amplitude value (giá trị biên độ)  $\rightarrow$  tín hiệu không chỉ có 2 mức (0 và 1), mà có 4 mức điện áp khác nhau.

- Time → mỗi vạch là một khoảng thời gian cho một symbol.

#### Bonus:

$n$  (number of discrete values) → số lượng mức giá trị khác nhau mà tín hiệu có thể nhận được.

- $n = 2 \rightarrow$  binary  $\Rightarrow$  tín hiệu có 2 mức  $\rightarrow$  chỉ được gửi 1 bit mỗi lần ( $\log_2 2 = 1$ ).
- $n = 3 \rightarrow$  ternary  $\Rightarrow$  tín hiệu có 3 mức.
- $n = 4 \rightarrow$  quaternary  $\Rightarrow$  tín hiệu có 4 mức  $\rightarrow$  chỉ được gửi 2 bit mỗi lần ( $\log_2 4 = 2$ ).
- $n = 8 \rightarrow$  octonary  $\Rightarrow$  tín hiệu có 8 mức  $\rightarrow$  chỉ được gửi 3 bit mỗi lần ( $\log_2 8 = 3$ ).
- $n = 10 \rightarrow$  denary  $\Rightarrow$  tín hiệu có 10 mức.

### 2.1.7 Bit Rate and Symbol Rate

Bit rate (tốc độ bit) → tổng số lượng các bit (0 và 1) được truyền đi trong một đơn vị thời gian (thường là giây)  $\Rightarrow$  đơn vị là **bps** (bits per second).

$\Rightarrow$  Đây là tốc độ thực tế người ta quan tâm (tải file nhanh hay chậm phụ thuộc vào bit rate).

Symbol rate (tốc độ kí hiệu) → số lượng các kí hiệu (symbols) được truyền đi trong một đơn vị thời gian; một symbol là một trạng thái cụ thể của tín hiệu (ví dụ: một điện áp +5V, một xung sóng); đơn vị là **baud** (symbols per second).

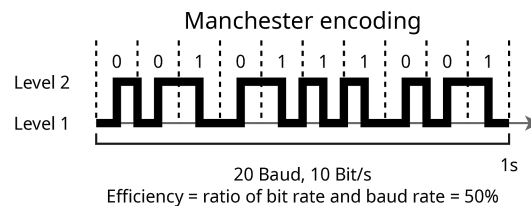
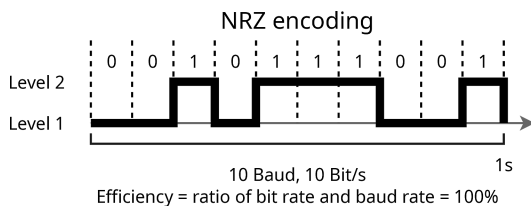
$\Rightarrow$  Đây là tốc độ làm việc của phần cứng (chẳng hạn như dây cáp phải đổi state (trạng thái) bao nhiêu lần).

The ratio between bit rate and symbol rate  $\rightarrow$  phụ thuộc vào công nghệ mã hóa đường truyền (line encoding scheme) được sử dụng.

#### Self-note:

Line code  $\rightarrow$  là quy tắc để máy tính truyền tín hiệu lên dây dẫn sao cho bên nhận có thể hiểu được.

$\Rightarrow$  Quy định: số lượng tín hiệu tối đa có thể truyền qua transmission media được sử dụng (số 1 thì điện áp như nào, số 0 thì điện áp như nào); xác định bởi layer protocol được sử dụng.



Bonus: (sẽ được nói kĩ hơn ở mục dưới)

NRZ (Non-Return to Zero) encoding → gặp số 0 thì giữ điện áp mức thấp, còn gặp 1 thì giữ ở mức cao.

Manchester Encoding → không quan tâm mức cao hay thấp, chỉ quan tâm đến sự thay đổi (transition) giữa bit:

- Số 0 → đang cao nhảy xuống thấp (falling edge).
- Số 1 → đang thấp nhảy lên cao (rising edge).

### 2.1.8 Data Rate

Data rate → tốc độ truyền dữ liệu tối đa mà đường truyền có thể tải được.

⇒ Nó trả lời cho câu "trong 1s thì gửi được bao nhiêu bit đi?".

→ **Hai yếu tố ảnh hưởng data rate:**

1. Bandwidth (băng thông), kí hiệu:  $H$  → độ rộng của đường vận chuyển. Đường càng rộng (Hertz càng cao) ⇒ bit chạy càng nhiều.
2. Levels (mức tín hiệu) (hoặc độ nhiễu (noise)), kí hiệu:  $V$  → nếu không có nhiễu ⇒ dùng càng nhiều mức tín hiệu ( $V$  càng cao; như 4 mức, 8 mức), tốc độ càng nhanh.

**Định luật Hartley (Hartley's law):**

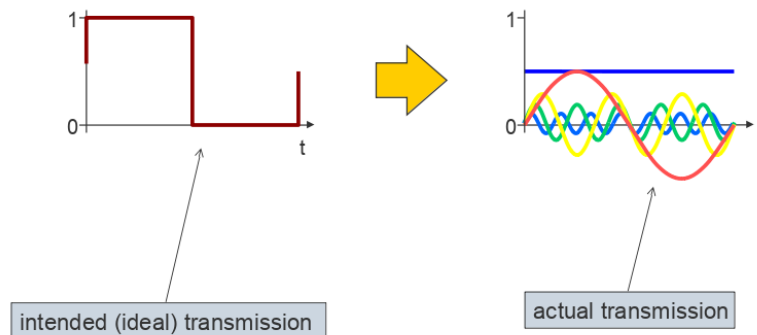
$$\text{Maximum data rate [bit/s]} = 2 * H * \log_2(V)$$

Trong đó:

- $H$  → độ rộng băng tần của đường truyền (tính bằng Hz) ⇒ Đường càng rộng thì chạy càng nhanh.
- $V$  → số lượng mức tín hiệu (như  $n = 2$ ,  $n = 4$ ).

Note: Định luật của Hartley là **điều kiện lý tưởng** → đường truyền hoàn hảo, không có nhiễu (noiseless) ⇒ tốc độ tối đa phụ thuộc vào việc mình tạo ra được bao nhiêu tín hiệu.

### 2.1.9 Ideal vs. Real Transmission



Giải thích:

- Bên lý tưởng  $\rightarrow$  tín hiệu hoàn hảo mà máy tính gửi đi dưới dạng sóng vuông (square wave)  $\Rightarrow$  không bị nhiễu, chuyển đổi tức thì từ mức 0 lên 1.
- Bên thực tế  $\rightarrow$  tín hiệu bị biến thành sóng do bị nhiễu, cộng thêm các đường sóng nhỏ (màu xanh, vàng) (theo như chuỗi Fourier (Fourier series)  $\rightarrow$  một sóng vuông thực tế là tổng hợp của vô số sóng hình sin có tần số khác nhau cộng lại)  $\Rightarrow$  tín hiệu không chuyển đổi tức thì từ mức 0 lên 1 mà cần thời gian để chuyển đổi.

$\Rightarrow$  Nếu đường truyền quá dài hoặc quá tẻ (do bị nhiễu quá nhiều chẳng hạn)  $\rightarrow$  sóng sẽ bị "méo" đến mức máy tính không thể phân biệt được đâu là mức 1, đâu là mức 0 nữa.

$\rightarrow$  Ở mặt lý tưởng, Claude Shannon cho rằng mục tiêu của truyền tin là tái tạo lại chính xác thông tin  $\rightarrow$  tuy nhiên, trên thực tế, có rất nhiều yếu tố làm suy giảm tín hiệu trong quá trình truyền dẫn.

### 2.1.10 Attenuation

Attenuation (suy hao)  $\rightarrow$  tín hiệu yếu đi khi truyền xa.

$\rightarrow$  Làm cho suy yếu tín hiệu (signal weakening)  $\rightarrow$  làm yếu đi biên độ của tín hiệu ngày càng nhiều, theo quãng đường truyền tin của các phương tiện truyền tải  $\Rightarrow$  năng lượng tín hiệu mất dần.

$\rightarrow$  **Hậu quả:** nếu biên độ (amplitude) của tín hiệu quá thấp so với mức yêu cầu  $\Rightarrow$  máy sẽ không thể nhận biết được tín hiệu nữa  $\Rightarrow$  mất dữ liệu.

$\Rightarrow$  Attenuation giới hạn khoảng cách tối đa có thể truyền tải tín hiệu với mọi transmission media.

**Quy luật:** Tần số càng cao, suy hao càng lớn.

Idea: người ta có thể khắc phục điều này bằng việc sử dụng repeater (bộ lặp) hoặc amplifier (bộ khuếch đại) để tăng cường tín hiệu, để tín hiệu có thể truyền đi tiếp.

### 2.1.11 Noise and Distortion

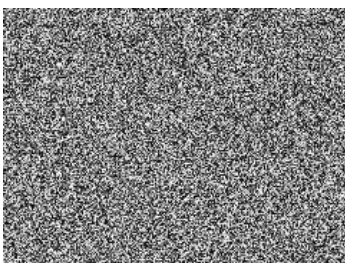
**Noise** (nhiều) → tín hiệu không mong muốn làm thay đổi tín hiệu gốc.

- Thermal noise (nhiều nhiệt) → do chuyển động ngẫu nhiên của các electron trong dây dẫn tạo ra.
- Intermodulation noise (nhiều tương hỗ) → do các tần số không mong muốn kết hợp với tín hiệu ban đầu.
- Crosstalk (nhiều chéo) → do tín hiệu từ dây dẫn khác truyền sang.
- Impulse noise (nhiều xung) → do các sự kiện đột ngột như sét đánh hoặc các sự cố điện.

**Distortion** (biến dạng) → tín hiệu bị thay đổi hình dạng do các đặc tính của phương tiện truyền dẫn.

- Echoes (tiếng vang) → tín hiệu phản xạ trở lại.
- Extreme low frequency (ELF) (tần số cực thấp) → tín hiệu tần số thấp bị suy giảm nhiều hơn.
- Delay distortion (biến dạng trễ) → các thành phần tần số khác nhau truyền với tốc độ khác nhau.

→ Plus attenuation, refraction (làm lệch hướng), reflection (phản xạ), etc.



AWGN (Additive White Gaussian Noise) → mô hình nhiễu phổ biến nhất.

Giải thích:

- Additive (cộng tính) → nhiễu được thêm vào tín hiệu gốc.

- White → phổ tần số rộng (giống như ánh sáng trắng) ⇒ nhiễu trắng chứa tất cả các tần số với công suất bằng nhau.
- Gaussian Noise (nhiều Gauss) → cường độ nhiễu tuân theo phân phối Gaussian (phân phối chuẩn - hình chuông).

**Gaussian channel** (kênh Gauss) → khi một kênh truyền (đường dây) chịu tác động chủ yếu của loại nhiễu này (nhiều nhiệt, bức xạ nền, ...).

### 2.1.12 Bit Error Rate (BER)

Bit Error Rate (Tỷ lệ lỗi bit) → đánh giá độ tin cậy của một đường truyền dữ liệu.

⇒ Nhiễu có thể làm suy giảm chất lượng đường truyền → có thể khiến máy tính khi nhận đọc sai (như 0 thành 1, hay 1 thành 0 chẳng hạn).

⇒ BER được tính theo tỉ lệ phần trăm các bit bị sai so với tổng số bit đã được truyền đi.

**Công thức:**

$$\text{BER} = \frac{\text{Number of erroneous bits (số bit bị lỗi)}}{\text{Number of transmitted bits (tổng số bit đã truyền)}}$$

→ Ví dụ: gửi 1.000 bit và có 1 bit bị sai ⇒  $\text{BER} = \frac{1}{1000} = 0.001$  (hay  $10^{-3}$ ).

Typical BER values for different link types:

- POTS (Plain Old Telephone Service) →  $2 * 10^{-4}$ .
- Radio links →  $10^{-3}$  to  $10^{-4}$ .
- Ethernet →  $10^{-9}$  to  $10^{-10}$ .
- Fiber (cáp quang) →  $10^{-10}$  to  $10^{-12}$ .

Self-note:

→ Mạng radio có BER tệ nhất ⇒ do chịu nhiều tác động từ môi trường xung quanh như thời tiết, vật cản, ... ⇒ dễ bị nhiễu hơn so với mạng dây (như Ethernet hay fiber).

BER **càng nhỏ** (số có mũ càng âm lớn) ⇒ đường truyền **càng tốt**.

Bonus: Giảm BER → tăng cường độ tín hiệu để át nhiễu.

⇒ Việc này có thể gây tốn kém (như tốn điện do phải tăng biên độ → tăng mức tiêu thụ năng lượng), và gây nhiễu cho các thiết bị xung quanh.



### 2.1.13 Data Rate on a Noisy Channel

→ Mọi kênh truyền thực tế đều bị nhiễu bởi nhiễu.

→ Tốc độ truyền không chỉ phụ thuộc vào số mức tín hiệu ( $V$ ), mà còn phụ thuộc vào cường độ tín hiệu mạnh hơn nhiễu bao nhiêu lần.

⇒ Signal-to-Noise Ratio (SNR) → tỉ lệ công suất tín hiệu ( $S$ ) và công suất nhiễu ( $N$ ).

$$\text{SNR} = \frac{\text{Signal Power } (S)}{\text{Noise Power } (N)}$$

⇒ SNR càng cao ⇒ tín hiệu càng mạnh hơn nhiễu.

Ví dụ: nói chuyện trong một quán karaoke, giọng mình ( $S$ ) so với tiếng nhạc ồn ( $N$ ), nếu  $S < N$ , người ta sẽ không nghe được mình nói gì cả → tốc độ truyền tin = 0.

**Định luật Shannon-Hartley (Shannon-Hartley theorem):**

$$\text{Maximum data rate [bit/s]} = H * \log_2\left(1 + \frac{S}{N}\right)$$

Trong đó:

- Signal strength ( $S$ ) → cường độ tín hiệu.
- Noise strength ( $N$ ) → cường độ nhiễu.
- Bandwidth ( $H$ ) → độ rộng băng tần (tính bằng Hz).

The SNR is commonly expressed in decibel (dB):

$$\text{SNR [dB]} = 10 * \log_{10}\left(\frac{S}{N}\right)$$

Self-note: công thức Shannon ở trên dùng tỷ số thường ( $S/N$ ) → nếu đề bài cho SNR là 30dB, mình **không được** thay số 30 vào công thức Shannon

⇒ Phải chuyển đổi 30dB thành 1000 lần ( $S/N = 10^{30/10} = 10^3 = 1000$ ) rồi mới thay vào công thức Shannon.

Finalize: định lý Shannon-Hartley cho ta biết giới hạn lý thuyết về tốc độ truyền dữ liệu tối đa trên một kênh truyền có nhiễu.

## 2.2 Data Encoding

### 2.2.1 Line Encoding

#### 2.2.1.1 Baseband and Broadband (part 1)

Vấn đề được đặt ra: làm thế nào để có thể đưa các bit (0 và 1) lên đường truyền?

→ Data encoding (mã hóa dữ liệu) cần xác định được kí hiệu nào là 0, kí hiệu nào là 1.

Therefore:

→ **Baseband** (dải cơ sở) → truyền tín hiệu nguyên bản (transmitted as is); tín hiệu số (xung vuông) được đẩy trực tiếp lên dây ⇒ cần có data encoding để quy định rõ các mức điện áp 0 hay 1  
→ thường dùng cho mạng LAN hoặc kết nối ngắn.

#### 2.2.1.2 Encoding Schemes

Cơ chế mã hóa (encoding schemes) → tập hợp các quy tắc để chuyển đổi các bit (0 và 1) thành các tín hiệu vật lý để truyền.

**Có 3 yếu tố chính:**

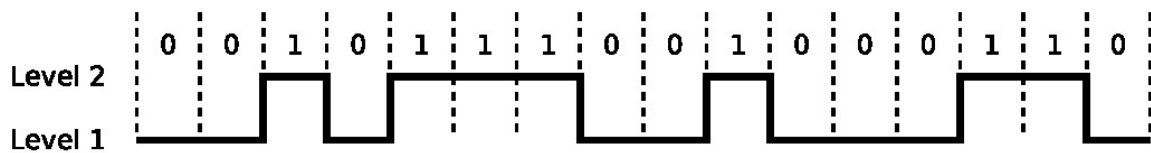
- Robust (bền vững) → tín hiệu phải đủ mạnh và rõ ràng để chống lại nhiễu và distortion (biến dạng) → cho dù bị biến đổi thì bên nhận vẫn đọc được đâu là 0, đâu là 1.
- Efficient (hiệu quả) → mục tiêu: đạt được tốc độ truyền dữ liệu cao nhất có thể ⇒ cơ chế nào cho phép truyền nhiều bit nhất trong cùng một khoảng thời gian (hoặc cùng dải tần) thì hiệu quả nhất.
- Synchronous (đồng bộ) → máy gửi và máy nhận phải cùng "nhịp" với nhau thì mới biết được khi nào một bit bắt đầu và kết thúc.

Cách cách để đồng bộ hóa:

- Dùng explicit clock signal (tín hiệu xung nhịp rõ ràng) → gửi một tín hiệu xung nhịp riêng biệt cùng với dữ liệu ⇒ tốn kém.
- Synchronize on certain points (đồng bộ theo thời điểm) → sử dụng như "start bit" để báo hiệu bắt đầu.
- Self-synchronizing signal (tự đồng bộ) → bản thân tín hiệu luôn chứa thông tin về xung nhịp (nhờ việc đảo chiều liên tục) ⇒ máy nhận không bao giờ lạc nhịp (như Manchester encoding).

→ There are many ways to encode binary data onto a line; many different encodings are used in different technologies.

### 2.2.1.3 Non-Return-to-Zero (NRZ)



→ là kĩ thuật mã hóa đường truyền đơn giản nhất.

**Nguyên lý hoạt động:** NRZ thực hiện việc gán mức điện áp trực tiếp cho các bit:

- Bit 0 → được mã hóa bằng mức điện áp thấp (level 1, chẳng hạn như -5V).
- Bit 1 → được mã hóa bằng mức điện áp cao (level 2, chẳng hạn như +5V).

Lý do gọi là "Non-Return-to-Zero" → trong suốt thời gian truyền một bit (ví dụ như bit 1) → điện áp giữ nguyên ở mức cao từ đầu đến cuối ⇒ không trở về mức 0 (zero) giữa các bit (khác với mã RZ).

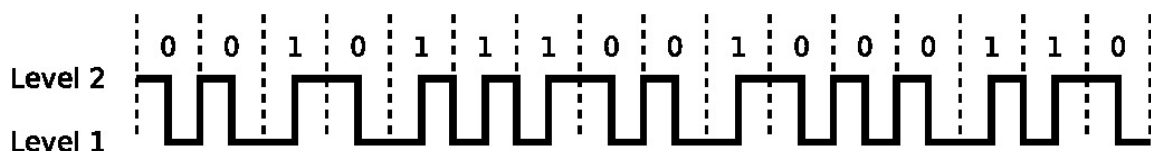
→ Khác với mã RZ (Return-to-Zero) → nếu là bit 1 (mức cao), điện áp nhảy lên mức cao ⇒ nhưng bắt buộc phải trở về mức 0 trước khi bit tiếp theo bắt đầu (khi đi được nửa thời gian của bit đó).

- **Pros:** đơn giản và hiệu quả (simple and efficient) → sử dụng băng thông rất tốt do tần số tín hiệu không phải thay đổi quá nhanh (so với lại Manchester encoding).
- **Cons:** gặp rắc rối lớn khi truyền dữ liệu có một chuỗi dài các bit giống nhau (như đã nói ở trên) ⇒ đường tín hiệu sẽ **gặp 2 lỗi**: clock recovery (máy sẽ không nhận biết mình đang ở bit thứ mấy) và baseline wander (giữ điện áp quá lâu có thể bị thay đổi mức điện áp trung bình ⇒ máy thu sẽ không biết đâu là mức cao, đâu là mức thấp nữa).

Bonus:

→ Kĩ thuật này được sử dụng trong CAN bus system (Controller Area Network) → một hệ thống dùng để kết nối với các bộ điều khiển trong xe hơi do Bosch phát triển từ những năm 1980.

### 2.2.1.4 Manchester Code



→ là kĩ thuật giải quyết cho vấn đề đồng bộ hóa.

⇒ Tập trung vào hướng di chuyển (transition) của dòng điện ở giữa khoảng thời gian mỗi bit (khác với NRZ chỉ quan tâm đến điện áp cao hay thấp).

- Logical 1 → encoded with a rising edge (sườn lên) ⇒ thay đổi từ signal level 1 (mức thấp) sang signal level 2 (mức cao).
- Logical 0 → encoded with a falling edge (sườn xuống) ⇒ thay đổi từ signal level 2 (mức cao) sang signal level 1 (mức thấp).

→ **Quy tắc đặt biệt:** nếu có 2 bit giống nhau đi liền (ví dụ: 0 và 0 ở sát nhau), thì tại cuối bit thứ nhất, tín hiệu sẽ phải đổi trạng thái để phân biệt cho việc đổi trạng thái của bit sau.

Self-note:

⇒ Nếu quan sát kĩ trên hình vẽ thì Manchester encoding sẽ luôn có một transition ở giữa bit; nửa trái là để phân biệt giữa 2 bit, nửa phải là để biểu diễn giá trị của bit đó (0 hay 1 thông qua mức thấp hay cao).

- **Pros:** tự đồng bộ (self-synchronizing) → do ở chính giữa của mỗi bit luôn có một transition ⇒ máy nhận có thể dùng cái này để đếm xung (clock) ⇒ không bao giờ bị nhầm lẫn giữa các bit dù có gửi cả chuỗi bit giống nhau dài cách mấy.
- **Cons:** hiệu suất thấp, chỉ đạt cỡ 50% so với NRZ ⇒ vì khi 1 bit được gửi → tín hiệu phải thay đổi trạng thái tới 2 lần (nửa đầu khác, nửa sau khác) ⇒ tốc độ Baud phải gấp đôi so với tốc độ bit → tốn băng thông hơn so với NRZ.

Bonus:

→ Do tính ổn định và khả năng tự đồng bộ cao, mã này được sử dụng trong mạng Ethernet 10 Mbps (10BASE-T hay 10BASE2).

## 2.2.2 Modulation

### 2.2.2.1 Baseband and Broadband (part 2)

Vấn đề được đặt ra: làm thế nào để truyền dữ liệu đi xa hơn và truyền được nhiều luồng dữ liệu cùng một lúc?

→ Modulation (biến đổi tín hiệu) → biến đổi digital signal thành analog signal trước khi truyền  
⇒ by using different carrier signals (frequencies) → several transmissions can happen simultaneously.

Therefore:

→ **Broadband** (dải rộng) → tín hiệu số được biến đổi (modulate) thành analog trước khi truyền  
⇒ cho phép chia đường truyền thành nhiều kênh tần số khác nhau (nhiều luồng dữ liệu có thể truyền cùng một lúc) → dùng cho truyền dẫn quãng dài như truyền hình cáp hay 4G chẳng hạn.

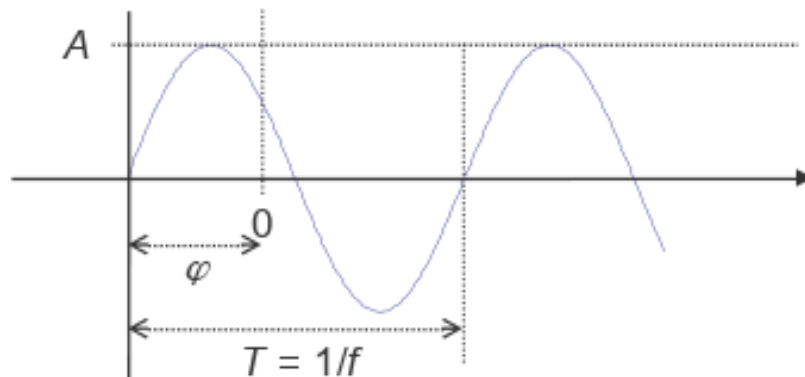
### 2.2.2.2 Principle of Modulation

Công thức electromagnetic signal (tín hiệu điện từ):

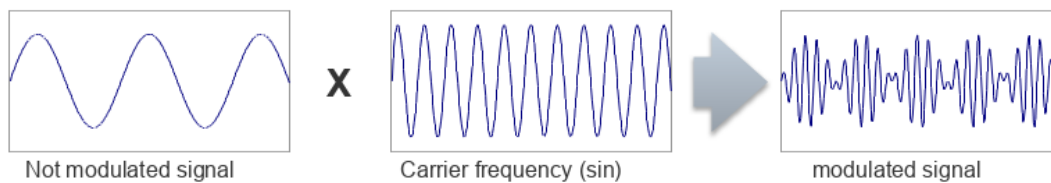
$$s(t) = A * \sin(2\pi ft + \phi)$$

Trong đó:

- $A \rightarrow$  amplitude (biên độ).
- $f \rightarrow$  frequency (tần số).
- $\phi \rightarrow$  phase (pha).
- $t \rightarrow$  time (thời gian).



$\Rightarrow$  Dữ liệu đã được biến đổi thành tần số mang (carrier frequency)  $\rightarrow$  có thể truyền đi xa hơn và nhiều luồng dữ liệu có thể truyền cùng lúc.



$\rightarrow$  **Modem: Modulation-Demodulation process** (quá trình biến đổi và giải biến đổi).

### 2.2.2.3 Amplitude Shift Keying (ASK)

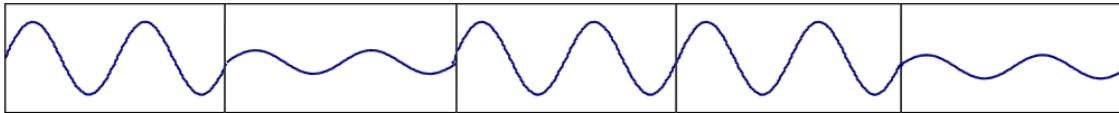
ASK: điều chế dịch biên độ (hoặc gọi là điều chế biên độ cũng được).

→ Là phương pháp điều chế cơ bản được sử dụng trong broadband (phương pháp thứ nhất).

⇒ Nơi dữ liệu được mã hóa bằng cách **thay đổi biên độ** (amplitude) của sóng mang điện từ.



Amplitude Modulation (discrete, Amplitude Shift Keying - ASK) → biến đổi biên độ ⇒ kí hiệu  $A$  từ công thức trên.



#### Nguyên lý hoạt động:

- Bit 1 → được mã hóa bằng một sóng có biên độ cao ( $A_{high}$ ).
- Bit 0 → được mã hóa bằng một sóng có biên độ thấp ( $A_{low}$ ).

→ Máy phát sóng mạnh khi gửi bit 1, và yếu khi gửi bit 0.

- **Pros:** easy to realize (dễ thực hiện) ⇒ chỉ cần thay đổi biên độ của sóng mang; giải mã ở cả 2 đầu máy phát và thu; does not need much bandwidth (không tốn nhiều băng thông) → do ASK thay đổi biên độ, tần số được giữ nguyên ⇒ nó không tốn nhiều dải tần (so với FSK sẽ nói ở dưới).
- **Cons:** nhạy cảm với noise; attenuation và noise luôn làm thay đổi biên độ của tín hiệu trong môi trường thực tế → nếu gửi bit 1 (biên độ cao) nhưng quá nhiều yếu tố gây nhiễu thì máy sẽ đọc lộn thành bit 0 (biên độ thấp).

⇒ Do đó, ASK không được sử dụng rộng rãi cho truyền dữ liệu tốc độ cao, hay ở khoảng cách xa vì dễ gây ra lỗi bit (BER cao).

Ứng dụng: thường được sử dụng trong truyền dẫn quang (optical transmission) → low noise.

#### Self-note:

→ Cứ xem ASK như nhìn dây đèn ở khoảng cách xa → đèn sáng (bit 1) và đèn tắt (bit 0) ⇒ dễ thao tác, nhưng nếu trời bị sương mù dày đặc (nhiều nhiễu) → nhìn nhầm hoặc không thấy.

#### 2.2.2.4 Frequency Shift Keying (FSK)

FSK: điều chế dịch tần số (hoặc gọi là điều chế tần số cũng được).

→ Là phương pháp điều chế sử dụng trong broadband (phương pháp thứ hai).

⇒ Nơi dữ liệu được mã hóa bằng cách **thay đổi tần số** (frequency) của sóng mang điện từ.



Frequency Modulation (discrete, Frequency Shift Keying - FSK) → biến đổi tần số ⇒ kí hiệu  $f$  từ công thức trên.



#### Nguyên lý hoạt động:

- Bit 1 → được mã hóa bằng một sóng có tần số cao ( $f_{high}$ ).
- Bit 0 → được mã hóa bằng một sóng có tần số thấp ( $f_{low}$ ).

→ Máy phát sóng nhanh khi gửi bit 1, và chậm khi gửi bit 0.

- **Pros:** ít nhạy cảm với noise hơn ASK ⇒ vì nhiễu thường làm thay đổi biên độ (độ cao) của sóng chứ không phải tần số (độ nhanh/chậm của sóng) ⇒ tín hiệu ổn định hơn ASK.
- **Cons:** lãng phí tần số (waste of frequencies) → cần dành riêng 2 khoảng tần số khác nhau cho bit 0 và bit 1; tốn nhiều băng thông (needs a lot of bandwidth) → do phải phân biệt rõ ràng giữa 2 tần số cao thấp ⇒ 2 đĩa đó nằm cách xa nhau, dẫn đến tốn nhiều tiết diện trên dải băng tần.

→ Do tốn nhiều băng thông ⇒ ít được dùng cho các ứng dụng cần tốc độ cực cao, tuy nhiên độ bền (robustness) của nó uy tín.

Ứng dụng: nguyên lý ban đầu được dùng để truyền dữ liệu qua điện thoại (phone lines).

#### Self-note:

→ Cứ xem FSK như giọng hát → nốt cao (bit 1) và nốt trầm (bit 0) ⇒ dù có đang trong phòng ồn ào (bị noise) → vẫn phân biệt được nốt cao hay thấp (khó nhầm lẫn hơn so với ASK) → tuy nhiên, phải có quãng giọng rộng (được xem như cần nhiều bandwidth) mới có thể hát được cả 2 nốt này rõ ràng.

### 2.2.2.5 Phase Shift Keying (PSK)

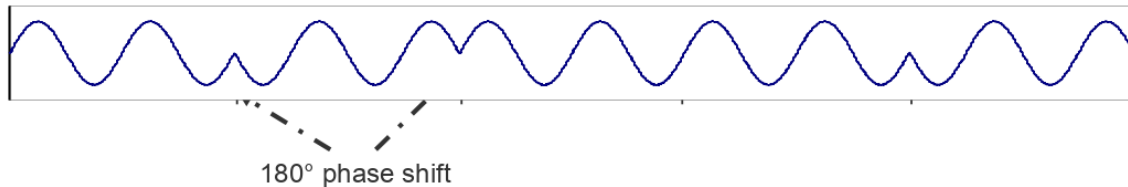
PSK: điều chế dịch pha (hoặc gọi là điều chế pha cũng được).

→ Là phương pháp điều chế sử dụng trong broadband (phương pháp thứ ba).

⇒ Nơi dữ liệu được mã hóa bằng cách **thay đổi pha** ( $\phi$  phase) của sóng mang điện từ.



Phase Modulation (discrete, Phase Shift Keying - PSK) → biến đổi pha ⇒ kí hiệu  $\phi$  từ công thức trên.



#### Nguyên lý hoạt động:

- Bit 1 → sóng bắt đầu ở một góc pha nhất định (như  $0^\circ$ ).
- Bit 0 → sóng bắt đầu ở góc pha khác (như  $180^\circ$ ).

→ Tín hiệu bị ngắt và đảo ngược trạng thái mỗi khi bit thay đổi (như hình trên, có thể thấy được ngay chỗ sóng bị đảo chiều).

- **Pros:** chống nhiễu tốt (robust against noise) ⇒ vì nhiễu thường tác động vào biên độ, ít khi làm thay đổi pha của sóng ⇒ ổn định hơn so với ASK; là giải pháp tổng thể tốt nhất → cân bằng giữa hiệu suất và chất lượng.
- **Cons:** quá trình giải điều chế phức tạp (complex demodulation process) ⇒ receiver phải cực kì chính xác để có thể phát hiện ra sự lệch pha (dựa theo góc của sóng) → đòi hỏi phần cứng đắt tiền và phức tạp hơn so với việc chỉ cần đo độ cao (như ASK) hay đo tần số (như FSK).

#### Self-note:

→ Cứ xem như PSK là 2 người chạy tiếp sức:

- Bit 1 → người đầu tiên xuất phát ngay vạch xuất phát.



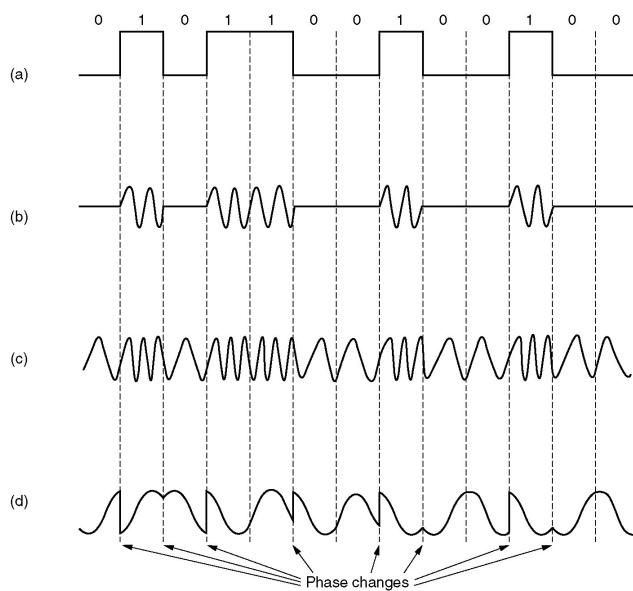
- Bit 0 → người thứ hai xuất phát ở vạch đối diện với người đầu tiên (lệch  $180^\circ$ ).

→ Dù trời mưa có to đi chẳng nữa (noise) ⇒ vẫn biết được người đang chạy xuất phát từ đâu, do vị trí xuất phát cách rất xa nhau nên không bị nhầm.

⇒ Tuy nhiên, trọng tài (receiver) phải cố nhìn mới có thể phân biệt được ai với ai (như cách máy phải chuẩn lắm mới phát hiện được, không đơn giản như ASK).

### 2.2.2.6 Overview

Finalize:



- a. Binary signal.
- b. Amplitude modulation.
- c. Frequency modulation.
- d. Phase modulation.

#### Binary signal (tín hiệu nhị phân):

- Đây là dữ liệu đầu vào (input data).
- Là các bit 0 và 1.
- Hình sóng vuông ⇒ mức cao là 1, thấp là 0.
- Là nơi điều khiển sóng mang phải thay đổi như thế nào.

#### Amplitude modulation (điều chế biên độ) - ASK:

- Binary bảo sao thì **biên độ** của sóng cũng vậy.

→ Bit = 1 ⇒ sóng dao động mạnh; bit = 0 ⇒ sóng dao động yếu.

→ Tần số và pha giữ nguyên không đổi.

### Frequency modulation (điều chế tần số) - FSK:

→ Binary báo sao thì **tần số** của sóng cũng vậy.

→ Bit = 1 ⇒ sóng bị ép lại, dao động nhanh (tần số cao); bit = 0 ⇒ sóng giãn ra, dao động chậm (tần số thấp).

→ Biên độ và pha giữ nguyên không đổi.

### Phase modulation (điều chế pha) - PSK:

→ Binary thay đổi (từ 0 → 1 hoặc 1 → 0) thì sóng phải **đảo chiều** (đổi pha).

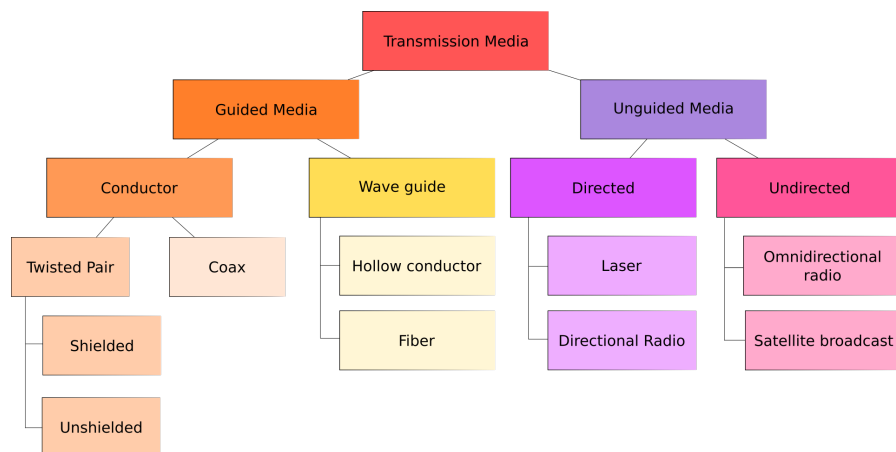
→ Mỗi khi bit thay đổi → sóng bị gãy và đi xuống (hoặc ngược lại) ⇒ tạo sự lệch pha (nhằm để phân biệt giữa 2 bit khác nhau).

→ Biên độ và tần số giữ nguyên không đổi (sóng cao bằng nhau và nhanh như nhau).

## 2.3 Transmission Media

### 2.3.1 Classification of Transmission Media

Mô hình tổng thể:



→ Chia làm 2 loại chính: **guided media** (phương tiện hữu tuyến) và **unguided media** (phương tiện vô tuyến).

### 2.3.2 Guided Transmission Media

Guided transmission media (phương tiện truyền dẫn có hướng) → tín hiệu được dẫn đường cụ thể bên trong một vật thể vật lý (như dây dẫn chẳng hạn).

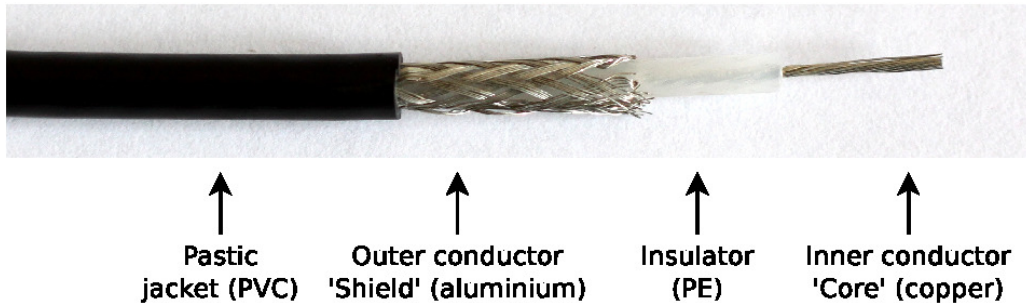
⇒ Tín hiệu không truyền bên ngoài.

Được chia làm **2 loại**:

- Conductor (dây dẫn) → dùng dòng điện để truyền tín hiệu.
- Wave guide (dẫn sóng) → dùng sóng ánh sáng (light waves) để truyền tín hiệu.

#### 2.3.2.1 Coaxial Cables (Coax Cables)

Coaxial cable (cáp đồng trục) → là loại dây dẫn hữu tuyến phổ biến (dùng cho truyền hình cáp, mạng thế hệ cũ, ...).



→ Một loại cáp đồng trục dùng để mang tín hiệu bipolar.

Self-note: tín hiệu dạng bipolar là tín hiệu điện có 3 mức điện áp: dương (+V), âm (-V) và không (0V) → thường dùng nhiều trong viễn thông và một số hệ thống truyền dữ liệu.

**Cấu tạo:**

- **Inner conductor** (dây dẫn bên trong) → thường làm bằng đồng ⇒ có nhiệm vụ mang tín hiệu điện (carries electrical signal) → đây là nơi dữ liệu chạy qua.
- **Insulation** (lớp cách điện) → làm bằng nhựa PE ⇒ ngăn cách, không cho lõi chạm vào lớp vỏ lưới bên ngoài nhằm tránh chập mạch.
- **Outer conductor/shield** (lớp vỏ dẫn điện) → được làm bằng nhôm hoặc lưới kim loại ⇒ có nhiệm vụ giữ mức điện thế đất (ground potential); bao bọc xung quanh lõi trong ngăn chặn nhiễu điện từ (noise) từ bên ngoài xâm nhập vào lõi.
- **Plastic jacket** (lớp vỏ nhựa) → bảo vệ vật lý cho sợi cáp (chống nước, trầy xước các kiểu).

### 2.3.2.2 Twisted Pair Cables

Twisted pair cable (cáp xoắn đôi) → là loại dây dẫn hữu tuyến phổ biến nhất, rẻ nhất, thường thấy dùng trong mạng LAN (Ethernet) và điện thoại bàn.

→ Dây của cáp xoắn đôi được xoắn cặp (pairwise twisted) với nhau.

**Cấu tạo cơ bản:** gồm 2 dây dẫn đồng được bọc cách điện.

**Nguyên tắc "twist":** 2 sợi dây đồng được xoắn lại như buộc tóc đuôi sam → để chống nhiễu từ (EMI - Electromagnetic Interference) và crosstalk (xuyên âm) giữa các cặp dây bên trong.

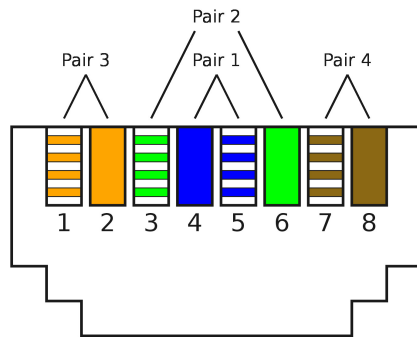
⇒ Twisted pairs are better protected against alternating magnetic fields and electrostatic interference from the outside than parallel signal wires.

Self-note:

- Xuyên âm → là sự nhiễu loạn lẫn nhau giữa các dây dẫn chạy song song ⇒ khi dòng điện chạy trong dây dẫn thứ nhất, nó sẽ tạo ra từ trường xung quanh nó → từ trường này gây hiện tượng cảm ứng sang dây kế bên mà nó nằm song song, tạo ra một dòng điện không mong muốn trên dây kia.
- EMI → là nhiễu do các nguồn bên ngoài phát ra, tác động vào sợi cáp (như sấm sét, tia lửa điện, vi sóng, thiết bị Bluetooth, ...).

**Công dụng của việc xoắn:**

- Chống nhiễu (EMI) từ tác động bên ngoài → do xoắn, sóng nhiễu sẽ tác động lên mỗi sợi dây trong cặp ngược chiều nhau ⇒ khi receiver đọc tín hiệu (bằng cách lấy hiệu giữa 2 sợi dây) thì nhiễu sẽ tự động triệt tiêu.
- Chống xuyên âm (crosstalk) từ trong → mỗi cặp dây có tốc độ xoắn khác nhau ⇒ giảm thiểu tối đa lượng nhiễu bị truyền từ cặp này sang cặp khác.



→ Tất cả các chuẩn Ethernet dùng cáp xoắn đôi đều sử dụng đầu nối 8P8C, thường gọi là RJ45 (Registered Jack).

### 2.3.2.3 Shielding of different Twisted Pair Cables

→ Cáp xoắn đôi thường được trang bị thêm lớp vỏ kim loại (metal shield) ⇒ ngăn chặn nhiễu điện từ (EMI) từ bên ngoài.

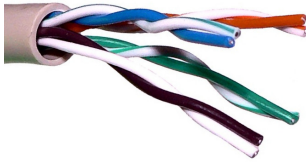
→ Các cặp dây hoặc toàn bộ sợi cáp có thể được bọc thêm lớp chống nhiễu (bằng lưới kim loại (braided shield) hoặc lá kim loại (foil shield)).

⇒ Lớp chống nhiễu chỉ hiệu quả khi hai đầu cáp được nối đất tại cùng một điện thế (ground potential).

Self-note: hai đầu cáp phải nối đất cùng một hiệu điện thế → do lớp chống nhiễu chỉ hoạt động đúng khi không có dòng điện chạy qua (đúng nghĩa cái từ "chống nhiễu").

→ Theo định luật Ohm, nếu có hiệu điện thế chênh lệch thì sẽ có dòng điện chạy qua.

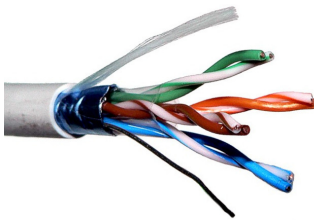
**UTP:** Unshielded Twisted Pair (cáp xoắn không vỏ bọc) → không có lớp vỏ kim loại bảo vệ bên ngoài.



→ Chỉ có dây đồng xoắn và lớp vỏ nhựa PVC.

⇒ Rẻ và phổ biến nhất, nhưng chống nhiễu kém (chỉ dựa theo nguyên lý xoắn dây để chống).

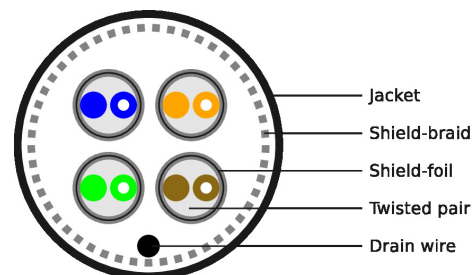
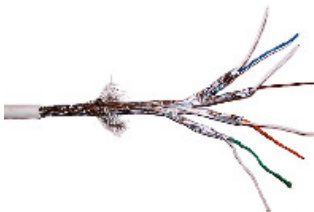
**FTP / F-UTP:** Foiled Twisted Pair (cáp xoắn đôi có lá bọc) → có lớp lá kim loại bọc bên ngoài từng cặp dây.



→ Các cặp dây được bao bọc bởi một lớp kim loại mỏng bao quanh toàn bộ bó dây (twisted pairs).

⇒ Chống nhiễu tốt hơn UTP một chút.

**SFTP:** Shielded Foiled Twisted Pair (cáp xoắn đôi có lá bọc và vỏ bọc) → có lớp lá kim loại bọc bên ngoài từng cặp dây, đồng thời có lớp lưới kim loại bọc bên ngoài toàn bộ bó dây.



- Shield-foil → từng cặp dây hoặc bó dây được bao bọc lá nhôm.
- Shield-braid → ngoài cùng có thêm một lớp lưới kim loại (braided).
- Drain wire → dây nối đất (ground wire) để dòng điện nhiễu xuống đất.

⇒ Chống nhiễu cực tốt, cáp rất to, chi phí đắt.

#### 2.3.2.4 Categories of Twisted Pair Cables

##### Nguyên tắc:

Hiệu suất của toàn bộ kết nối được quyết định từ category thấp nhất → ví dụ như dùng Cat 7 (xịn nhất) mà đi cắm vào một cái ổ siêu lỗi thời (như Cat 5 chẳng hạn) thì tốc độ nó sẽ ở mức của Cat 5 chứ không phải ngang mức cái dây.

- Cat 1/2/3/4 → đã lỗi thời. Nay chỉ dùng cho điện thoại bàn.
- Cat 5/5e → phổ biến trong các mạng LAN ngày nay; có tốc độ ổn cho các nhu cầu bình thường (100Mbps - 1Gbps).
- Cat 6/6A → có tốc độ cao, hỗ trợ lên đến 10Gbps ở khoảng cách 100m; dùng cho mạng đòi hỏi tốc độ nhanh.
- Cat 7/7A → thật ra chả có lợi gì hơn so với Cat 6A.
- Cat 8 → được thiết kế cho các data center; hỗ trợ với tốc độ cực cao nhưng chỉ với khoảng cách ngắn ( $\approx 30\text{m}$ ).

**Câu hỏi:** tại sao Cat càng lớn càng xịn hơn, nhanh hơn?

⇒ Number of twists (số vòng xoắn) per wire length (cm) and thickness of the jacket (lớp vỏ bọc bên ngoài).

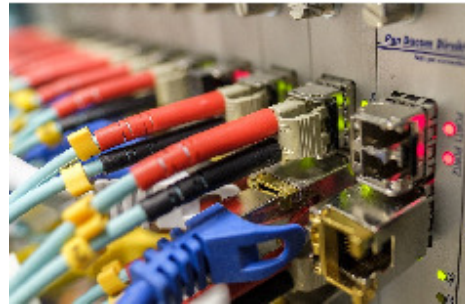
- Càng xoắn nhiều vòng ⇒ càng ít nhiễu (more twists ⇒ less interference).
- Lớp vỏ bọc càng dày ⇒ càng chống nhiễu tốt (thickness of the cladding (lớp bao phủ) ⇒ less crosstalk).

→ Cat 5/5e có 1-2 vòng xoắn trên mỗi cm. Cat 6 có 2 vòng xoắn trở lên trên mỗi cm.

→ Crosstalk là hiện tượng tín hiệu bị nhiễu lẫn nhau giữa các đường dây song song.

### 2.3.2.5 Fiber-optic Cables

Fiber-optic cable (cáp quang) → là loại dây dẫn hữu tuyến dùng sóng ánh sáng để truyền tín hiệu thay vì dây lõi đồng.



#### Nguyên lý hoạt động:

- Light source (nguồn sáng) → sử dụng đèn LED hoặc laser LED để bắn tín hiệu.
- Use wavelengths (dùng bước sóng) → sử dụng các bước sóng 850, 1300, hoặc 1550 nm.
- Môi trường truyền → ánh sáng chạy trong sợi thủy tinh với tốc độ gần 200,000 km/s.

#### So sánh với coaxial (cáp đồng trục) và twisted pair (cáp xoắn đôi):

##### Pros:

- High data rates and large distances → tốc độ cực cao và truyền được rất xa (hàng chục km không cần khuếch đại).
- No electromagnetic emission → không phát sóng điện từ ra ngoài ⇒ rất khó bị leak (bảo mật cao).
- Insensitive against electromagnetic interference (miễn nhiễm với EMI) → do nó là ánh sáng nên không sợ sấm sét hay điện lưới gây nhiễu.

##### Cons:

- Higher cost → chi phí cáp và thiết bị (như các module quang/LED) đắt đỏ hơn so với dây đồng.
- Infrastructure → không thể tận dụng được hạ tầng cũ (như dây đồng) ⇒ phải xây mới hoàn toàn.

→ Chỉ dùng khi cáp đồng không thể cung cấp đủ bandwidth hoặc khoảng cách quá xa.

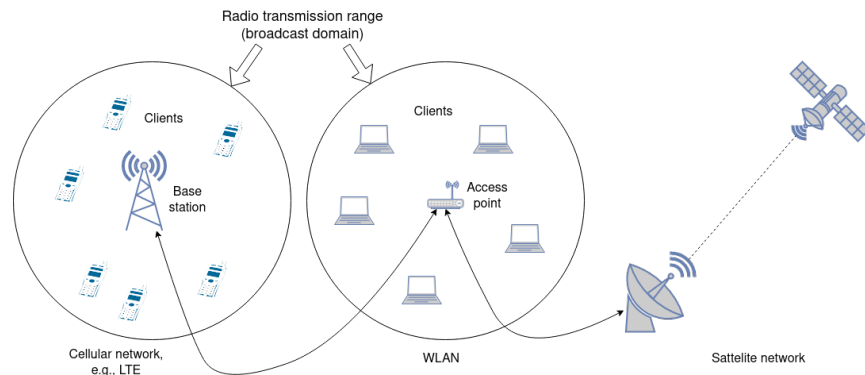
### 2.3.3 Unguided Transmission Media

Unguided transmission media (phương tiện truyền dẫn vô tuyến) → tín hiệu không chạy trong một vật thể vật lý (như dây cáp) mà truyền tự do trong không gian (như không khí, chân không).

⇒ Tín hiệu truyền tự do ra bên ngoài.

- Directed (có hướng) → tín hiệu được tập trung thành một chùm tia hẹp (beam) bắn thẳng từ máy phát đến máy thu ⇒ yêu cầu hai thiết bị phải thấy nhau (line-of-sight), không có vật cản ở giữa; ví dụ như điều khiển TV.
- Undirected (vô hướng/đa hướng) → sóng được phát mọi hướng ⇒ máy thu có thể ở bất kỳ nơi nào trong vùng phủ sóng là được; ví dụ: truyền hình vệ tinh (satellite TV).

#### 2.3.3.1 Wireless Communication



#### Đặc điểm kĩ thuật:

- Medium is an electromagnetic wave → môi trường truyền là sóng điện từ.
- Data is modulated → dữ liệu được điều chế lên sóng mang.
- Range → phụ thuộc vào công suất phát (signal power) và môi trường (environment).
- Can be directed or undirected → có thể định hướng (như sóng micro) hoặc không định hướng (như sóng radio).

**Shared medium** (môi trường truyền dẫn chung) → các tần số không thể sử dụng tùy ý muốn (cannot be used arbitrarily) mà phải được phân bổ (must be allocated).

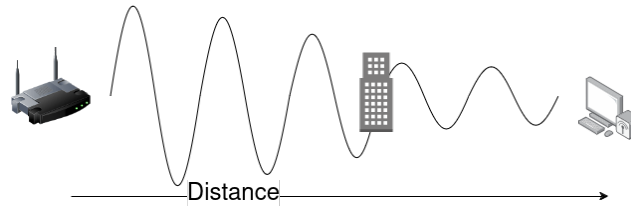


### 2.3.3.2 Challenges of Wireless Networks

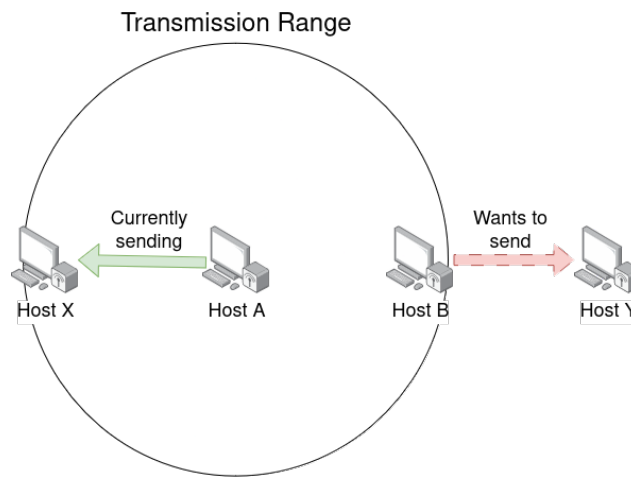
#### Fading over distance (suy giảm tín hiệu theo khoảng cách)

→ Sóng điện từ suy giảm bị yếu dần (gradually weaken) khi truyền trong không gian hoặc gặp vật cản vật lý (chẳng hạn như tường).

⇒ Càng đi xa Wifi hay vào chỗ khuất → sóng càng yếu ⇒ mạng yếu đi hoặc mất kết nối.



#### Exposed terminal problem (vấn đề "trạm lộ thiên")

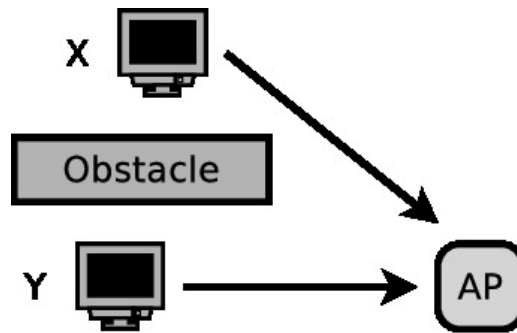


→ Máy B (theo hình trên) muốn gửi dữ liệu cho máy Y; tuy nhiên thì máy B đang nằm trong vùng phủ sóng của máy A, và máy A đang bận gửi tin cho máy X.

A node is prevented from sending packets to other nodes because of being within transmission range of a neighboring transmitter.

⇒ Máy B bị chặn không cho gửi dữ liệu đến máy Y (theo quy tắc CSMA sẽ được mention sau: trước khi muốn truyền dữ liệu thì phải xem môi trường có máy nào đang truyền không) → lãng phí băng thông, giảm hiệu suất mạng.

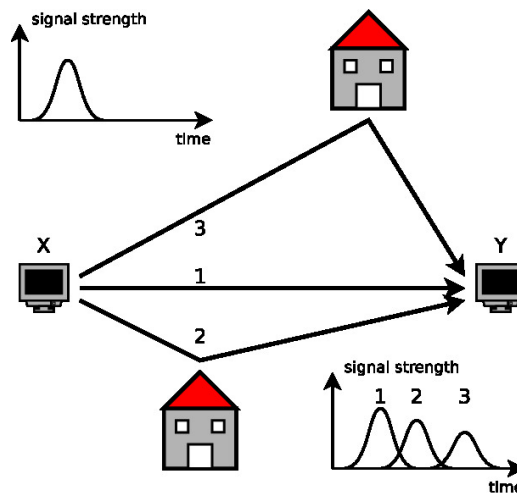
**Hidden terminal problem** (vấn đề "trạm ẩn"; invisible or hidden terminal devices)



→ 2 thiết bị (như hình trên) nằm ở hai vị trí đối diện của một bức tường (đóng vai trò là vật cản). Cả 2 thiết bị không nhìn thấy nhau, muốn nói chuyện với cục AP (Access Point) ở giữa.

⇒ X nghĩ không có dữ liệu nào đang truyền cho cục AP, và Y cũng nghĩ như thế → cả 2 tín hiệu cùng truyền lên AP ⇒ bị xung đột (collision), AP không nghe được cái nào cả.

**Multipath propagation** (đa đường truyền)



**Giải thích:** (theo hình trên)

- Một phần sóng đi thẳng đến receiver (đường số 1).
- Các phần sóng khác bị phản xạ từ tường (đường số 2 và 3) rồi mới đến receiver.

→ Do các quãng đường có độ dài khác nhau ⇒ chúng sẽ tới receiver vào những thời điểm khác nhau.

→ Sóng điện từ bị phản xạ (reflected) → các đường truyền có độ dài khác nhau từ nơi gửi đến nơi nhận ⇒ tín hiệu khó có thể giải mã do phản xạ làm ảnh hưởng đến các tín hiệu tiếp theo.

→ Nếu các objects di chuyển giữa sender và receiver ⇒ đường truyền có thể bị thay đổi.

**External interference** (nhiều từ bên ngoài)

→ **Vấn đề:** môi trường truyền phi vật lý là môi trường truyền dẫn chung (shared medium).

⇒ Các tín hiệu phải cạnh tranh nhau để sử dụng tần số (frequency), hoặc bị nhiễu từ các thiết bị điện tử khác.

**Ví dụ:**

- WLAN và Bluetooth cùng sử dụng chung tần số (same spectrum); cùng dùng chung 2.4 GHz ⇒ gây cạnh tranh đường truyền.
- Electromagnetic noise từ các thiết bị như lò vi sóng, hay motor, khi hoạt động cũng phát ra sóng gây nhiễu.

### 2.3.4 The Last Mile

→ Là đoạn kết nối cuối cùng từ trạm cung cấp (ISP - Internet Service Provider) đến người dùng (customer premises).

#### 2.3.4.1 Bridging the Last Mile

→ **Vấn đề:** làm thế nào để kết nối hàng triệu người dùng vào mạng Internet với chi phí rẻ nhất (cost-efficient)?

Common solutions:

- **DSL (Digital Subscriber Line)** → tận dụng đường dây điện thoại (access through phone lines).
- **Cable** → tận dụng tín hiệu truyền hình cáp (access through TV broadcast system).
- **3G/4G** → tận dụng các trạm phát sóng di động (access through deployed cellular networks).

Other solutions:

- **Powerline** → Internet truyền qua đường dây điện trong nhà (AC infrastructure).
- **Satellite** → dùng vệ tinh (chẳng hạn như Starlink của Elon Musk) ⇒ có thể phủ sóng vùng sâu vùng xa.
- **Wireless** → như WiMAX, directed WIFI.
- **Fiber (cáp quang)** → có tốc độ cao nhất, nhưng rất tốn kém do phải xây đường đi dây mới hoàn toàn (requires infrastructure expansion).

#### 2.3.4.2 History: The Classical Modem

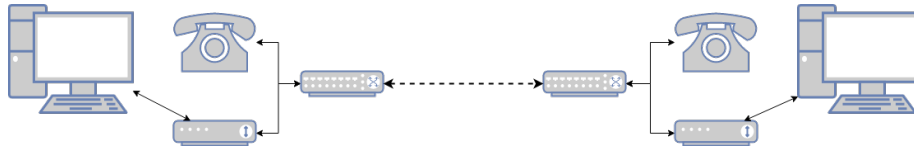
Modem (modulator-demodulator) → chuyển đổi dữ liệu số (digital data, bit 0, 1) từ máy tính sang tín hiệu tương tự (analog) để truyền đi và ngược lại.

→ **Nguyên lý hoạt động:** sử dụng đường dây điện thoại cố định (POTS - Plain Old Telephone Service) → modem truyền dữ liệu giống như cách truyền tín hiệu radio bình thường (normal audio signals) ⇒ biến các bit dữ liệu thành các âm thanh.

⇒ Modem chịu trách nhiệm xử lý thông tin tín hiệu (takes care of the signaling information).

#### Đặc điểm kỹ thuật:

- High error rate (tỉ lệ lỗi cao) → do đường dây điện thoại vốn chỉ thiết kế cho giọng nói ⇒ rất dễ bị nhiễu và dẫn đến làm sai dữ liệu.
- Speed (tốc độ) → rất chậm, tốc độ tối đa chỉ đạt cỡ 56 kbps.



→ Modem cổ điển tựa như việc đọc chính tả cho phía bên kia chép vậy.

#### 2.3.4.3 Digital Subscriber Line (DSL)

DSL (đường dây thuê bao số) → là công nghệ tiên tiến của mạng điện thoại cố định (POTS) để truyền tốc độ cao.

→ Tận dụng được hạ tầng cũ ⇒ vẫn dùng sợi cáp đồng của điện thoại bàn.

#### Điểm khác: (so với modem cổ điển)

- Modem cũ chỉ dùng một dải tần số âm thanh rất hẹp, chỉ để vừa đủ nghe tiếng nói.
- DSL khai thác toàn bộ phổ tần (whole spectrum) của sợi cáp đồng để nhồi nhét dữ liệu vào.



### Thiết bị:

- Tại gia → dùng DSL Modem để giải mã dữ liệu tải về (downstream).
- Tại nhà mạng → dùng DSLAM (DSL Access Multiplexer - thiết bị ghép kênh truy cập DSL) để gom tín hiệu từ nhà dân lại với nhau.

### Đặc điểm kỹ thuật:

- Distance dependency (phụ thuộc vào khoảng cách) → tốc độ mạng phụ thuộc vào khoảng cách từ nhà đến trạm DSLAM và chất lượng dân cáp truyền dẫn tín hiệu ⇒ nhà càng xa trạm → mạng càng chậm và chập chờn.
- VDSL2 (Very-high-bit-rate DSL 2) → là phiên bản DSL nhanh nhất, có tốc độ lên đến 100 Mbps trong điều kiện khoảng cách ngắn (< 500m) → nếu xa hơn sẽ giảm tốc độ đáng kể.

#### 2.3.4.4 Data Over Cable Service Interface Specification (DOCSIS)

DOCSIS (thông số giao diện dịch vụ dữ liệu qua cáp) → là chuẩn công nghệ để truyền Internet qua đường dây truyền hình cáp (cable TV).

⇒ Tận dụng hạ tầng → sử dụng lại hệ thống cáp đồng trục (coaxial cables) vốn dùng để phát sóng truyền hình cáp.

### Thiết bị:

- Tại gia → dùng cable modem để nhận tín hiệu (downstream).
- Tại nhà mạng → dùng hệ thống (CMTS - Cable Modem Termination System) để quản lý tín hiệu gửi lên (upstream).

→ Sử dụng các kênh tần số vô tuyến thấp (low-end radio spectrum) (6-8 MHz) để truyền dữ liệu.



### Đặc điểm kỹ thuật:

- Tốc độ bất đối xứng → tải xuống (downstream) lên đến 160 Mbps; trong khi tải lên (upstream) chỉ khoảng 20 Mbps ⇒ ưu tiên cho việc lướt web, xem phim (những công việc cần tải về nhiều hơn gửi đi).
- Kiến trúc HFC (Hybrid Fiber-Coaxial) → các mạng hiện tại dùng kiến trúc lai → cáp quang kéo đến khu phố; còn từ khu phố về nhà thì dùng cáp đồng trục ⇒ giúp tăng tốc độ.
- Shared medium (chia sẻ đường truyền) → khác với DSL (đường dây riêng), cáp đồng trục có thể chia sẻ băng thông chung đường truyền.

### 2.3.4.5 3GPP Standards

3GPP (3rd Generation Partnership Project) → là tổ chức quốc tế chịu trách nhiệm ban hành các chuẩn cho mạng di động (mobile networks) mà mình dùng hằng ngày trên điện thoại.

Các thế hệ (evolution):

- GPRS (2.5G) (General Packet Radio Service) → tốc độ rất thấp, tối đa 144 kbps → thời kì đầu của Interface di động.
- UMTS (3G) (Universal Mobile Telecommunications System) → tốc độ nhanh hơn, lên đến 42 Mbps → đủ mạnh để có thể lướt web có ảnh, nghe nhạc, bắt đầu gọi video call (nhưng chất lượng thấp, mờ).
- LTE (4G) (Long Term Evolution) → tốc độ lên đến 168 Mbps → dư sức xem HD video, livestream mượt ⇒ là chuẩn phổ biến nhất hiện nay.
- 5G → tốc độ siêu nhanh, lên đến 10 Gbps → không chỉ để lướt web, mà còn phục vụ cho mục đích liên quan đến IoT và M2M (machine to machine) → phổ biến như xe tự lái, thành phố thông minh.

## 2.4 Technologies

### 2.4.1 Standardization

**Vấn đề:** Tại sao phải cần chuẩn hóa (standardization)?

⇒ Để đảm bảo các thiết bị từ các nhà sản xuất khác nhau có thể hoạt động cùng nhau (interoperability).

→ **Tổ chức quan trọng nhất:** IEEE SA (Institute of Electrical and Electronics Engineers Standards Association) → là một cộng đồng trung lập (neutral community), không chịu sự quản lý từ bất kì quốc gia nào.

→ **Nhiệm vụ:** ban hành các quy chuẩn kỹ thuật quốc tế.

→ **Bộ tiêu chuẩn trọng điểm:** IEEE 802.

→ Là các family of standards dành cho mạng máy tính cục bộ (LAN, PAN, MAN).

- **IEEE 802.1** → chuyên về các giao thức mạng lớp cao (higher layer LAN protocols); chẳng hạn như, các vấn đề về cầu nối (bridging), quản lý mạng.
- **IEEE 802.3** → là chuẩn cho Ethernet ⇒ mọi mạng có dây đều phải tuân theo chuẩn này.
- **IEEE 802.11** → là chuẩn mạng cho WLAN (mạng không dây, Wifi) ⇒ mọi thiết bị phát Wifi đều phải tuân theo chuẩn này; như 802.11n, 802.11ac, 802.11ax.
- **IEEE 802.15** → là chuẩn cho WPAN; chẳng hạn như Bluetooth.

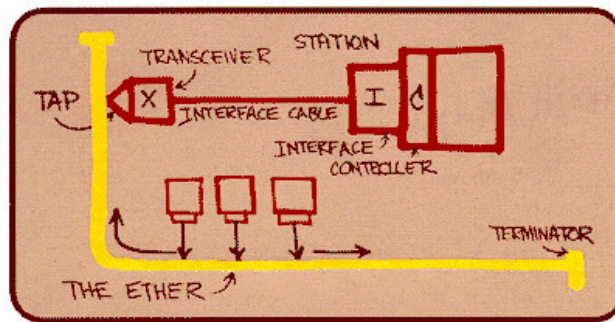
## 2.4.2 Ethernet

### 2.4.2.1 Ethernet (IEEE 802.3)

Ethernet → là chuẩn công nghệ cho mạng LAN có dây (wired LAN).

→ Được phát triển bởi Xerox PARC vào những năm 1970s.

→ Với bản vẽ này, Robert Metcalfe đã trình bày nguyên lý hoạt động của Ethernet tại Hội nghị Máy tính Quốc gia vào tháng 6 năm 1976.



#### Cách Ethernet hoạt động (xưa):

- The Ether (sợi cáp vàng) → là một sợi cáp đồng trục to và dài chạy dọc theo hành lang văn phòng ⇒ được gọi là Ether do lấy cảm hứng từ "Ether ánh sáng" (môi trường giả định truyền ánh sáng) → ý nói đến dữ liệu lan truyền đi khắp nơi.
- Tap (kẹp/mũi khoan) → để kết nối máy tính vào mạng thì người ta dùng một thiết bị kẹp chặt và siết thùng vỏ cáp để chạm vào lõi đồng bên trong (hay còn gọi là Vampire tap).
- Terminator (nút chặn cuối) → hai đầu sợi cáp phải có nút chặn để tín hiệu không bị dội ngược lại (gây nhiễu đường truyền).
- Transceiver and controller (bộ thu phát & bộ điều khiển) → thiết bị thu phát tín hiệu và bộ mạch điều khiển.

⇒ Thông số kỹ thuật của Ethernet → bao gồm cả lớp vật lý, và các dịch vụ từ lớp liên kết dữ liệu.

### 2.4.2.2 The Evolution of Ethernet

Ethernet đã trải qua nhiều lần nâng cấp theo thời gian:

- 1973 → Ethernet chính thức được phát triển tại Xerox PARC bởi Robert Metcalfe và các cộng sự → tốc độ 2.94 Mbps, sử dụng cáp đồng trục.

- 1983 → IEEE ban hành thông số kỹ thuật cho 10BASE5 (còn được gọi là thick Ethernet) → dùng cáp đồng trục to, tốc độ khoảng 10 Mbps ⇒ khó lắp đặt do cáp cứng, phải khoan lỗ (như hình trên mô tả).
- 1988 → AT&T ra mắt StarLAN 10 (the basis của 10BASE-T) → chuyển sang dùng cáp xoắn đôi (twisted pair) mềm dẻo, dễ lắp đặt hơn.
- 1990 → Ethernet trở thành công nghệ mạng LAN có dây (wired LAN) được sử dụng phổ biến trên thế giới.
- Mạng ngày nay có thể đạt đến 1 Tbps, sử dụng đa dạng phương thức truyền khác nhau như cáp đồng trục, cáp xoắn đôi, cáp quang.

### 2.4.3 Wireless Local Area Network (WLAN)

#### 2.4.3.1 WLAN (IEEE 802.11)

##### Tổng quan:

- Tên chuẩn → IEEE 802.11.
- Tên thương mại → Wifi (Wireless Fidelity) ⇒ đây là tên thường gọi chứ không phải là tên kỹ thuật.
- Là công nghệ mạng không dây phổ biến nhất hiện nay.
- Các chuẩn hiện tại có thể đạt tốc độ lên đến 7 Gbps.

##### Các mô hình kết nối (communication models):

- **Infrastructure mode** (chế độ hạ tầng) → tất cả các thiết bị (clients) kết nối vào một cục phát sóng trung tâm gọi là access point (AP) ⇒ đây là cách dùng phổ biến nhất, muốn vào mạng thì phải có cục Router/AP.
- **Ad-hoc mode** (chế độ tự phát) → các thiết bị kết nối trực tiếp với nhau tạo thành một mạng lưới (mesh network), không cần cục AP trung tâm; chẳng hạn như chuyển file qua AirDrop.

#### 2.4.3.2 The Evolution of WLAN standards

- 1995 → US FCC (Federal Communications Commission) giải phóng băng tần ISM (Industrial, Scientific, and Medical) cho công chúng miễn phí, bao gồm dải tần 2.4 GHz.
- 1997 → chuẩn 802.11 đầu tiên ban hành.



- 1999 → chuẩn 802.11b ban hành với tốc độ lên đến 11 Mbps.

→ Sau đó, các chuẩn như 802.11a hoặc 802.11n được ban hành, mở rộng dải tần 5 GHz ⇒ tránh bị tắc nghẽn.

| IEEE Standard | Maximum (gross) Data Rate | Realistic (net) Data Rate |
|---------------|---------------------------|---------------------------|
| 802.11        | 2 Mbps                    | 1 Mbps                    |
| 802.11a       | 54 Mbps                   | 20–22 Mbps                |
| 802.11b       | 11 Mbps                   | 5–6 Mbps                  |
| 802.11g       | 54 Mbps                   | 20–22 Mbps                |
| 802.11n       | 600 Mbps                  | 200–250 Mbps              |
| 802.11ac      | 1.733 Gbps                | 800–850 Mbps              |

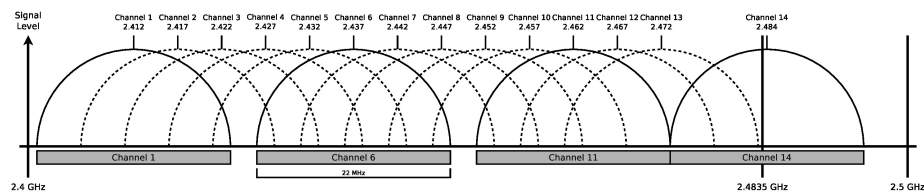
#### Self-note:

- Gross data rate (tốc độ tổng) → tốc độ truyền dữ liệu theo lý thuyết, không có nhiễu hay các tác nhân ngoại vi ảnh hưởng.
- Net data rate (tốc độ thực) → tốc độ thực tế đo được khi có ngoại vi gây ảnh hưởng đến đường truyền.

⇒ Do nhiễu, gửi kèm các header, sửa lỗi, ... nên sẽ bị giảm tốc độ so với lý thuyết → hiệu suất thực tế chỉ đạt khoảng 30-50% so với lý thuyết.

#### 2.4.3.3 Non-overlapping Channels of IEEE 802.11b

Các kênh không chồng lấn (non-overlapping channels) → là vấn đề nan giải nhất của Wifi băng tần 2.4 GHz.



- Tần số (nằm ở trục ngang) → dải tần 2.4 GHz được chia thành 11 đến 14 kênh nhỏ, mỗi kênh cách nhau 5 MHz.

- Độ rộng kênh → mỗi kênh Wifi cần một khoảng rộng 22 MHz để hoạt động.

⇒ Do các kênh được đặt quá sát nhau (cách nhau 5 MHz), trong khi cái độ rộng quá to (22 MHz) → kênh 1 sẽ lấn sang chỗ của kênh 2, 3, 4, 5.

Dải tần 2.4 GHz được chia thành các kênh (channels), mỗi kênh rộng 22 MHz nhưng chỉ cách nhau 5 MHz → các kênh nằm cạnh sẽ đè lên nhau (overlapping).

→ **Giải pháp:** chỉ có 3 kênh duy nhất là không bị chồng lên nhau là 1, 6, và 11 ⇒ trong một khu vực dày đặc người dùng, nếu người đầu tiên dùng kênh 1, còn người thứ hai dùng kênh 2 → bị nhiễu ⇒ người thứ hai phải bắt buộc đổi sang kênh 6 hoặc 11 để dùng được mạng tốt hơn.

⇒ Quy tắc 1-6-11 → để không bị chồng lên nhau, mình phải chọn các kênh cách xa nhau → chỉ có bộ ba 1-6-11 là hoàn toàn cách xa nhau, không chồng lấn lên nhau (như hình trên mô tả).

Fact: ở Nhật nó có kênh 14, nghĩa là nó có tới tận 4 kênh không chồng lấn (1, 6, 11, 14).

#### 2.4.3.4 WLAN Security: WEP and WPA

**WEP (Wired Equivalent Privacy)** → là chuẩn bảo mật đời đầu của Wifi.

⇒ Sử dụng khóa tĩnh (static keys) với độ dài 40-bit hoặc 104-bit → đã suspended do rất dễ bị hack; phần đầu gói tin (protocol header) có quy luật dễ đoán nên có thể bẻ khóa trong thời gian cực ngắn.

**WPA (Wi-Fi Protected Access)** → là chuẩn bảo mật thay thế WEP; có các phiên bản WPA 1, 2, và 3.

- WPA1 → dùng thuật toán mã hóa RC4 (Rivest Cipher 4) ⇒ hiện tại không còn an toàn nữa.
- WPA2 (phổ biến ngày nay) → dùng thuật toán AES (Advanced Encryption Standard) ⇒ rất an toàn nếu đặt mật khẩu đủ dài và khó đoán.

**Các phương thức xác thực (key distribution)** → làm sao để đưa key cho người dùng?

- PSK (Pre-Shared Key) → dùng chung một mật khẩu cho tất cả mọi người → dùng ở nhà (WPA-Personal).
- WPA-Enterprise → sử dụng một máy chủ xác thực riêng (RADIUS server - Remote Authentication Dial-In User Service) → mỗi người dùng sẽ có một tài khoản và mật khẩu riêng để đăng nhập (như Eduroam trong FRA-UAS).
- WPS (Wifi-Protected Setup) → kết nối nhanh bằng cách bấm nút trên Router hoặc nhập mã PIN, không cần gõ mật khẩu dài dòng.

Self-note: (tổng quan về thuật toán RC4 và AES)

- RC4 → tạo ra một dòng các bit ngẫu nhiên vô tận (keystream), sau đó lấy dòng này XOR với các bit dữ liệu để mã hóa ⇒ do sử dụng XOR nên nó tính reverse, nghĩa là nếu mà lấy được 2 gói tin được mã hóa bằng cùng một đoạn keystream (do nó lặp lại) → có thể dùng XOR để loại bỏ khóa và tìm ra dữ liệu gốc dễ dàng.
- AES → nó không mã hóa từng bit, mà cắt dữ liệu thành các khối vuông (ví dụ như 4x4 ô), sau đó thực hiện các phép biến đổi phức tạp: ShiftRows, SubBytes, MixColumns, lặp đi lặp lại nhiều lần. ⇒ phá vỡ hoàn toàn cấu trúc của dữ liệu gốc → một thay đổi nhỏ ở đầu vào sẽ làm thay đổi toàn bộ khối dữ liệu đầu ra (Avalanche Effect - hiệu ứng tuyết lở).

**Công thức RC4:** Ciphertext = Plaintext  $\oplus$  Keystream

## Chapter 3

# Data Link Layer - Framing and Switching

### 3.1 Framing

→ Là quá trình đóng gói và mở gói dữ liệu ở hai đầu dây:

- Receiver → cần phải tách dòng dữ liệu (bit stream) từ Physical Layer thành các khung (frame) để xử lý.
- Sender → đóng gói các tin từ Network Layer thành các khung trước khi đẩy xuống Physical Layer.

Self-note:

Tại sao cần phải framing?

→ Tầng vật lý chỉ biết truyền các tín hiệu điện/quang liên tục (bit 0 và 1) từ nơi gửi đến nơi nhận.

⇒ Không hề biết đâu là bắt đầu, đâu là kết thúc → chỉ là một dòng bit stream liên tục.

#### 3.1.1 Frame Detection

Cách xác định ranh giới của frame → giúp máy thu nhận ra điểm bắt đầu và điểm kết thúc của một khung trong bit stream hỗn tạp (→ the start of each frame needs to be marked).

**Các phương pháp để đánh dấu ranh giới các frame (frames' borders):**

- Character count in the header (đếm ký tự trong phần header).

- Byte/Character stuffing (nhồi byte/ký tự).
- Bit stuffing (nhồi bit).
- Line code violation of Physical Layer with illegal signals (vi phạm mã đường truyền).

### 3.1.1.1 Character Count in the Frame Header

#### Nguyên lý hoạt động:

Trong phần header của mỗi frame, bên gửi sẽ ghi rõ tổng số lượng bytes có trong khung đó.  
 ⇒ Receiver đọc con số này → đếm đủ số lượng bytes thì biết được điểm kết thúc của frame đó.

**Ví dụ:** giao thức DDCMP (Digital Data Communications Message Protocol) của hãng DEC (Digital Equipment Corporation) ra đời năm 1974 sử dụng phương pháp này.

|                 |                            |        |        |        |        |                    |         |                     |
|-----------------|----------------------------|--------|--------|--------|--------|--------------------|---------|---------------------|
| 8 Bits          | 14 Bits                    | 2 Bits | 8 Bits | 8 Bits | 8 Bits | 16 Bits            |         | 16 Bits             |
| SOH             | Count                      | Flags  | RESP   | NUM    | ADDR   | CRC 1              | Body    | CRC 2               |
| Start of Header | Number of bytes in payload |        |        |        |        | Checksum of header | Payload | Checksum of payload |

#### Cấu trúc khung DDCMP:

- Bắt đầu bằng SOH (Start of Header).
- Tiếp theo là field Count (14 bits) chứa số lượng byte trong khung.
- Sau đó là các field khác (như Flags, Address, ...) và cuối cùng là Payload (Body).

#### Vấn đề (potential issue):

Field Count bị nhiễu trên đường truyền.

⇒ Receiver sẽ đếm sai điểm cuối của khung hiện tại (nó sẽ tưởng dài hơn hoặc ngắn hơn so với thực tế) → mất luôn cả phần đầu của khung kế tiếp do nhầm lẫn với phần cuối khung hiện tại.

⇒ Mất đồng bộ dây chuyền (desynchronization) → receiver không thể xác định đâu là điểm bắt đầu hay kết thúc của những khung sau đó nữa → error recovery sẽ rất phức tạp.

#### Self-note:

Cứ tưởng tượng như mình là chủ kho và có ông giao hàng đến đưa mình tờ giấy ghi số lượng thùng hàng được lấy → nếu tờ giấy bị nhiễu thì mình sẽ không xác định rõ số lượng thùng hàng được lấy.

### 3.1.1.2 Control Sequences

**Chuỗi điều khiển (control sequences)** → là việc sử dụng các ký tự đặc biệt (gọi là sentinel characters - ký tự canh gác) để đánh dấu ranh giới của frame.

⇒ Thay vì đếm số lượng (như character count) → receiver sẽ truy các dòng bit liên tục cho đến khi gặp đúng ký tự đặc biệt này thì nó mới bắt đầu nhận hay kết thúc frame.

→ **Rủi ro:** nếu trong phần dữ liệu (data field) có xuất hiện các ký tự đặc biệt đó → receiver sẽ bị nhầm lẫn và tưởng đó là điểm kết thúc/bắt đầu của frame.

⇒ Giải pháp là sử dụng kỹ thuật nhồi byte/ký tự để phân biệt được.

### 3.1.1.3 Byte/Character stuffing

**Nguyên lý hoạt động:**

- Sender → chèn (stuff) thêm ký tự vào dữ liệu để đánh dấu.
- Receiver → gỡ bỏ (destuff) các ký tự thừa đó để lấy lại dữ liệu gốc.

**Nhược điểm:**

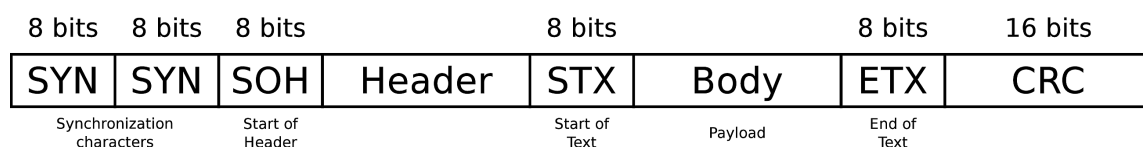
→ Phụ thuộc chặt chẽ vào bảng mã (strongly relationship with the character encoding):

- **Yêu cầu:** phương pháp này bắt buộc hệ thống phải dùng một bảng mã cụ thể (chẳng hạn như ASCII) để định nghĩa.
- **Hạn chế:** nếu mình muốn truyền dữ liệu dùng bảng mã khác (ví dụ như Unicode 16-bit) hoặc dữ liệu nhị phân thuần túy (binary data) không theo chuẩn ký tự nào → phương pháp này sẽ gặp rắc rối.

→ Các giao thức mới hơn của lớp này không còn hoạt động theo dạng byte (byte-oriented), mà chuyển sang dạng bit (bit-oriented).

⇒ Cách này cho phép sử dụng mọi kiểu mã hóa ký tự (allows using any character encoding) → không cần quan tâm ký tự điều khiển là gì.

### 3.1.1.4 Example: Byte/Character stuffing (BISYNC)



### Tổng quan:

- Tên đầy đủ → Binary Synchronous Communication (BISYNC - truyền thông đồng bộ nhị phân).
- Do IBM phát triển vào những năm 1960s.

→ Là giao thức byte (character) oriented.

### Các kí tự đặc biệt (special characters) cần nhớ:

- SYN (Synchronous Idle) → đánh dấu bắt đầu của một frame.
- SOH (Start of Header) → đánh dấu bắt đầu phần header.
- STX (Start of Text) → đánh dấu bắt đầu phần payload (phần dữ liệu chính).
- ETX (End of Text) → đánh dấu kết thúc phần payload.
- DLE (Data Link Escape) → kí tự thoát ⇒ dùng để xử lý những trường hợp khác biệt (stuffing).

### Cơ chế stuffing:

→ Khi dữ liệu người dùng (body/payload) vô tình chứa các kí tự đặc biệt ⇒ dùng DLE để có thể preserve được chúng:

- **Nếu dữ liệu có ETX** → receiver sẽ tưởng là hết frame ⇒ sender sẽ chèn thêm DLE vào trước → gửi đi DLE ETX → receiver sẽ hiểu được ETX này là dữ liệu.
- **Nếu dữ liệu có DLE** → receiver sẽ tưởng là bắt đầu kí hiệu thoát ⇒ chèn thêm DLE vào trước (nhân đôi) → gửi đi DLE DLE → receiver sẽ hiểu được DLE này là dữ liệu.

### Self-note:

→ Cứ nghĩ như trong C++, muốn in "Hello World" thì mình phải viết \"Hello World\" để thoát kí tự ngoặc kép vậy.

⇒ Nói tóm lại, nếu không muốn bị nhầm với các kí tự đặc biệt trong payload → cứ chèn thêm DLE vào trước là xong.

### 3.1.1.5 Bit Stuffing

→ Để khắc phục nhược điểm của việc phụ thuộc vào bảng mã trong phương pháp byte/character stuffing.

⇒ Bit stuffing hoạt động ở mức độ bit (bit-oriented - 0 và 1) → dùng được cho mọi loại dữ liệu (chẳng hạn như văn bản, video, zip file) mà không cần quan tâm đến bảng mã.

#### Ưu điểm:

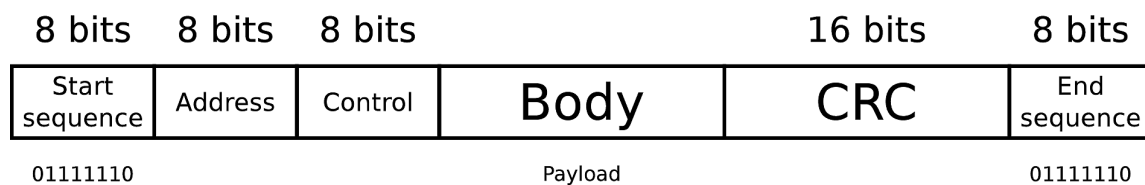
- Đảm bảo được chuỗi bắt đầu/kết thúc không xuất hiện trong payload (sẽ giải thích ở mục dưới).
- Mọi character encoding đều có thể sử dụng được phương pháp này → do làm việc ở mức bit (0 và 1) ⇒ không quan tâm dữ liệu là văn bản ASCII, Unicode, hay file ảnh, video, ...

### 3.1.1.6 Example: Bit Stuffing (HDLC)

**Giao thức sử dụng:** PPP (Point-to-Point Protocol).

→ Là giao thức phổ biến trên LLC (Logical Link Control).

⇒ Triển khai HDLC (High-Level Data Link Control) → quy định rằng mỗi frame bắt đầu và kết thúc bằng dãy bit 01111110 (gọi là Flag Sequence).



#### Nguyên lý hoạt động:

- **Sender:** quét dòng dữ liệu từ Network Layer → khi phát hiện 5 bit 1 liên tiếp (5 consecutive 1s) ⇒ tự động chèn (stuff) thêm 1 bit 0 vào ngay sau dãy đó ⇒ phá vỡ chuỗi để không bao giờ tạo thành 6 số 1 liên tiếp (trùng với flag).
- **Receiver:** quét dòng bit nhận được từ Physical Layer → khi phát hiện 5 bit 1 liên tiếp ⇒ kiểm tra bit tiếp:
  - Theo sau là bit 0 → xác định được đây là bit được stuff vào ⇒ xóa (destuff/remove) bit 0 đó để lấy lại dữ liệu gốc.
  - Theo sau là bit 1 (nghĩa là thành 6 bit 1 - 01111110) → đây là flag kết thúc/bắt đầu frame.



### Self-note:

Trong payload, nếu có xuất hiện bao nhiêu số 1 đi chẳng nữa, thì sender sẽ luôn chèn thêm 0 vào sau mỗi dãy 5 số 1 liên tiếp → đảm bảo không bao giờ có 6 số 1 liên tiếp xuất hiện trong payload.  
→ Nhưng nếu gặp 6 số 1 liên tiếp ⇒ chắc chắn đó là flag kết thúc/bắt đầu frame.

### **Ví dụ minh họa:**

Giả sử sender cần gửi dữ liệu sau (chưa có framing):

... 0 0 0 1 1 1 1 1 1 1 0 0 ...

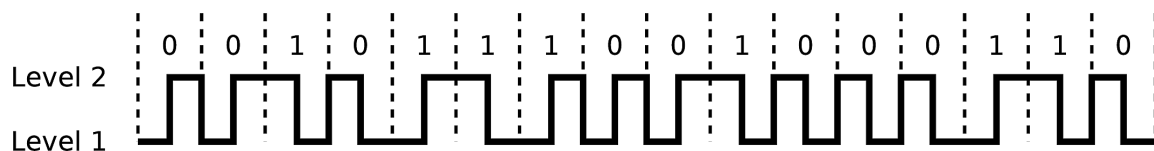
- **Sender:** tách 5 bit 1 ra thành 000 11111 110 → gửi đi: 000 11111 **0** 110.
- **Receiver:** nhận được 000 11111 **0** 110 → xóa bit 0 thừa → lấy lại dữ liệu gốc: 000 11111 110.

### **3.1.1.7 Line Code Violation**

→ Thay vì chèn thêm các kí tự vào dữ liệu (để làm tăng dung lượng) → phương pháp này tận dụng physical rules của tín hiệu để đánh dấu.

⇒ Cố tình tạo ra các tín hiệu bất hợp lệ (illegal signals) không xuất hiện trong dữ liệu chuẩn  
→ làm dấu hiệu bắt đầu/kết thúc frame.

**Ví dụ:** Token Ring (dùng differential Manchester encoding).



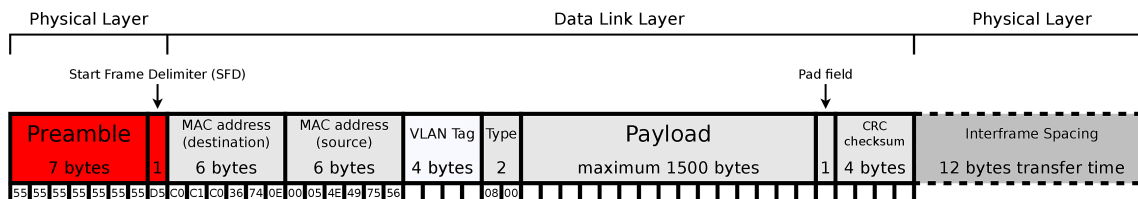
- **Valid Signal** → trong mã differential Manchester, **bắt buộc** phải có một sự thay đổi mức điện áp (đảo chiều tín hiệu) ở giữa mỗi chu kỳ bit ⇒ dù là bit 0 hay 1, điện áp cũng phải nhảy (lên hoặc xuống).
- **Violation** → để đánh dấu frame thì sender sẽ cố tình giữ nguyên điện áp (giữ cao hoặc giữ thấp) trong suốt chu kỳ bit mà không đảo chiều ⇒ receiver sẽ phát hiện ra tín hiệu này → biết được đây không phải là dữ liệu mà là delimiter (một cái dấu phân cách).

Timing diagram showing two signals, Level 1 and Level 2, over time. Level 2 is a clock signal with periods labeled J, K, 0, J, K, 0, 0, 0. Level 1 is a red signal that is high during the J and K periods of Level 2.

- Các frame bắt đầu và kết thúc bằng một byte đặc biệt gọi là starting/ending delimiter.
- Trong byte này chứa 4 lần trái quy luật (4 line code violations).

→ Số 0 đứng ở giữa J và K mà không phải gửi những dải J hay K liên tiếp  $\Rightarrow$  giúp receiver lấy lại nhịp clock trước khi nhận bit vì phạm tiếp theo  $\rightarrow$  làm như vậy để tránh bị mất tín hiệu do đảo chiều quá lâu.

### 3.1.2.1 Preamble and SFD



- **Độ dài:** 7 bytes.
- **Cấu trúc bit:** một chuỗi xen kẽ 1 và 0 lặp đi lặp lại dưới dạng 10101010 (lặp 7 lần - 1 byte = 8 bits) (→ dưới dạng hex, theo LSB first là 0x55).

⇒ Dùng để báo cho receiver biết rằng sắp có frame đến; giúp clock của receiver khớp nhịp hoàn toàn (clock synchronization) với tốc độ của sender (bit synchronization).

### SFD (Start Frame Delimiter - flag báo bắt đầu frame):

- **Độ dài:** 1 byte.
- **Cấu trúc bit:** giống như preamble nhưng thay đổi ở 2 bit cuối cùng, dưới dạng 10101011 (→ dưới dạng hex, theo LSB first là 0xD5).

⇒ Báo hiệu cho receiver biết rằng phần tiếp theo là dữ liệu thật sự của frame (theo hình là MAC address tiếp theo) → đánh dấu sự bắt đầu chính thức của Ethernet Frame.

### Self-note:

Tại sao lại là 0x55 và 0xD5 (LSB first)?

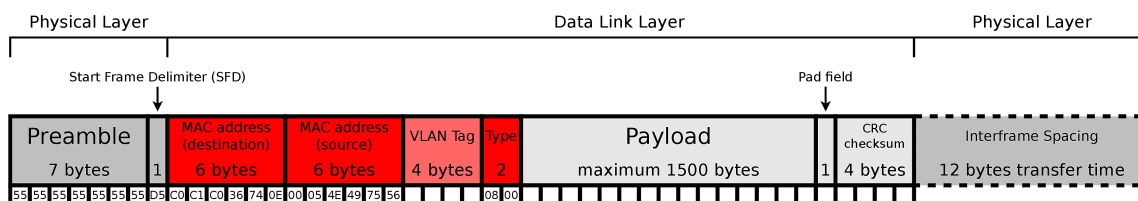
→ Do Ethernet hoạt động theo nguyên tắc LSB first (gửi từ đuôi lên đầu) ⇒ nghĩa là bit nào nằm ở hàng đơn vị (bên phải ngoài cùng) sẽ được bắn đi trước.

⇒ Trong bộ nhớ, máy tính phải lưu là 0x55 để khi đi ngược lại thì nó sẽ thành 10101010.

→ SFD với preamble chỉ khác nhau ở 2 bit cuối hòng để phân biệt được trước khi bắt đầu frame chính thức.

→ Preamble có tận 7 bytes để đảm bảo receiver có đủ thời gian đồng bộ xung nhịp với sender → tránh bị lệch pha (phase shift) khi truyền dữ liệu tốc độ cao ⇒ khi nó đã đồng bộ rồi thì chỉ cần thêm 1 byte SFD nữa để xác nhận bắt đầu frame.

### 3.1.2.2 Addresses and VLAN Tag



→ Sau khi pass SFD ⇒ 12 bytes tiếp theo sẽ đọc destination MAC address và source MAC address (mỗi cái 6 bytes).

### Destination MAC Address:

- **Độ dài:** 6 bytes.

- **Vị trí:** đứng trước source MAC address.

⇒ Chứa địa chỉ MAC address của **receiver** (hoặc địa chỉ broadcast `FF:FF:FF:FF:FF:FF` nếu gửi cho tất cả).

#### Source MAC Address:

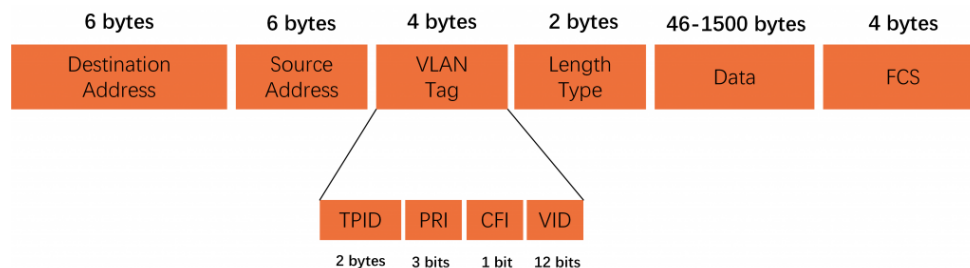
- **Độ dài:** 6 bytes.
- **Vị trí:** đứng sau destination MAC address.

⇒ Chứa địa chỉ MAC address của **sender** (để bên nhận biết ai gửi để trả lời hoặc xử lý).

#### VLAN Tag (optional):

→ VLAN (Virtual Local Area Network) → là một mạng LAN ảo được tạo ra bên trong một mạng LAN vật lý để phân đoạn lưu lượng mạng.

⇒ VLAN tag chỉ được giới thiệu trong IEEE 802.1Q hoặc IEEE 802.1ad.



#### Cấu trúc VLAN Tag:

- **Độ dài:** 4 bytes (= 32 bits).
- **Vị trí:** đứng sau source MAC address.

→ Trong VLAN tag: (các thông tin khác có thể xem hình trên)

- VLAN ID → dài 12 bits ⇒ có thể xác định được tới 4096 VLANs khác nhau.
- Priority information → dài 3 bits ⇒ dùng cho độ ưu tiên; ví dụ như voice được ưu tiên đi trước chẳng hạn.

### Field Type:

- **Độ dài:** 2 bytes.
- **Vị trí:** đứng sau source MAC address (hoặc sau VLAN tag nếu có).

⇒ Xác định protocol nào đang được sử dụng ở network layer (chẳng hạn như: 0x800 cho IPv4, 0x86DD cho IPv6, 0x806 cho ARP).

→ Khi máy tính nhận được frame → sau khi bỏ lớp Ethernet đi (MAC header) ⇒ nó sẽ còn lại phần payload.

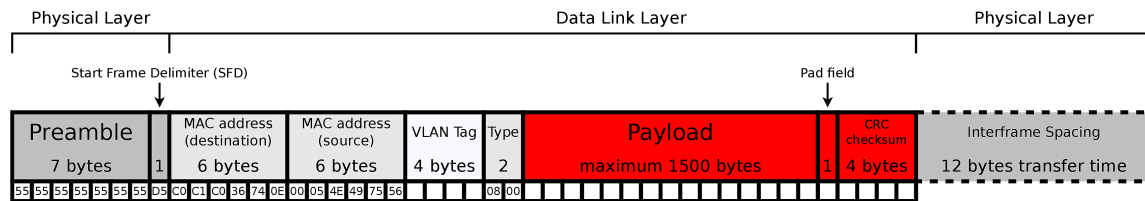
⇒ Cần biết phần này là gì (IPv4 hay ARP chẳng hạn) → dựa vào đó để xác định cách xử lý gói tin cho đúng.

### Self-note:

→ 2 bytes đầu ở VLAN tag là TPID (Tag Protocol Identifier) → thường mang giá trị là 0x8100.

⇒ Đây là mã báo hiệu cho switch biết gói tin này có chứa VLAN tag.

### 3.1.2.3 Frame Size and Checksum



### Payload:

→ Là phần dữ liệu thực sự được chuyển từ tầng trên xuống (chẳng hạn như IP packet từ network layer).

- MTU (Maximum Transmission Unit) → kích thước tối đa của payload là 1500 bytes ⇒ nếu dữ liệu lớn hơn thì sẽ bị cắt nhỏ (fragmentation).

### Pad field:

→ Dùng để tăng độ dài khung lên mức tối thiểu.

⇒ Nếu payload quá nhỏ (không đủ tạo thành khung tối thiểu) → máy sẽ tự động thêm các bit không có nghĩa (pad) vào sau payload để cho đủ kích thước.



- Mạng có tốc độ 10 Mbps → IFG là 9.6 microseconds.
- Mạng có tốc độ 100 Mbps → IFG là 0.96 microseconds.
- Mạng có tốc độ 1 Gbps → IFG là 96 nanoseconds.

→ Để receiver có thể recovery → sau khi nhận xong một gói tin, receiver cần khoảng thời gian ngắn để xử lý (chẳng hạn như chuyển từ bộ nhớ đệm vào bộ nhớ chính, kiểm tra CRC) → setup lại để đón nhận gói tin tiếp theo.

⇒ Tránh dính nhau giữa các frame liên tiếp → phân tách rõ ràng đâu là frame trước, đâu là frame sau.

Self-note:

⇒ Thời gian nghỉ thực tế nhanh hay chậm phụ thuộc vào tốc độ mạng.

$$\text{IFG time (thời gian nghỉ)} = \frac{96 \text{ bits}}{\text{Link Speed (tốc độ mạng - bps)}}$$

→ Nếu giảm IFG thì:

- **Pros:** tăng data rate ⇒ ít thời gian nghỉ → có nhiều thời gian truyền dữ liệu hơn.
- **Cons:** có thể không detect được frame border (may become impossible) ⇒ receiver không phân biệt được đâu là hết frame trước, đâu là đầu frame sau → the number of errors may rise.

## 3.2 Addresses

### 3.2.1 MAC Address

→ Các giao thức của Data Link Layer quy định cách định dạng (format) của physical network addresses.

→ Physical network address thường được gọi là MAC address (Media Access Control address).

⇒ MAC address theo chuẩn IEEE 802 có độ dài 48 bit (6 bytes).

**Định dạng:** Viết dưới dạng hexadecimal, separated bởi dấu – (dash) hoặc dấu : (colon).

**EUI Addresses:**

- EUI → nghĩa là Extended Unique Identifier.

→ Đây là numbering space do IEEE quản lý để tạo ra các địa chỉ MAC.

- **Phân loại:** có 2 loại EUI phổ biến dựa trên độ dài bit:
  - **EUI-48 (48 bits)** → như Ethernet, WLAN (Wifi), Bluetooth.
  - **EUI-64 (64 bits)** → dài hơn 8 byte; như Firewire, 6LoWPAN, Zigbee.

### 3.2.2 Broadcast and Multicast MAC Address

#### Individual/Group (I/G) bit:

- Là bit thấp nhất (LSB - Least Significant Bit) của byte đầu tiên trong MAC address.
- ⇒ Xác định xem gói tin gửi cho một người (unicast) hay cho nhiều người (multicast/broadcast).

#### MAC broadcast address:

- Trong IEEE 802 networks → tất cả 48 bits đều mang giá trị 1.
- Hex representation: FF-FF-FF-FF-FF-FF (: hay – đều như nhau, như đã nói trên).

#### Cơ chế hoạt động:

→ Các frame có I/G bit set (nghĩa là bit I/G bằng 1) chỉ cần gửi đi một lần → nhưng sẽ được Switch chuyển tiếp ra nhiều port (hoặc tất cả các port).

#### Self-note:

Trong chuyên ngành: "bit set" nghĩa là bit được đặt thành giá trị 1, còn "bit cleared" nghĩa là bit được đặt thành giá trị 0.

⇒ Theo câu trên, bit có trọng số thấp nhất (LSB) của byte đầu tiên trong MAC address mà bằng 1 → là frame này gửi cho nhiều người (multicast/broadcast); còn nếu bằng 0 → là gửi cho một người (unicast).

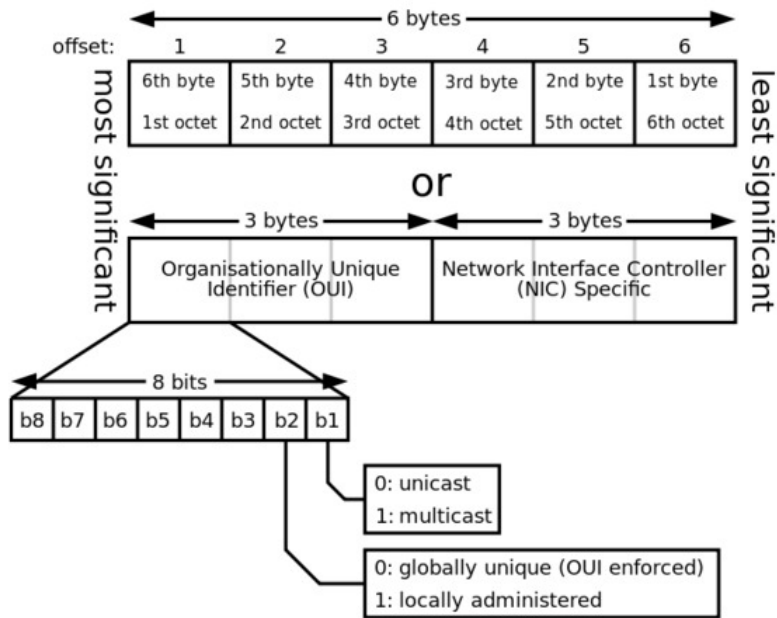
#### Ví dụ:

- 4A: . . . → Đầu là 0100 1010 → I/G bit = 0 ⇒ unicast.
  - 4B: . . . → Đầu là 0100 1011 → I/G bit = 1 ⇒ multicast.
- ⇒ Chỉ có broadcast là full 1s → FF:FF:FF:FF:FF:FF.

### 3.2.3 Uniqueness of MAC Addresses

- Mỗi thiết bị mạng phải có một MAC address duy nhất và được gán cứng vĩnh viễn.
- ⇒ Nhưng trên thực tế thì người ta vẫn có thể thay đổi được MAC address (gọi là spoofing) bằng phần mềm.





**Cấu trúc:** (như hình trên mô tả)

- 24 bit đầu - OUI (Organizationally Unique Identifier) → là mã định danh của nhà sản xuất, do IEEE cấp phát.
- 24 bit sau - vendor specific → do nhà sản xuất tự đánh số cho từng thiết bị; cho phép tạo ra khoảng 16 triệu ( $2^{24}$ ) địa chỉ cho mỗi mã OUI.

| MAC addresses     | Manufacturer    | MAC addresses     | Manufacturer |
|-------------------|-----------------|-------------------|--------------|
| 00-20-AF-xx-xx-xx | 3COM            | 00-0C-6E-xx-xx-xx | Asus         |
| 00-00-0C-xx-xx-xx | Cisco           | 08-00-2B-xx-xx-xx | DEC          |
| 00-01-E6-xx-xx-xx | Hewlett-Packard | 00-02-B3-xx-xx-xx | Intel        |
| 00-04-5A-xx-xx-xx | Linksys         | 00-04-E2-xx-xx-xx | SMC          |
| 00-03-93-xx-xx-xx | Apple           | 00-50-8B-xx-xx-xx | Compaq       |
| 00-02-55-xx-xx-xx | IBM             | 00-09-5B-xx-xx-xx | Netgear      |

### 3.2.4 Security Aspects of MAC Addresses

#### MAC filters:

→ Thường được sử dụng trong mạng Wifi (WLAN) để bảo vệ AP (Access Point).

⇒ Administrator sẽ tạo ra một danh sách các MAC address được phép truy cập vào mạng (whitelist) → chỉ những thiết bị có MAC address trong danh sách này mới được phép kết nối.

#### Vấn đề:

→ Mức độ an toàn thấp.

⇒ Mặc dù MAC là physical address → vẫn có thể hoàn toàn bị thay đổi bằng phần mềm (gọi là MAC spoofing như nãy nói trên).

#### Cách đổi MAC trong môi trường Linux:

- Đọc MAC hiện tại của mình: `ip link` hoặc `ifconfig`.
- Đọc MAC thiết bị khác (chủ yếu là Routers): `ip neigh`.
- Set MAC address mới: `ip link set dev <Interface> address <MAC Address>`.

#### Self-note:

Thao tác MAC spoofing:

- Hacker nghe lén (sniff) mạng Wifi và bắt được MAC address hợp lệ đang kết nối (chẳng hạn như của mình).
- Hacker dùng phần mềm thay đổi MAC address của máy nó sang địa chỉ máy mình.
- Access point tưởng hacker là mình → cho phép nó kết nối vào mạng.

⇒ MAC address không được thiết kế để làm nhiệm vụ bảo mật → không bao giờ dùng nó để xác thực người dùng.

## 3.3 Switching

### 3.3.1 Devices

#### 3.3.1.1 Devices of the Data Link Layer: Bridges

#### Remember:

→ Các thiết bị của physical layer (như repeater, switch, hub) làm tăng độ dài của physical network.

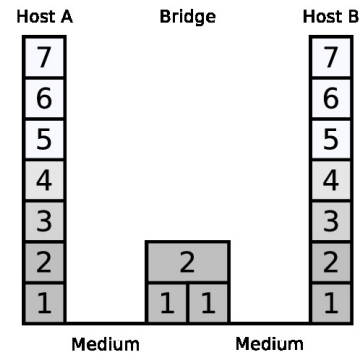
⇒ Tín hiệu có thể truyền xa hơn trong mạng mà không bị mất dữ liệu hoặc giảm chất lượng.

→ **Vấn đề:** ở physical layer, các thiết bị như repeater hoặc hub chỉ đơn thuần là nối dài dây cáp để có thể truyền đi xa hơn → nó không hiểu gói tin là gì ⇒ không thể nào xử lý hơn thế được.

⇒ Để kết nối các physical network khác nhau (chẳng hạn như nối dây mạng với dây Wifi, hoặc nối 2 đoạn LAN lại) → cần bridge ở data link layer để xử lý.

### Bridge:

- **Đặc điểm:** một bridge có ít nhất 2 cổng (ports).
- **Phân loại:**
  - Switch → là bridge có nhiều hơn 2 cổng ( $> 2$  ports).
  - Virtual bridge → là bridge ảo được tạo ra từ phần mềm (chẳng hạn như khi dùng virtual machine hoặc nối các mạng ảo trên Linux).
  - WLAN bridge → nối các máy tính dùng dây (như desktop) với mạng Wifi không dây.



### Cơ chế hoạt động:

- Simple bridge → forward tất cả các gói tin nhận được (giống như Hub nhưng khác ở chỗ có check lỗi).
- Kiểm tra lỗi → bridge sẽ kiểm tra checksum (FCS - Frame Check Sequence) của gói tin ⇒ nếu gói tin bị lỗi → nó sẽ loại ngay chứ không chuyển tiếp.
- Transparent → các máy tính (hosts) không cần biết có bridge tồn tại hay không → vẫn forward gói tin như bình thường.

#### 3.3.1.2 Example: WLAN Bridge



### Chức năng:

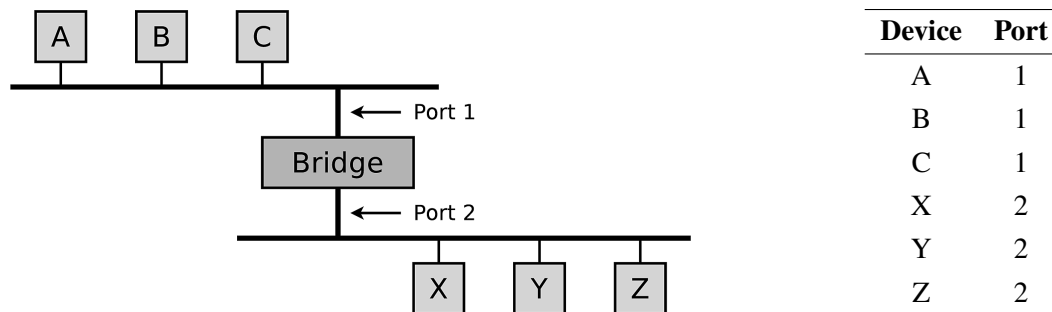
- Kết nối một mạng dùng dây (cable-based network) với một mạng không dây (wireless network).
- ⇒ Tích hợp các thiết bị chỉ có cổng mạng dây (như RJ45 jacks - như network printers, desktops, gaming consoles) vào mạng Wifi (WLAN) mà không cần phải đi dây cáp.

### Self-note:

- WLAN bridge giống như một cái card mạng dành cho các thiết bị cũ.
- ⇒ Cắm dây mạng từ thiết bị đó vào cục WLAN bridge → cục đó sẽ phát sóng Wifi để kết nối vào mạng.

## 3.3.2 Forwarding

### 3.3.2.1 Learning Bridges



**Vấn đề:** khi một bridge/switch mới khởi động, nó hoàn toàn không biết máy nào đang cắm vào cổng nào.

- ⇒ Thay vì administrator phải làm thủ công để configure → bridge có khả năng tự học vị trí của các thiết bị.
- Theo như hình trên, nếu một frame từ host B sang host A → nó không nhất thiết phải forward qua bridge để sang port 2.

### Cách bridge học:

- Nó học bằng cách dựa vào địa chỉ người gửi (sender address) trong frame nó nhận được.
- ⇒ Nếu host A gửi một frame đến host khác → bridge lưu thông tin frame nhận được từ host A là port 1 → sau đó ghi thông tin này vào bảng chuyển tiếp (forwarding table).

### Đặc điểm của forwarding table:

- Bootup → bảng hoàn toàn empty.
- Update → bảng được cập nhật dần theo thời gian mỗi khi có dữ liệu gửi qua.

→ Mỗi entry có thời hạn nhất định (→ Time To Live - TTL) ⇒ nếu một máy không gửi dữ liệu gì trong một khoảng thời gian dài → bridge sẽ xóa tên nó ra khỏi bảng để tiết kiệm bộ nhớ (optimization) và cập nhật nếu máy đó đổi vị trí (→ đây không phải là problem).

⇒ Nếu không tìm thấy address tồn tại trong bảng → frame sẽ được gửi ra tất cả các cổng (gọi là flooding).

### 3.3.2.2 Forwarding Strategies

*\*List strategies trong switch, chứ không phải là step để tạo thành một strategy.*

#### Store-and-Forward (lưu và chuyển tiếp):

→ Switch sẽ nhận toàn bộ frame và lưu vào bộ nhớ đệm (buffer) → tính toán lại checksum (CRC - Cyclic Redundancy Check) để kiểm tra lỗi ⇒ nếu gói tin không bị lỗi thì chuyển đi tiếp, còn nếu có thì loại ngay.

⇒ Cách này an toàn nhất do không chuyển gói tin rác → tuy nhiên có latency cao nhất do phải chờ nhận hết frame.

#### Cut-Through (chuyển mạch liền):

→ Switch bắt đầu chuyển tiếp frame đi ngay lập tức sau khi nó đọc được destination address → vị trí address này nằm ở ngay đầu frame ⇒ tốc độ xử lý cực nhanh.

⇒ Cách này có latency thấp → nhưng rủi ro cao do có thể chuyển tiếp cả những frame lỗi vì chưa pass cái đuôi có checksum.

#### Adaptive Cut-Through (chuyển mạch thích ứng):

→ Đây là sự kết hợp thông minh giữa 2 phương pháp trên:

- Bình thường → nó sẽ chạy ở chế độ Cut-Through cho nhanh.
- ⇒ Nếu nó phát hiện tỉ lệ lỗi trên đường truyền vượt quá một ngưỡng nhất định (certain error threshold) → tự động chuyển sang Store-and-Forward để đảm bảo an toàn.

### Fragment-Free Cut-Through (chuyển mạch không phân mảnh):

→ Switch chờ nhận 64 bytes đầu tiên của frame rồi mới chuyển đi ⇒ đây là kích thước tối thiểu của một frame Ethernet để phát hiện collision do hầu hết các lỗi này đều xảy ra trong 64 bytes đầu này.

⇒ Cân bằng giữa tốc độ, nhanh hơn Store-and-Forward; an toàn hơn do lọc được các collision fragments.

### 3.3.3 Loops

### 3.3.3.1 Loops on the Data Link Layer

**Nguyên nhân:**

Khi thiết kế mạng  $\rightarrow$  người ta đôi khi cố tình tạo ra các đường kết nối dự phòng (redundant connections)  $\Rightarrow$  nếu một dây cáp bị đứt thì vẫn còn đường khác để truyền đi.

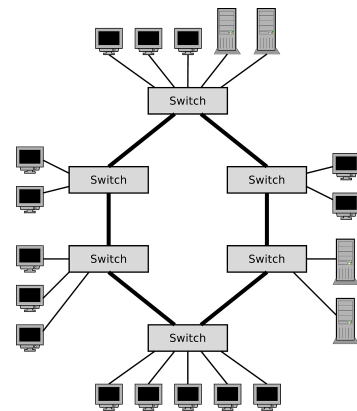
⇒ Chính đường dự phòng này tạo ra các loops về mặt vật lý.

→ Khác với gói tin IP (có TTL để tự hủy khi quá lâu) → gói tin Ethernet không có cơ chế TTL  
⇒ một khi gói tin đã rơi vào vòng lặp → nó sẽ loop vĩnh viễn.

**Vấn đề:** tại một thời điểm  $\rightarrow$  chỉ được tồn tại một đường truyền duy nhất đến mỗi thiết bị trong mạng  $\Rightarrow$  loops sẽ làm phá vỡ nguyên tắc này.

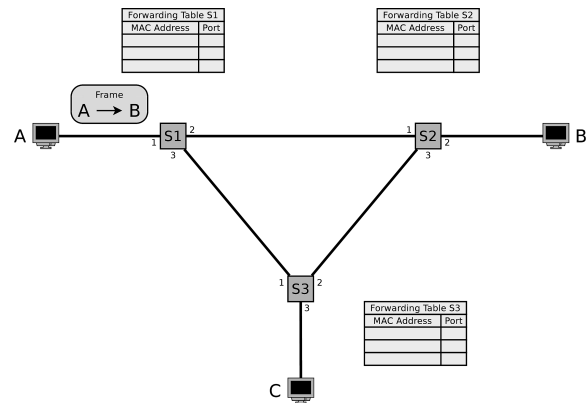
### Hậu quả:

- Làm nhân bản frame (frames get duplicated) → các gói tin bị sao chép và đến đích nhiều lần.
- Giảm hiệu năng (reduce the performance) hoặc sập mạng (network failure) → loops có thể làm giảm tốc độ mạng, hoặc tệ hơn là dẫn đến sập toàn bộ hệ thống.

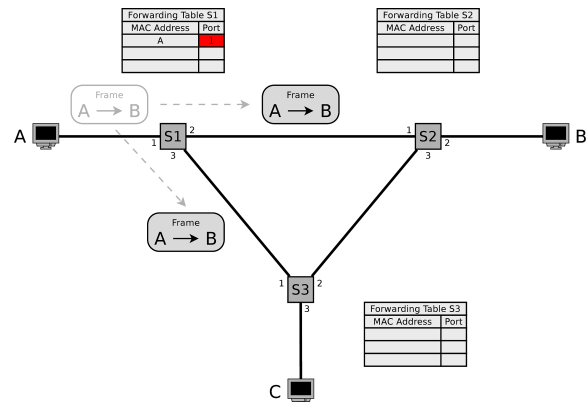


### 3.3.3.2 Example of Loops in a LAN

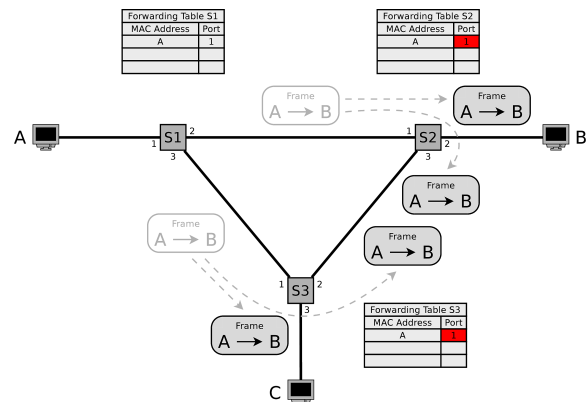
- Mạng LAN có 3 switch (S1, S2, S3) được kết nối với nhau tạo thành loop ở data link layer.
- Forwarding table của mỗi switch đang trống do chưa học được gì.
- Máy A (đang nối với S1) muốn gửi tin cho máy B (đang nối với S2).



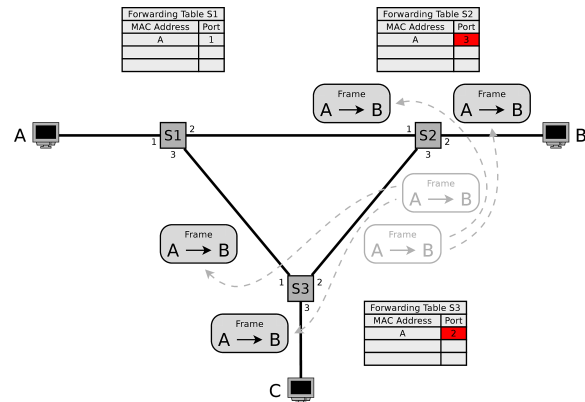
- Frame đi qua S1 → S1 ghi nhớ địa chỉ của A vào forwarding table.
- Do S1 chưa biết B ở đâu → nó flood gói tin ra các cổng còn lại (trừ cổng S1) ⇒ gói tin đi tới S2 và S3.



- S2 và S3 đều nhận được frame từ S1 sang → cả hai đều lưu địa chỉ của A vào bảng "A ở port 1".
- Cả S2 và S3 đều vẫn chưa biết máy B ở đâu → tụi nó lại tiếp tục flood tiếp các gói tin ra các cổng khác ngoại trừ cổng nhận từ S1.

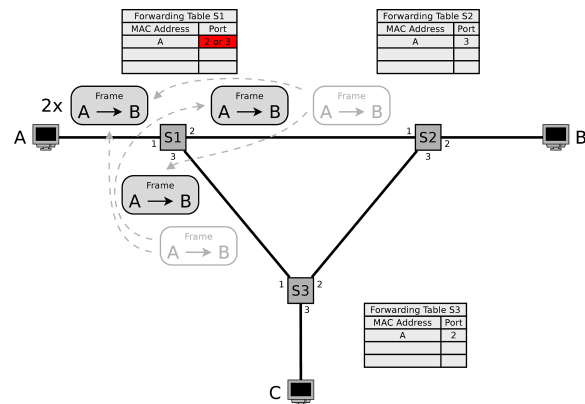


- Các copies của frame lại tiếp tục đi qua S2 và S3.
- S2 và S3 cập nhật lại forwarding table của chính nó.

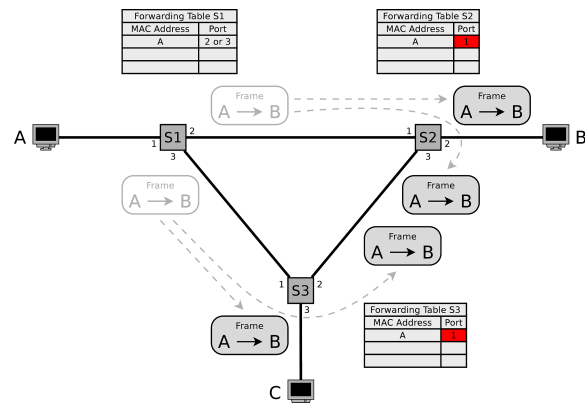


→ Do S2 đã biết port 3, và S3 đã biết port 2  $\Rightarrow$  2 tại nó sẽ không gửi cho port đã biết và chính nó.  
 $\Rightarrow$  Tại nó gửi lại cho S1.

- 2 copies của frame truyền tới S1  $\Rightarrow$  xảy ra loop.
- S1 gửi các copies của frame nhận được:
  - Từ port 2 ra port 1 và 3.
  - Từ port 3 ra port 1 và 2.
- S1 cập nhật forwarding table 2 lần  $\rightarrow$  không thể dự đoán được thứ tự mà các frame đến S1.



- Mỗi frame do A gửi tạo thành 2 copies  $\rightarrow$  các copies này loop vô tận trong mạng.
- Nếu A tiếp tục gửi thêm frames  $\rightarrow$  mạng sẽ bị flooded  $\Rightarrow$  đến một thời điểm thì nó sẽ collapsed.
- Các copies của frame tiếp tục đi qua S2 và S3  $\rightarrow$  S2 và S3 liên tục cập nhật bảng.



→ Ethernet không có TTL hay HopLimit  $\Rightarrow$  vì thế loop sẽ không bao giờ kết thúc cho đến khi forwarding table của các switch có entry cho B.



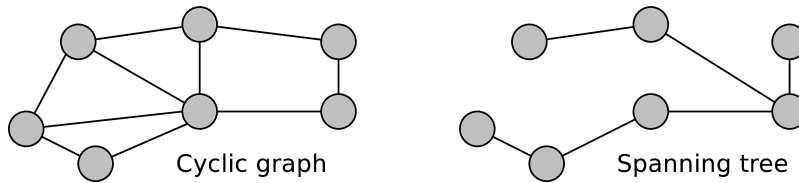
⇒ **Ví dụ này chứng minh:** nếu cắm dây mạng thành vòng tròn mà không có cấu hình gì thêm (chẳng hạn như STP - Spanning Tree Protocol, sẽ nói ở mục dưới) → switch sẽ tự hủy network do cơ chế flooding dùng để tìm destination address.

### 3.3.3.3 Handle Loops in the LAN

Bridge cần phải handle loops để tránh các vấn đề như trên xảy ra.

⇒ **Giải pháp:** tạo cấu trúc phân cấp logic (create logical hierarchy).

Mạng máy tính gồm nhiều physical network → được biểu diễn bằng graph mà có thể có loops.



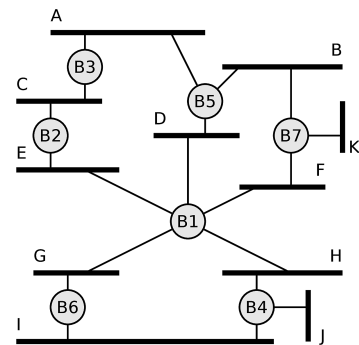
- Về vật lý → các dây cáp vẫn được cắm nối vòng tròn để dự phòng.
- Về logic → các bridge/switch sẽ được configure để vô hiệu hóa (block) một số cổng nhất định.

⇒ Tạo ra một subgraph bao phủ tất cả các nút nhưng không có vòng lặp (cycle-free) → gọi là Spanning Tree Protocol (STP).

### 3.3.3.4 Spanning Tree Protocol

**Thông tin tổng quát:**

- **Chuẩn:** IEEE 802.1D.
- **Chức năng:** biến một physical network có loops (mesh/-cyclic) thành một mạng logic hình cây (tree) không có loops.
- **Nguyên lý hoạt động:** các bridge trao đổi thông tin để vote root bridge → các cổng dẫn đến đường đi ngắn nhất sẽ được mở, còn lại sẽ bị khóa (blocked).



→ Thông qua STP → một nhóm các bridges có thể thỏa thuận với nhau để tạo thành spanning tree.

⇒ Bằng phương pháp vô hiệu hóa một số cổng trên bridge (removing single ports) → network sẽ chuyển thành cycle-free tree.

→ Thuật toán hoạt động linh hoạt (dynamic way) ⇒ nếu một bridge lỗi → mạng sẽ tự tạo một spanning tree mới.

### 3.3.3.5 Spanning Tree Protocol - Bridge ID

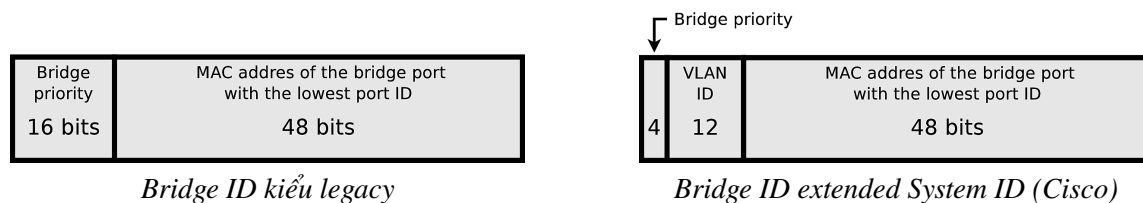
Để STP hoạt động → mỗi bridge cần có một mã định danh duy nhất (gọi là bridge ID).

- **Độ dài:** 8 bytes.
- **Chức năng:** mỗi bridge cần một mã định danh duy nhất (unique identifier) để vote làm root bridge.

#### Có 2 cách triển khai:

- Kiểu cổ điển (traditional/legacy 802.1D) → gồm 2 bytes priority + 6 bytes MAC address ⇒ chỉ dùng cho 1 mạng LAN duy nhất, không hỗ trợ tốt cho VLAN.
- Kiểu mở rộng (extended System ID - 802.1t) → khi chia nhiều mạng con ảo (VLAN), mỗi VLAN cần một spanning tree riêng ⇒ giúp cho mỗi VLAN có 1 ID riêng biệt dù chạy trên cùng một physical switch.

⇒ Người ta lấy 2 bytes priority đem chia thành 4 bit đầu là priority, 12 bit sau là VLAN ID, 6 bytes cuối vẫn là MAC address.



#### Cấu trúc: bridge ID gồm 2 phần ghép lại:

- Phần đầu (16 bits = 2 bytes) → bridge priority (độ ưu tiên của bridge) ⇒ giá trị mặc định thường là 32768; administrator có thể chỉnh số này với bất kì giá trị nào trong khoảng 0 đến 65535.
- Phần sau (48 bits = 6 bytes) → MAC address ⇒ đây là địa chỉ cứng của thiết bị, không đổi được (như đã nói ở phần trước).

#### Self-note:

→ Người ta chọn 32768 làm giá trị mặc định → là bit thứ 16 (bit cao nhất trong 16 bit, nếu tính từ 1) ⇒ đại diện cho vị trí trung bình để dễ dàng thay đổi lên xuống.

### 3.3.3.6 Spanning Tree Protocol - Cisco Bridge IDs

#### **Mục đích:**

→ Cisco hỗ trợ các bridge mà mỗi VLAN sẽ có một spanning tree riêng ⇒ để làm được điều này thì cấu trúc bridge ID cần thay đổi để chứa được thông tin VLAN.

#### **Cấu trúc mới của phần priority:** (nhắc lại ở trên)

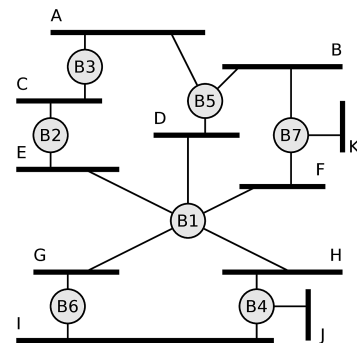
Phần 2 bytes (16 bits) đầu của bridge ID được chia thành 2 phần:

- Bridge priority (4 bits đầu - theo hình phải) → còn 4 bit để biểu diễn độ ưu tiên ⇒ chỉ có thể tạo ra 16 giá trị khác nhau ( $2^4 = 16$ ).
  - Quy tắc: giá trị priority phải là 0 hoặc bội số của 4096; ví dụ như: 0000 = 0, 0001 = 4096, 0010 = 8192, ..., 1111 = 61440.
- Extended System ID (12 bits sau) → dùng để mã hóa VLAN ID ⇒ giá trị này khớp với VLAN tag trong Ethernet frame → với 12 bit, nó có thể đánh địa chỉ cho 4096 VLAN khác nhau.

### 3.3.3.7 Spanning Tree Protocol - Functioning

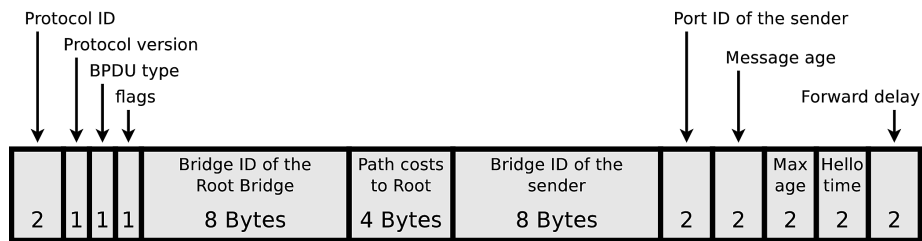
Các bridge trong mạng trao đổi thông tin với nhau về ID của bridge và path cost thông qua các frames đặc biệt → gọi là BPDU (Bridge Protocol Data Units).

⇒ Nó được gửi trong payload field của Ethernet frame thông qua broadcast tới các bridge lân cận.



#### **Chọn root bridge:**

- Khi mạng khởi động → tất cả các bridge đều tự coi mình là root bridge và gửi thông tin BPDU cho nhau để so sánh ID.



- Quy tắc: so sánh bridge ID của tất cả các thiết bị  $\Rightarrow$  bridge có ID nhỏ nhất sẽ được chọn làm root bridge (chỉ có **1 root bridge** trong mạng).

$\Rightarrow$  Nếu có nhiều bridge có cùng priority  $\rightarrow$  bridge có MAC address nhỏ nhất sẽ được chọn.

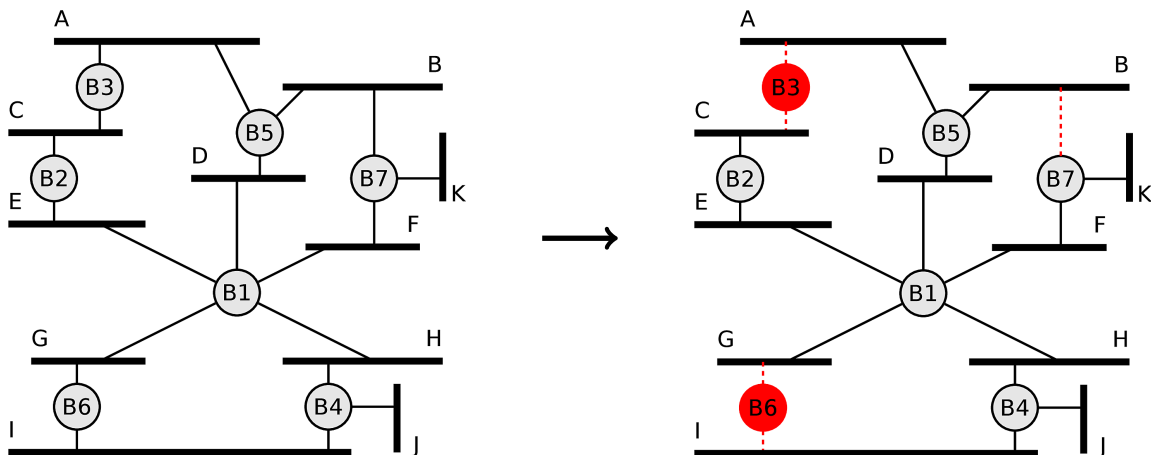
### Xác định đường đi ngắn nhất (root ports):

$\rightarrow$  Đối với mỗi physical network  $\rightarrow$  cần chọn ra 1 bridge duy nhất trong số các bridge được kết nối để chuyển tiếp các frame theo hướng về root bridge.

$\Rightarrow$  Đây được gọi là designated bridge (bridge được chỉ định) cho mạng đó.

- Mọi bridge còn lại (non-root bridge) phải tìm đường nhanh nhất để về root bridge.
- Các bridge tính toán path cost trên các port của mình  $\Rightarrow$  trên mỗi bridge, port nào có đường về root ngắn nhất (lowest path cost) sẽ được chọn làm root port  $\rightarrow$  port này sẽ luôn mở để chuyển dữ liệu.

$\Rightarrow$  Path cost tới root bridge là tổng các path cost của các physical network khác nhau trên đường từ bridge đó đến root bridge.



**Khóa các đường thừa (blocking):**

- **Vấn đề:** sau khi chọn xong đường về root → vẫn còn dư các đường nối giữa switch với nhau do các dây cáp dự phòng.
- STP sẽ tính toán để giữ lại một đường duy nhất trên mỗi đoạn (designated port) → sau đó block tất cả các port còn lại.
- ⇒ Các port bị block sẽ không chuyển tiếp dữ liệu nhưng vẫn ở đó để phòng khi đường chính bị sự cố thì mở lại.

## Chapter 4

# Data Link Layer - Medium Access Control and Error Control

### 4.0.1 Coordinated Medium Access

#### Tại sao cần điều phối (coordination)?

Khi có nhiều card mạng (network interface card - NIC) cùng kết nối vào một đường truyền chung → việc truy cập cần phải được điều phối (need to be coordinated).

⇒ Nếu không có sự điều phối → collision sẽ xảy ra khi các thiết bị truyền dữ liệu cùng một lúc.

→ Có 2 nhóm phương pháp điều phối.

#### Contention-based Protocols (giao thức cạnh tranh):

- **Cơ chế:** các thiết bị tham gia phải cạnh tranh (compete) để truy cập vào đường truyền.
- **Đặc điểm:**
  - Xử lý xung đột (collision handled) → do có nhiều thiết bị có thể gửi cùng lúc → xung đột là điều chắc chắn xảy ra ⇒ protocol phải có cách xử lý chúng.
  - Non-deterministic (không xác định trước được) → không thể biết trước ai sẽ gửi.
- **Hiệu suất:** hoạt động tốt khi lưu lượng mạng thấp đến trung bình, và lưu lượng có tính chất đột biến (bursty).
- **Ví dụ:** ALOHA, CSMA/CA (tránh xung đột bằng cách nghe trước khi gửi - Carrier Sense Multiple Access with Collision Avoidance), MACA (Multiple Access with Collision Avoidance).

### Contention-free Protocols (giao thức không cạnh tranh):

- **Cơ chế:** quyền truy cập đường truyền được phân bổ trước (allocated in advance).
- **Đặc điểm:**
  - Tránh xung đột hoàn toàn (collision avoided) → collision không bao giờ xảy ra vì đã được plan trước.
  - Deterministic (xác định trước được) → biết rõ ai sẽ gửi và gửi khi nào.
- **Hiệu suất:** hoạt động tốt khi lưu lượng mạng cao (high utilization) → đảm bảo tính công bằng (guarantee fair use) về dung lượng mạng cho các thiết bị.
- **Ví dụ:** Token Passing, TDMA (phân chia theo thời gian - Time Division Multiple Access), FDMA (phân chia theo tần số - Frequency Division Multiple Access), CDMA (phân chia theo mã - Code Division Multiple Access).

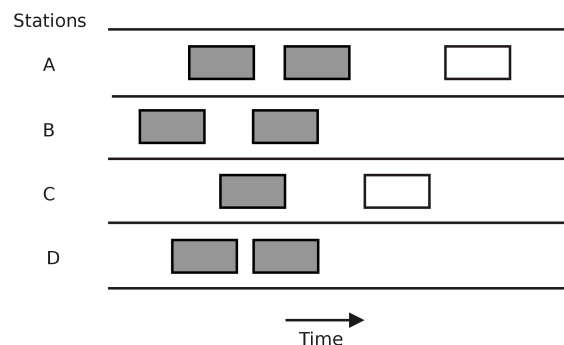
## 4.1 Contention-based

### 4.1.1 ALOHA

#### 4.1.1.1 ALOHA

ALOHA → Additive Links On-line Hawaii Area → giao thức được phát triển tại Đại học Hawaii vào những năm 1970 để kết nối các trạm từ xa qua sóng vô tuyến.

**Pure ALOHA** → đây là phiên bản sơ khai và đơn giản nhất.



- **Nguyên lý hoạt động:** không có kiểm soát trung tâm hay sự phối hợp nào giữa các trạm → các trạm có thể bắt đầu gửi dữ liệu bất cứ khi nào chúng muốn.

- **Vấn đề:** do ai cũng có thể nói bất cứ lúc nào → collision dễ xảy ra ⇒ nếu 2 gói tin chồng lên nhau, dù chỉ một chút → cả 2 đều bị hỏng.

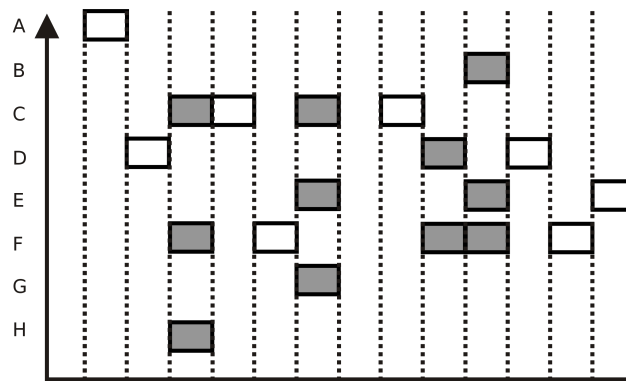
- **Đặc điểm:**

- Collision có thể xảy ra (như này nói trên).
- Không có tính công bằng (no fairness).

⇒ Phương pháp rất đơn giản, không yêu cầu thêm gì.

#### 4.1.1.2 Slotted ALOHA

Slotted ALOHA → ALOHA phân khe ⇒ đây là phiên bản cải tiến của pure ALOHA để giảm thiểu collision.



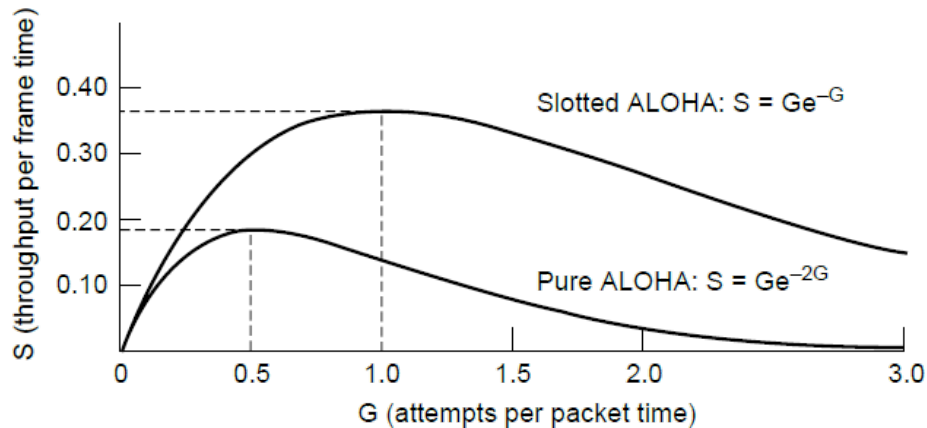
Slotted ALOHA protocol (shaded slots indicate collision)

- **Nguyên lý hoạt động:** thời gian được chia thành các khe (slots) có độ dài cố định ngang với thời gian truyền một gói tin (packet).
  - Các trạm chỉ được phép bắt đầu gửi tại thời điểm bắt đầu của một slot.
 ⇒ Yêu cầu phải đồng bộ hóa thời gian chung (common time-base for synchronization).
- **Ưu điểm (so với pure ALOHA):** collision chỉ xảy ra hoàn toàn (complete collision - trùng khít cả slot) chứ không bị chồng lên một phần → giảm xác suất lỗi; thông lượng (throughput) được cải thiện.
- **Nhược điểm:** chưa đảm bảo được fairness; delay tăng do phải chờ đến đầu slot tiếp theo thì mới được gửi.



⇒ Được dùng trong GSM network (Global System for Mobile Communications - mạng di động thế hệ đầu tiên).

#### 4.1.1.3 Performance of ALOHA



Để đánh giá hiệu suất của ALOHA, người ta sử dụng 2 thông số chính:

- $S$  (throughput) → lượng dữ liệu truyền được thành công trên đơn vị thời gian.
- $G$  (load) → tổng lượng dữ liệu mà các trạm cố gắng gửi gói tin (bao gồm cả các gói tin bị collision) trên đơn vị thời gian.

**Pure ALOHA:** → protocol có hiệu suất thấp nhất.

- **Công thức:**  $S = G \cdot e^{-2G}$ .
- **Hiệu suất tối đa:** đạt chỉ  $\approx 18.4\%$  ( $1/(2e)$ ) dung lượng kênh truyền (channel's capacity) ⇒ đạt được khi  $G = 0.5$  (nghĩa là trung bình cứ 2 khoảng thời gian truyền thì có 1 gói tin được gửi thành công).

⇒ **Lý do:** vì các trạm gửi tự do bất cứ lúc nào → kể cả một cái collision nhỏ cũng làm hỏng gói tin.

**Slotted ALOHA:** → cải tiến hiệu suất bằng cách ép buộc các trạm chỉ gửi ở đầu mỗi slot (do phải sync thời gian).

- **Công thức:**  $S = G \cdot e^{-G}$ .

- **Hiệu suất tối đa:** đạt chỉ  $\approx 37\%$  ( $1/e$ ) dung lượng kênh truyền  $\Rightarrow$  đạt được khi  $G = 1$  (nghĩa là trung bình cứ 1 khoảng thời gian truyền thì có 1 gói tin được gửi thành công).

$\Rightarrow$  Slotted ALOHA cho throughput cao gấp đôi so với pure ALOHA  $\Rightarrow$  nó phải đánh đổi bằng delay cao  $\rightarrow$  phải chờ đến đầu slot tiếp theo mới được gửi.

Self-note: cái hiệu suất nó mang tính "trong tổng số  $G$  attempt gửi đi thì có bao nhiêu phần trăm là may mắn không bị collision".

$\rightarrow$  Dùng Poisson để tính xác suất lần thử thành công:  $S = G \cdot P$  (vì mô hình này giả định số lượng trạm tham gia rất lớn).

$\rightarrow$  Với pure ALOHA: xác suất để không có trạm nào gửi tin trong khoảng thời gian  $2t$  (tương ứng  $2G$ )  $\rightarrow P = e^{-2G}$ .

$\rightarrow$  Với slotted ALOHA: xác suất để không có trạm nào gửi tin trong khoảng thời gian  $t$  (tương ứng  $G$ )  $\rightarrow P = e^{-G}$ .

$\Rightarrow$  Từ đây có thể thấy được slotted ALOHA giảm phân nửa khoảng thời gian collision.

## 4.1.2 CSMA

### 4.1.2.1 CSMA

CSMA  $\rightarrow$  **C**arrier **S**ense **M**ultiple **A**ccess  $\rightarrow$  hoạt động dựa trên nguyên tắc "nghe trước khi nói" (listen before talk).

**Nguyên lý hoạt động:** trạm phát phải lắng nghe (sense) đường truyền.

- Nếu đường truyền bận (busy)  $\rightarrow$  trạm phải đợi.
- Nếu đường truyền rảnh (idle)  $\rightarrow$  trạm mới xem xét gửi dữ liệu.

Có 2 phương pháp chính để thực hiện CSMA.

**p-persistent CSMA (kiên trì xác suất  $p$ ):**

- Khi đường truyền bận  $\rightarrow$  trạm tiếp tục đợi cho đến khi rảnh.
- Khi đường truyền rảnh  $\rightarrow$  trạm gửi gói tin với xác suất  $p$ , hoặc phải chờ đến slot tiếp theo với xác suất  $1 - p$ .

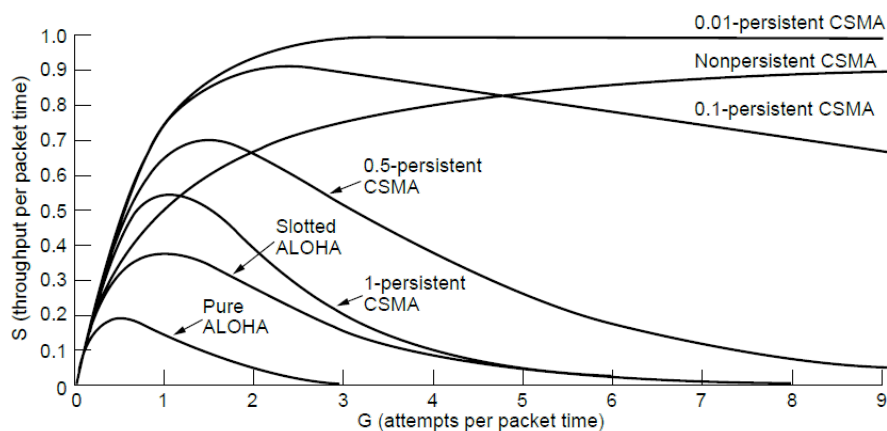
Self-note: gửi  $p$  xác suất ở đây như trò tung đồng xu → thấy đường dây rảnh thì bắt đầu random ⇒ nếu rơi vào mặt ngửa ( $p$ ) thì gửi, còn rơi vào mặt sấp ( $1 - p$ ) thì đợi đến slot tiếp theo rồi random lại → nếu đang tung mà thấy đường dây có đĩa khác gửi thì chờ đến slot tiếp theo rồi random tiếp.

⇒ Sẽ có lúc bị collision do dính xác suất cả 2 cùng chọn gửi trong cùng 1 slot, hoặc có thể bị delay do cả 2 đĩa đều chờ mà nguyên đường dây không đĩa nào gửi.

#### Non-persistent CSMA (không kiên trì):

- Khi đường truyền bận → trạm không ngồi nghe liên tục mà sẽ đợi một khoảng thời gian ngẫu nhiên → sau đó quay lại nghe tiếp.
- Khi đường truyền rảnh → gửi gói tin ngay lập tức.

#### 4.1.2.2 Performance of CSMA



Theo đồ thị biểu diễn hiệu suất của các phương pháp → cơ chế carrier sense giúp giảm xung đột đáng kể ⇒ tăng dữ liệu truyền thành công cao hơn so với việc gửi tự do như ALOHA.

#### Với CSMA:

- **Hiệu suất cao** →  $p$  càng nhỏ càng giúp tránh collision tốt hơn vì các trạm nhường nhau.
- **Delay cao** → do  $p$  nhỏ → xác suất gửi đi thấp ⇒ gói tin phải chờ rất lâu mới được gửi đi dù đường truyền trống.

Self-note: non-persistent khác với 1-persistent:

- **Non-persistent:** đường truyền đợi một khoảng thời gian random rồi mới nghe lại → nếu đường truyền rảnh thì gửi ngay.
- **1-persistent:** đường truyền cứ nghe liên tục đến khi rảnh thì gửi ngay.

### 4.1.3 CSMA/CA and MACA

#### 4.1.3.1 CSMA/CA

→ Tránh xung đột (collision avoidance) thay vì phát hiện xung đột (collision detection).

**Tại sao không dùng CSMA/CD (collision detection)?**

- Trong mạng không dây → việc vừa gửi vừa nghe để bắt xung đột là rất khó ⇒ vì trạm phát ra một tín hiệu rất mạnh → át hoàn toàn các tín hiệu yếu hơn từ xa tới → máy không thể biết gói tin có bị collision không.
- Có vấn đề trạm ẩn (hidden terminal) → các trạm không nghe thấy nhau dẫn đến việc gửi cùng lúc với nhau → gây ra xung đột.

⇒ Chính vì không thể phát hiện xung đột → chọn cách tránh xung đột ngay từ đầu là hợp lý nhất.

**Ở sender:**

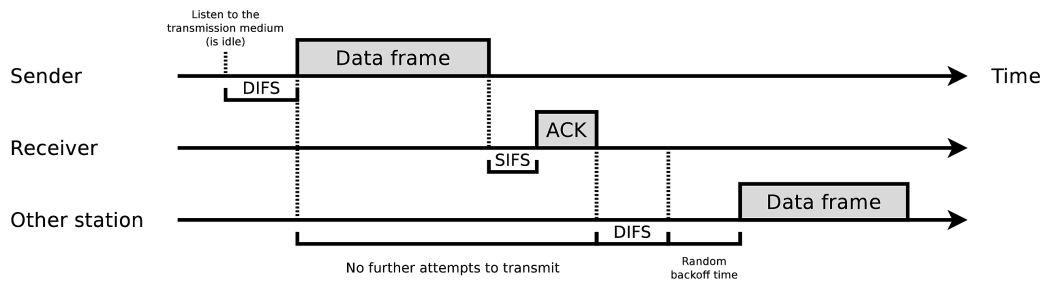
- Nếu thấy đường truyền rảnh → nó không gửi ngay mà chờ thêm một khoảng thời gian (time slots/DIFS - Distributed Interframe Space) để chắc chắn rồi mới gửi.
- Dựa vào ACK (acknowledgment) để biết → do sender không tự biết có lỗi hay không → phụ thuộc hoàn toàn vào response của receiver.
  - ⇒ Nếu gửi xong mà không nhận được ACK → sender tự hiểu là bị mất hoặc collision → nó sẽ gửi lại (retransmit).

**Ở receiver:**

- Khi nhận được gói tin → kiểm tra xem có lỗi không (dùng checksum/CRC).
- Nếu gói tin error-free → receiver sẽ đợi một khoảng thời gian ngắn (short time delay/SIFS - Short Interframe Space) → gửi ACK trở lại cho sender.

⇒ Mạng không dây không thấy được trạm ẩn khi phát sóng, và không thấy được collision → nó không thể tự bắt lỗi được → phải dựa vào ACK từ receiver để biết có lỗi hay không.

#### 4.1.3.2 Functioning of CSMA/CA



##### Ở sender:

Đầu tiên, sender lắng nghe kênh truyền ( $\rightarrow$  carrier sense)  $\rightarrow$  kênh truyền cần phải không có tín hiệu trong một khoảng thời gian ngắn.

$\Rightarrow$  Khoảng thời gian này được gọi là DIFS (Distributed Interframe Space)  $\approx 50\mu s$ .

$\rightarrow$  Nếu kênh truyền không có tín hiệu trong suốt 1 DIFS  $\rightarrow$  trạm có thể bắt đầu gửi frame.

##### Ở receiver:

Nếu trạm nhận được success frame (đã pass CRC check)  $\rightarrow$  nó sẽ đợi một khoảng thời gian ngắn.

$\Rightarrow$  Khoảng thời gian này được gọi là SIFS (Short Interframe Space)  $\approx 10\mu s$ .

Sau đó, receiver gửi một acknowledgment frame (ACK).

$\Rightarrow$  DIFS và SIFS đảm bảo được khoảng cách tối thiểu cho CSMA/CA khi một chuỗi frame sẽ được gửi.

##### Random backoff (tranh chấp ngẫu nhiên):

$\rightarrow$  Nếu sau khi chờ DIFS mà có nhiều trạm cùng muốn gửi, hoặc đường truyền đang bận  $\rightarrow$  cơ chế random backoff sẽ được kích hoạt để tránh va chạm.

Backoff calculation  $\rightarrow$  sender sẽ chọn một giá trị ngẫu nhiên (nằm trong khoảng min và max của contention window - CW)  $\rightarrow$  đem nhân với thời gian của một slot  $\Rightarrow$  sender bắt đầu đếm ngược thời gian này.

$\Rightarrow$  Nếu backoff time kết thúc  $\rightarrow$  sender được phép gửi frame.

Nếu trong thời gian backoff mà có một trạm khác nhảy vào chiếm đường truyền  $\rightarrow$  sender sẽ dừng đếm (the counter variable is stopped).

$\rightarrow$  Đợi cho đến khi đường truyền rảnh trở lại trong ít nhất 1 DIFS  $\rightarrow$  tiếp tục đếm ngược giá trị backoff còn lại.

Self-note: (Tóm tắt)

- Muốn gửi data → phải chờ DIFS ⇒ tốn thời gian hơn.
- Muốn gửi ACK → chỉ cần chờ SIFS ⇒ nhanh hơn vì ACK được ưu tiên chạy trước.
- Nếu đường bận → chọn random backoff time để xếp hàng chờ gửi ⇒ nếu có đứa nào chen vào thì dừng đếm, chờ DIFS, rồi đếm tiếp.

#### 4.1.3.3 MACA (Multiple Access with Collision Avoidance)

CSMA/CA giảm thiểu collision → nhưng không thể loại bỏ hoàn toàn.

⇒ MACA/MACA-W là phiên bản nâng cấp của CSMA/CA → được thiết kế đặc biệt để giải quyết bài toán trạm ẩn (hidden terminal problem) mà CSMA/CA không xử lý triệt để được.

MACAW → **M**ultiple **A**ccess with **C**ollision **A**voidance for **W**ireless ⇒ là phiên bản mở rộng của MACA → bổ sung thêm một số tính năng để cải thiện hiệu suất và độ tin cậy.

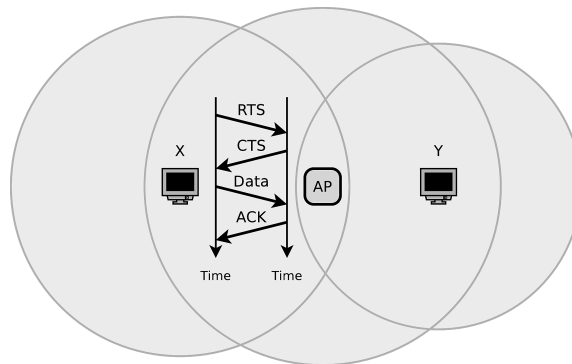
##### Nguyên lý hoạt động:

Thay vì chỉ nghe và gửi như CSMA/CA → MACA yêu cầu sender và receiver phải trao các frame điều khiển (control frames) trước khi gửi dữ liệu.

⇒ Nhờ cách này mà tất cả các trạm trong mạng đều biết rằng sắp có một phiên truyền (transmission) bắt đầu → tránh được việc truy cập đồng thời.

Các frame điều khiển bao gồm **RTS** (Request to Send - yêu cầu gửi) và **CTS** (Clear to Send - cho phép gửi).

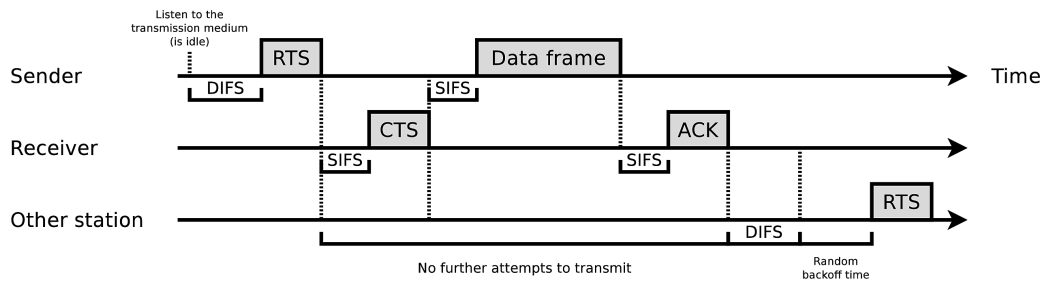
⇒ Những khung này chứa thông tin về khoảng thời gian kênh truyền sẽ bị sử dụng (occupation time of the channel).



⇒ Theo hình trên: trạm Y không nhận được RTS frame do trạm X gửi → nhưng vẫn nhận được tín hiệu cho phép gửi (CTS) phát ra từ access point (AP).

→ Collision chỉ có thể xảy ra trong quá trình truyền các frame RTS và CTS → do ảnh hưởng của hidden terminal.

#### 4.1.3.4 Functioning of MACA



##### Ở sender:

Đầu tiên, sender lắng nghe kênh truyền → sau khoảng thời gian DIFS, sender gửi một RTS frame đến receiver.

##### Ở receiver:

→ Xác nhận yêu cầu sử dụng kênh (reservation request) → chờ một khoảng thời gian SIFS → gửi lại (transmit) một CTS frame chứa khoảng thời gian mà sender muốn dành riêng để sử dụng transmission medium.

⇒ Sau khi receiver nhận thành công dữ liệu data frame → nó sẽ chờ một khoảng thời gian SIFS → gửi ACK trở lại cho sender.

Khi transmission medium bận (occupied) → các trạm khác sẽ không cố để truyền data frame mới cho đến khi NAV (Network Allocation Vector - bộ đếm thời gian dành riêng cho kênh) kết thúc.

##### Cơ chế NAV:

- NAV → là một biến đếm mà mỗi trạm tự quản lý.
- **Chức năng:** chứa thông tin về thời gian dự kiến mà đường truyền sẽ bị sử dụng.
- **Nguyên lý hoạt động:**
  - Khi các trạm khác nghe thấy RTS hoặc CTS (có chứa thông tin thời gian) → chúng sẽ cập nhật vào NAV của chính mình.
  - Biến NAV sẽ tự động giảm dần cho đến khi về 0.

- Trong khoảng thời gian NAV chưa về 0 → các trạm sẽ không tiếp tục gửi bất cứ thứ gì (no further attempts to transmit).

⇒ Giảm thiểu được số lượng xung đột.

- **Pros:**

- Ít bị xung đột hơn → giải quyết triệt để vấn đề trạm ẩn (→ xung đột chỉ có thể xảy ra ở bước RTS/CTS rất ngắn).
- Tiết kiệm năng lượng hơn → các trạm khác không cần tốn năng lượng để gửi tin trong thời gian NAV.

- **Cons:**

- Gây delay cao → việc phải xin phép RTS/CTS gây tốn thời gian.
- Tồn bằng thông (overhead) → các gói tin RTS và CTS tuy nhỏ nhưng vẫn sử dụng đường truyền đáng kể.

Self-note: (Tóm gọn MACA functioning)

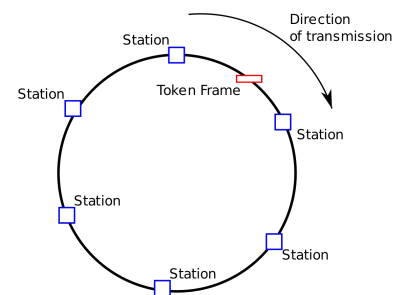
Xin (RTS) → cho phép (CTS) → các bên khác im lặng (chờ NAV) → gửi data → gửi ACK.

## 4.2 Contention-free

### 4.2.1 Token Passing

**Nguyên lý hoạt động:**

- Token frame → có một gói tin đặc biệt gọi là token → được truyền đi vòng quanh mạng (thường là cấu trúc dạng vòng - ring).
- Token này có tác dụng xác định thứ tự ai được phép gửi data (defining the sending order).
- **Quy luật:** chỉ có trạm nào đang giữ token thì mới có quyền gửi data (→ eliminates collisions).
  - Nếu trạm đó có dữ liệu cần gửi → nó giữ token, gửi data, rồi mới chuyển token cho trạm kế tiếp.
  - Nếu trạm đó không có dữ liệu cần gửi → nó chuyển token ngay cho trạm kế tiếp.



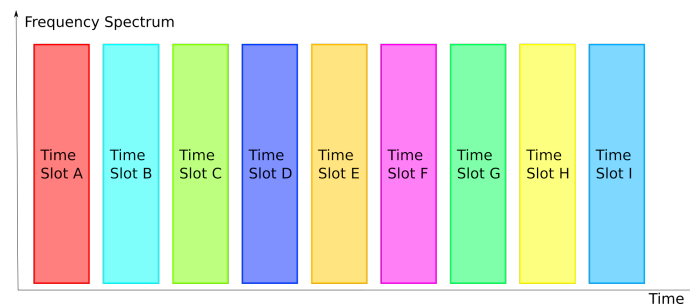


**Cấu trúc mạng (topology):** thường được hình dung trong cấu trúc mạng dạng vòng (ring) → nơi token đi từ trạm này sang trạm liền kề theo một chiều cố định.

⇒ Tuy nhiên, ý tưởng này cũng có thể áp dụng cho các cấu trúc khác; chẳng hạn như token bus.

## 4.2.2 TDMA

TDMA → Time Division Multiple Access → phân chia theo thời gian.



→ Tại một thời điểm, chỉ có 1 người dùng toàn bộ tần số → nhưng họ thay phiên nhau theo thời gian (hình minh họa trên).

### Nguyên lý hoạt động:

- Chia slot thời gian (time slots) → thời gian được chia nhỏ thành các slots.
  - Phân quyền → mỗi time slot được gán cố định cho một máy chủ (host) cụ thể.
- Trong suốt thời gian của slot đó → máy chủ được quyền sử dụng toàn bộ dung lượng đường truyền (full channel capacity) ⇒ không một máy nào được cắt ngang.

**Cách lên lịch (scheduling):** có thể thực hiện bằng 2 cách:

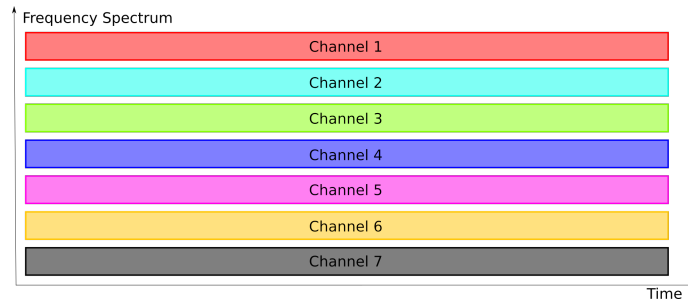
- Static (tĩnh) → lịch cố định; ví dụ như A luôn gửi ở slot 1, B luôn gửi ở slot 2.
- Dynamic (động) → lịch thay đổi tùy theo nhu cầu thực tế.

⇒ Note: một số time slot có thể được dành riêng cho broadcast traffic.

Self-note: TDMA giống kiểu như chia nhau theo giờ, đến giờ của ai thì người đó dùng, dùng xong thì nhường cho người tiếp theo → tuy nhiên thì trong lúc dùng, mình được phép dùng hết công suất.

### 4.2.3 FDMA

#### 4.2.3.1 Frequency Division Multiple Access (FDMA)



→ Chia thành các kênh con (sub-channels) riêng biệt → có thể hoạt động song song cùng một lúc với nhau (*hình minh họa trên*).

#### Nguyên lý hoạt động:

- Chia nhỏ phổ tần (subdivide spectrum) → thay vì dùng toàn bộ băng thông → FDMA chia nhỏ dải tần số thành các kênh con (sub-channels) riêng biệt.
  - Phân quyền → mỗi máy chủ (host) hoặc mỗi liên kết (link) được gán cho một kênh tần số riêng.
- ⇒ Mọi máy có thể truyền dữ liệu song song cùng một lúc (liên tục theo thời gian) → nhưng không lẫn nhau do mỗi bên truyền ở mỗi tần số khác nhau.

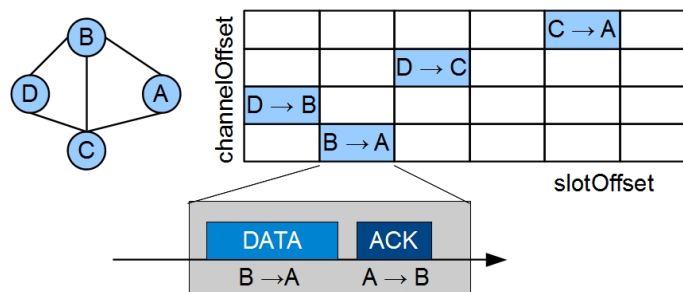
#### Quản lý kênh:

- Control traffic (kênh điều khiển) → thường được gửi trên các kênh cố định để đảm bảo kết nối.
- Cách gán kênh:
  - Static (tĩnh) → gán kênh cố định; ví dụ như đài A luôn phát ở 100MHz, đài B luôn phát ở 105MHz.
  - Dynamic (động) → kênh thay đổi theo nhu cầu sử dụng ⇒ việc phân bổ này dẫn đến một vấn đề là "graph coloring problem" → làm sao để gán màu (tần số) cho các trạm cạnh nhau mà không bị trùng nhau.

Self-note: (tổng kết lại)

- TDMA → chia thời gian thành các time slots → mỗi time slot chỉ được 1 host sử dụng, và máy đó được dùng toàn bộ băng thông.
- FDMA → chia băng thông thành các sub-channels → tất cả các host đều hoạt động song song nhưng chỉ trong một dải băng tần nhất định, không lẫn nhau.

#### 4.2.3.2 Example: IEEE 802.15.4e



→ Là một tiêu chuẩn (specification) cho mạng cá nhân không dây (PAN); chẳng hạn như Zigbee.

Là một bản chỉnh sửa (amendment) của tầng MAC ⇒ hỗ trợ cơ chế time-slotted channel hopping (TiSCH) dựa trên TDMA và FDMA (theo hình):

- **Time-slotted (TDMA)** → thực hoành là thời gian (*slotOffset*) → thời gian được chia thành các slot.  
→ Trong 1 slot, thiết bị sẽ làm trọn vẹn quy trình: gửi DATA → chờ → nhận ACK.
- **Channel-Hopping (FDMA)** → thực tung là tần số (*channelOffset*) → thay vì gửi mãi ở mỗi kênh (→ dễ bị nhiễu) → các cặp thiết bị sẽ chuyển sang kênh tần số khác nhau ở mỗi slot thời gian.

#### Cách hoạt động:

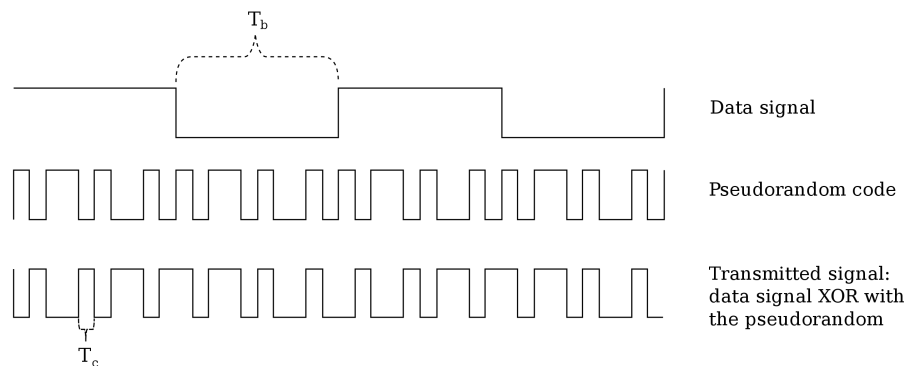
- Ở slot đầu tiên: D → B truyền tin ở một kênh tần số thấp.
- Ở slot tiếp theo: B → A nhưng chuyển qua một kênh có tần số khác.
- Tương tự vậy như D → C và C → A.

### Vì sao phải phức tạp vậy?

- Chống nhiễu cực tốt → giả sử kênh 1 bị nhiễu → slot sau nó chuyển qua kênh 5 ⇒ đảm bảo liên lạc không bị gián đoạn.  
⇒ Đây là ưu điểm của channel hopping.
- Tiết kiệm năng lượng → nhờ có cơ chế chia slot (TDMA) → thiết bị biết chính xác khi nào cần dậy để chuyển kênh và truyền, thời gian còn lại thì ngủ.

### 4.2.4 CDMA

CDMA → **C**ode **D**ivision **M**ultiple **A**ccess → phân chia theo mã.



Nếu FDMA chia tần số, TDMA chia thời gian → thì CDMA cho phép tất cả mọi người cùng dùng một thời điểm trên cùng một tần số.

#### Nguyên lý hoạt động:

- **Truy cập đồng thời** → các trạm (host) có thể truy cập môi trường truyền dẫn cùng một lúc và trên cùng một tần số.
- **Mã hóa riêng biệt** → mỗi trạm sử dụng một coding schemes (lược đồ mã hóa) khác nhau  
⇒ tránh bị lẫn lộn tín hiệu của nhau.
- **Orthogonal (tính trực giao)** → nghĩa là các mã này hoàn toàn độc lập nhau → khi cộng gộp lại, chúng không làm hỏng nhau, và người nhận có thể tách riêng từng mã được (giống kiểu trong phòng ồn ào mà có giọng bạn mình nói thì mình vẫn lọc được).

### Cách tạo tín hiệu:

Tín hiệu CDMA được tạo ra bằng phép XOR (như hình minh họa mô tả):

1. Data signal → tín hiệu dữ liệu gốc (tốc độ chậm).
2. Pseudorandom code (mã giả ngẫu nhiên) → một chuỗi mã ngẫu nhiên giả (tốc độ nhanh) được gán riêng cho người dùng.
3. Transmitted signal (tín hiệu truyền đi) → là kết quả của phép XOR giữa data signal và pseudorandom code.

⇒ Tín hiệu truyền đi trông giống như một chuỗi ngẫu nhiên (noise) → nhưng thật chất nó chứa dữ liệu đã được mã hóa.

→ CDMA thường kết hợp với FEC (Forward Error Correction - sửa lỗi tiền) để khôi phục các frame bị lỗi mà không cần gửi lại.

## 4.3 Error Control

### 4.3.1 Failure Causes

Các nguyên nhân gây nên:

- **Signal deformation** (biến dạng tín hiệu) → do sự suy hao (attenuation) của môi trường truyền dẫn ⇒ tín hiệu càng đi xa thì càng yếu đi → nếu quá yếu, máy thu sẽ không phân biệt được đâu là bit 1, đâu là bit 0.
- **Noise** → do nhiễu nhiệt hoặc nhiễu điện tử (thermal or electronic noise) ⇒ đây là nhiễu sinh ra từ chính các linh kiện điện tử nóng lên, hoặc chuyển động của các electron trong dây dẫn → tạo ra các tín hiệu ngẫu nhiên chen vào dữ liệu thật.
- **Crosstalk (nhiễu xuyên kênh)** → do sự can nhiễu từ các kênh lân cận (interference by neighboring channels) ⇒ thường do hiện tượng ghép tụ (capacity coupling), đặc biệt tăng mạnh khi tần số tăng cao.
- **Short-time disturbances (nhiễu ngắn hạn)** → do các tác động bất ngờ từ bên ngoài; chẳng hạn như bức xạ vũ trụ (cosmic radiation), lớp cách điện bị hỏng hoặc không đủ tốt (defective or insufficient insulation).

**Lưu ý:** (burst error - lỗi chùm) ⇒ lỗi thường không đi lẻ tẻ từng bit (single bit errors) mà thường xảy ra thêm chùm (burst errors).

→ Nếu nó đã bị nhiễu → sẽ hỏng cả một dãy liên tiếp nhiều bit → ảnh hưởng đến việc chọn thuật toán sửa lỗi sau này.

⇒ Data link layer đảm bảo phát hiện và xử lý lỗi truyền trong quá trình gửi/nhận frame.

### 4.3.2 Error Detection

#### 4.3.2.1 Checksum

→ Là một giá trị được tính toán dựa trên nội dung của khối dữ liệu (block of data) bằng một thuật toán định trước (pre-defined algorithm).

⇒ **Mục đích:** xác minh tính toàn vẹn của dữ liệu (data integrity).

**Quy trình:**

- **Sender** → tính toán checksum và gắn bó vào cuối mỗi frame.
- **Receiver** → khi nhận được frame → tính toán lại checksum dựa trên dữ liệu nhận được ⇒ nếu kết quả không khớp với checksum → frame bị lỗi và sẽ bị loại bỏ (discarded).

**Possible checksum:**

- Parity-check codes → kiểm tra tính chẵn lẻ (→ đơn giản).
- Cyclic Redundancy Check (CRC) → kiểm tra dư thừa theo chu kỳ (→ phức tạp và hiệu quả hơn).

#### 4.3.2.2 Hamming Distance

**Cấu trúc mã (codeword):** một message  $n$  bao gồm:

- $m$  bit dữ liệu (payload).
- $r$  bit checksum.

**Công thức:**  $n = m + r$ .

→ Trong tổng số  $2^n$  chuỗi bit có thể tạo ra → chỉ có một số ít là valid codewords (mã hợp lệ)  $2^m$ .

**Hamming distance** → là số lượng bit khác nhau nhỏ nhất giữa 2 valid codewords bất kỳ.

→ Ví dụ: nếu codeword A là 000 và codeword B là 011 → Hamming distance sẽ là 2 (do khác nhau 2 bit).

**Quy tắc để phát hiện lỗi:**

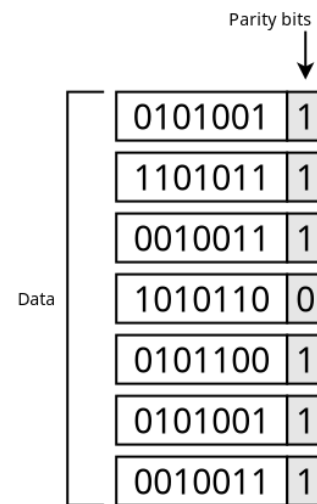
- Để phát hiện  $k$  lỗi → yêu cầu khoảng cách Hamming  $d \geq k + 1$ .  
⇒ Nếu sai  $k$  bit → nó biến thành một chuỗi vô nghĩa (không trùng với bất kỳ valid codeword nào) → xác định được đây là lỗi.

- Để sửa  $k$  lỗi  $\rightarrow$  yêu cầu khoảng cách Hamming  $d \geq 2k + 1$ .  
 $\Rightarrow$  Nếu sai  $k$  bit  $\rightarrow$  chuỗi bị lỗi vẫn gần với codeword gốc hơn là các codeword khác  $\rightarrow$  mình có thể đoán ngược lại để sửa.

#### 4.3.2.3 One-dimensional Parity-check Code

$\rightarrow$  Nó thêm 1 bit cuối vào dữ liệu  $\Rightarrow$  đảm bảo tổng số bit 1 luôn tuân theo quy tắc đã cho.

- **Cơ chế:** với mỗi khối dữ liệu (chẳng hạn như 1 byte hay 1 ký tự ASCII 7-bit)  $\rightarrow$  người ta tính toán và thêm vào 1 bit parity.
  - Protocol defines even parity (chẵn)  $\rightarrow$  bit parity được thêm vào sao cho tổng số bit 1 trong cả chuỗi (payload + parity) là một số chẵn.
  - Protocol defines odd parity (lẻ)  $\rightarrow$  bit parity được thêm vào sao cho tổng số bit 1 trong cả chuỗi là một số lẻ.
- **Ví dụ cụ thể:** (theo hình bên phải)
  - Với 0101001  $\rightarrow$  có 3 bit 1 (lẻ)  $\rightarrow$  thêm bit parity là 1 để tổng thành 4 (chẵn)  $\Rightarrow$  mã cuối cùng là 0101001 1.
  - Với 1010110  $\rightarrow$  có 4 bit 1 (chẵn)  $\rightarrow$  thêm bit parity là 0 để tổng vẫn là 4 (chẵn)  $\Rightarrow$  mã cuối cùng là 1010110 0.



#### What is the Hamming Distance here?

- Cấu trúc: với  $m = 7$  bit dữ liệu và  $r = 1$  bit parity  $\rightarrow$  có codewords dài  $n = 8$  bit.
- Khoảng cách Hamming  $d = 2 \Rightarrow$  bất kỳ 2 valid codewords nào cũng khác nhau ít nhất 2 bit (nếu chỉ thay đổi 1 bit dữ liệu  $\rightarrow$  bit parity cũng sẽ buộc phải thay đổi theo  $\Rightarrow$  tổng cộng khác 2 bit).

**Khả năng phát hiện lỗi:** do  $d = 2$ , theo công thức  $d \geq k + 1 \rightarrow$  mã này chỉ phát hiện được  $k = 1$  lỗi (single-bit error).

$\Rightarrow$  Nếu có 1 bit bị lật  $\rightarrow$  tính chẵn lẻ bị sai  $\Rightarrow$  phát hiện được error.

**Limitation:** nếu có 2 bit cùng bị lật (1  $\rightarrow$  0 và 0  $\rightarrow$  1)  $\Rightarrow$  không phát hiện được lỗi.

$\Rightarrow$  Phương pháp này đơn giản  $\rightarrow$  phù hợp cho các khối dữ liệu ngắn (short blocks of data); chẳng hạn như 7-bit ASCII characters.

$\rightarrow$  Không đáng tin cậy nếu đường truyền quá nhiễu (gây nhiều lỗi bit cùng lúc).

#### 4.3.2.4 Two-dimensional Parity-check Code

→ 2D parity-check code sẽ scan dữ liệu theo cả hàng lẫn cột, khác với 1D chỉ scan theo hàng ⇒ tạo thành matrix để bắt lỗi tốt hơn.

- **Cơ chế:** dữ liệu được sắp xếp thành một bảng (matrix).
  - Rows → với mỗi byte dữ liệu → mình tính 1 bit parity gắn vào cuối (giống y như phương pháp 1D).
  - Columns → tính cho tất cả các bit ở cột 1, 2, ... → kết quả tạo thành 1 byte đặc biệt nằm dưới cùng ⇒ gọi là parity byte.
- **Ví dụ cụ thể:** (theo hình bên phải)
  - Tính hàng ngang (như 1D ở trên).
  - Tính hàng dọc: tương tự như cách tính ở hàng ngang; ví dụ như cột đầu tiên ở mỗi hàng là 0, 1, 0, 1, 0, 0, 0 → tổng bit 1 là chẵn → bit parity cột là 0.

|             |         |             |
|-------------|---------|-------------|
|             |         | Parity bits |
|             |         | ↓           |
| Data        | 0101001 | 1           |
|             | 1101011 | 1           |
|             | 0010011 | 1           |
|             | 1010110 | 0           |
|             | 0101100 | 1           |
|             | 0101001 | 1           |
|             | 0010011 | 1           |
| Parity byte | 0010001 | 0           |

Ở loại 1D, nếu 2 bit trong cùng 1 hàng bị lỗi → tổng số bit 1 vẫn không đổi (chẵn/lẻ) ⇒ không phát hiện được lỗi.

⇒ Đối với 2D, nếu 2 bit trong cùng 1 hàng bị sai → parity hàng ngang không báo lỗi ⇒ nhưng parity hàng dọc của 2 cột chứa 2 bit sai đó sẽ bị lệch → phát hiện được lỗi.

→ **Phạm vi phát hiện:** tất cả các lỗi 1 bit, 2 bit, và 3 bit, và hầu hết các lỗi 4 bit đều được phát hiện.

#### 4.3.2.5 Cyclic Redundancy Check (CRC)

→ Là phương pháp phát hiện lỗi mạnh mẽ hơn nhiều so với parity check → được sử dụng rộng rãi (như trong Ethernet, Wifi).

- Mọi chuỗi bit đều có thể được viết dưới dạng một đa thức với hệ số là 0 hoặc 1.
- Một frame có  $k$  bit được xem như một đa thức bậc  $k - 1$ .

**Ví dụ:** chuỗi 10011010 tương ứng với đa thức:

$$\begin{aligned}
 M(x) &= 1x^7 + 0x^6 + 0x^5 + 1x^4 + 1x^3 + 0x^2 + 1x + 0 \\
 &= x^7 + x^4 + x^3 + x
 \end{aligned}$$



#### 4.3.2.6 CRC Generator Polynomial

Để sender và receiver hiểu nhau → phải thống nhất trước một công thức toán học cụ thể → là đa thức sinh  $C(x)$  (generator polynomial).

**Cách biểu diễn:** máy tính không hiểu  $x^5$  hoặc  $x^2$  → đa thức được chuyển đổi thành chuỗi bit dựa trên các hệ số.

- Nếu có  $x^n$  → bit tại vị trí  $n$  là 1.
- Nếu thiếu  $x^n$  → bit tại vị trí  $n$  là 0.

**Ví dụ:** đa thức  $C(x) = x^3 + x^2 + x^0 = 1 \cdot x^3 + 1 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0$ .  
→ Chuỗi bit tương ứng là 1101.

**Bậc của đa thức (degree -  $k$ ):**

- Bậc  $k$  là số mũ cao nhất trong đa thức → số lượng bit của chuỗi nhị phân luôn nhiều hơn bậc  $k$  một đơn vị (độ dài chuỗi là  $k + 1$ ).
- **Ví dụ:** 1101 có 4 bit → bậc  $k = 3$ .

→ Nếu dùng đa thức bậc  $k$  → phần dư (checksum) tạo ra sẽ luôn có  $k$  bit (được thêm vào sau dữ liệu).

**Representation:** với đa thức sinh bậc  $k$  → người ta thường lấy  $k - 1$  để biểu diễn; ví dụ, đa thức trên (1101) người ta sẽ lấy  $k - 1$  (tương ứng 101) để biểu diễn → thành 0x05.

#### 4.3.2.7 Selection of common Generator Polynomials

| Tên chuẩn    | Đa thức                             | Chuỗi bit (Hex/Bin) | Ứng dụng  |
|--------------|-------------------------------------|---------------------|-----------|
| CRC-5        | $x^5 + x^2 + x^0$                   | 0x05                | USB       |
| CRC-8        | $x^8 + x^7 + x^5 + x^2 + x^1 + x^0$ | 0xA7                | Bluetooth |
| CRC-16-CCITT | $x^{16} + x^{12} + x^5 + x^0$       | 0x1021              | HDLC      |
| CRC-32       | $x^{32} + \dots + x^0$              | 0x04C11DBB7         | Ethernet  |

#### 4.3.2.8 CRC Example

(Tự đọc)

#### 4.3.2.9 Properties of CRCs

##### Single bit errors (phát hiện tất cả các lỗi đơn):

- **Điều kiện:** nếu đa thức sinh  $G(x)$  có nhiều hơn một số hạng (nghĩa là có ít nhất 2 bit 1; chẳng hạn như  $x^5 + x^0$ ), và hệ số của  $x^0$  là 1 (bit cuối là 1 chứ không phải là 0).

⇒ Bất kì lỗi nào chỉ sai 1 bit duy nhất đều bắt được.

##### Double bit errors (phát hiện tất cả các lỗi kép):

- **Điều kiện:** đa thức sinh không chia hết cho  $x^k + 1 \forall k$  nhỏ hơn độ dài của khung tin tối đa.

⇒ Nếu có 2 bit bị sai nằm cách nhau một khoảng (trong giới hạn cho phép) → CRC vẫn phát hiện được.

##### Odd number of errors (phát hiện bất kì số lượng lỗi lẻ nào):

- **Điều kiện:** nếu đa thức sinh  $G(x)$  có chứa thừa số  $(x + 1)$ .

⇒ Dù có sai 3 bit, 5 bit, ... (miễn là số lượng bit sai là số lẻ) → CRC đều phát hiện được.

Self-note: đây là lý do tại sao nhiều chuẩn CRC (như CRC-16 hay CRC-32) đều có thừa số  $(x + 1)$  trong đa thức sinh của chúng.

##### Burst error (phát hiện lỗi chùm):

→ Nhiễu làm hỏng cả một dãy liên tiếp các bit.

Gọi  $r$  là bậc của đa thức sinh (chẳng hạn như CRC-16 thì  $r = 16$ ).

- **Chiều dài chùm lỗi là  $L \leq r$**  → CRC phát hiện được tất cả lỗi; ví dụ như dùng CRC-16, nếu bị nhiễu nguyên một đoạn 16 bit liên tiếp hoặc ít hơn thì nó vẫn bắt lỗi được hết.
- **Chiều dài chùm lỗi là  $L = r + 1$**  → xác suất phát hiện là  $1 - 2^{-(r-1)}$  (để cho biết chứ không cần nhớ); như CRC-16 có tỉ lệ phát hiện lỗi lên đến  $\approx 99.997\%$ .
- **Chiều dài chùm lỗi là  $L > r + 1$**  → xác suất phát hiện là  $1 - 2^{-r}$  (để cho biết chứ không cần nhớ); như CRC-16 có tỉ lệ phát hiện lỗi lên đến  $\approx 99.998\%$  với độ dài lên đến  $\geq 18$ .

→ CRC có thể được tính toán thông qua kỹ thuật dịch bit (shift register).

### 4.3.3 Error Correction

#### 4.3.3.1 Forward Error Correction (FEC)

- **Cách cũ:** error detection + retransmission → thấy lỗi, báo hỏng, rồi xin gửi lại ⇒ tốn thời gian, và độ trễ cao.
- **Với FEC:** khi thấy lỗi → dùng dữ liệu dư thừa (redundant information) để tính toán và khôi phục lại bit gốc.

**Điều kiện (dựa theo quy tắc Hamming):**

Codewords phải theo khoảng cách Hamming  $d \geq 2k + 1$ .

**Trong đó:**  $k \rightarrow$  số lượng bit lỗi tối đa mà mình muốn sửa được.

⇒ Nếu muốn sửa 1 bit lỗi ( $k = 1$ ) → khoảng cách giữa các valid codewords phải là:  $2 \cdot (1) + 1 = 3$ .

**Nguyên lý hoạt động:** người gửi sẽ chèn thêm rất nhiều redundancy information vào dữ liệu.

- Dữ liệu gốc ( $m$  bits) được mã hóa thành chuỗi dài hơn ( $n$  bits); tỉ lệ mã hóa (code rate)  $R = m/n$  nhỏ hơn 1.
- **Ví dụ:** muốn gửi 1 bit dữ liệu → có khi phải gửi kèm 2 bit sửa lỗi ⇒ gửi tổng cộng 3 bit → tốn băng thông nhưng an toàn.

**Khi nào dùng FEC?**

- High delay (độ trễ cao) → chẳng hạn như truyền vệ tin với khoảng cách xa → nếu chờ gửi lại thì tốn rất nhiều thời gian ⇒ tự sửa lỗi sẽ nhanh hơn.
- High error rates (tỉ lệ lỗi cao) → như mạng không dây → nếu cứ một xú là xin gửi lại thì sẽ gây ngẽn mạng.
- Không có kênh phản hồi (no backward channel) → như truyền hình hay đài phát thanh, người nghe chỉ nhận chứ không feedback lại cho nhà đài liền được.

**Nhược điểm:**

- Overhead (tốn băng thông) → phải gửi nhiều bit dư thừa hơn mức cần thiết.
- Phức tạp → thuật toán giải mã (decoding algorithm) để tìm và sửa lỗi thường phức tạp hơn nhiều so với việc chỉ kiểm tra (detection).

#### 4.3.3.2 Simple Example of Error Correction

Giả sử một code chỉ có 4 valid codewords:

- $w_1 = 00000 \ 00000$ .
- $w_2 = 00000 \ 11111$ .
- $w_3 = 11111 \ 00000$ .
- $w_4 = 11111 \ 11111$ .

Ta có:

- $w_1 \neq w_2 \rightarrow 5$  bit cuối.
- $w_1 \neq w_3 \rightarrow 5$  bit đầu.
- $w_1 \neq w_4 \rightarrow$  tất cả 10 bit.

$\Rightarrow$  Khoảng cách nhỏ nhất là  $5 \rightarrow d = 5$ .

- **Để detection:**  $d \geq k + 1 \rightarrow k \leq 4 \Rightarrow$  có thể phát hiện tối đa 4 bit lỗi.
- **Để correction:**  $d \geq 2k + 1 \rightarrow k \leq 2 \Rightarrow$  có thể sửa tối đa 2 bit lỗi.

**Ví dụ:**

- Nếu máy nhận được  $00000 \ 00111 \rightarrow$  nó sẽ đem so với  $w_1$  (sai 3 bit) và  $w_2$  (sai 2 bit)  $\Rightarrow$  chọn  $w_2$  làm codeword gốc  $\rightarrow$  sửa lại thành  $00000 \ 11111$ .

$\Rightarrow$  Tuy nhiên, nếu sender gửi đi  $00000 \ 00000$  nhưng bị nhiễu thành  $00000 \ 00111 \rightarrow$  máy thấy gần với  $w_2$  hơn  $\Rightarrow$  sửa thành  $00000 \ 11111 \rightarrow$  sai hoàn toàn với dữ liệu gốc.

$\rightarrow$  Với  $d = 5$ : hệ thống chỉ đảm bảo sửa đúng nếu lỗi  $\leq 2$  bit  $\Rightarrow$  nếu lỗi 3 bit trở lên, hệ thống sẽ bị lằm và sửa sai thành frame khác.